

IIT MADRAS | BS IN DATA SCIENCE

Deep Learning

A Handbook for the IIT Madras BS Data Science Programme

A structured guide to the foundations, mathematics, architectures, and intuition behind modern deep learning.

Compiled by Atharv Khare

Prepared with assistance from Claude

2026 Edition

Contents

How to Use This Book	vii
Notation Guide	viii
The Example Universe	ix
I Foundations: Neurons, Networks and Learning (Weeks 1–4)	1
1 A Short History of Deep Learning	2
1.1 From Biological Neurons to Threshold Logic	2
1.2 The Perceptron, the First Winter, and Backpropagation	2
1.3 The Deep Revival	2
2 The McCulloch–Pitts Neuron and Thresholding Logic	4
2.1 Working Principle and Intuition	4
2.2 Formal Definition	4
2.3 Implementing Boolean Functions	4
2.4 Geometry: the Decision Boundary Is a Hyperplane	5
2.5 Worked Examples	5
2.6 Failure Cases: What an MP Neuron Cannot Do	6
2.7 Why-Chain	7
2.8 Exam Toolkit	7
2.9 Practice Problems	8
3 The Perceptron and the Perceptron Learning Algorithm	9
3.1 Working Principle and Intuition	9
3.2 Formal Definition	9
3.3 Geometry: Why $\mathbf{w}$ Is the Key	9
3.4 The Perceptron Learning Algorithm	10
3.5 Worked Examples	11
3.6 Convergence of PLA	12
3.7 Failure Cases	12
3.8 Comparison: MP Neuron vs Perceptron	13
3.9 Exam Toolkit	13
3.10 Practice Problems	13
4 Networks of Perceptrons and Representation Power	15
4.1 Counting Boolean Functions	15
4.2 XOR by Composition	15
4.3 The 2^n Construction (Course Theorem)	16
4.4 Why-Chain	17

4.5	Exam Toolkit	17
4.6	Practice Problems	17
5	Sigmoid Neurons and Learning by Gradient Descent	18
5.1	Working Principle and Intuition	18
5.2	The Sigmoid Neuron	18
5.3	The Supervised Learning Setup	19
5.4	Gradient Descent: Derivation from First Principles	19
5.5	Worked Examples	20
5.6	Representation Power of Sigmoid Networks	21
5.7	Why-Chain	22
5.8	Exam Toolkit	22
5.9	Practice Problems	22
6	Feedforward Neural Networks	23
6.1	Architecture and Notation	23
6.2	Counting Parameters	23
6.3	Output Functions and Loss Functions	24
6.4	The Forward Pass	24
6.5	Linear Activations Collapse the Network	25
6.6	Representation Power of FFNNs	25
6.7	Exam Toolkit	26
6.8	Practice Problems	26
7	Backpropagation	27
7.1	Working Principle: the Chain Rule on a Graph	27
7.2	Warm-up: a Two-Neuron Chain	27
7.3	Derivation for a General Network	27
7.4	Worked Example: Backprop Through ATLAS	28
7.5	Symmetry: Why Weights Must Not Start Equal	29
7.6	Vanishing and Exploding Gradients	29
7.7	Common Failure Modes	30
7.8	Exam Toolkit	30
7.9	Practice Problems	30
8	Gradient Descent and Its Accelerations: Momentum, NAG, SGD	31
8.1	Gradient Descent on a Quadratic: an Exact Analysis	31
8.2	Momentum-Based Gradient Descent	32
8.3	Nesterov Accelerated Gradient (NAG)	33
8.4	Stochastic and Mini-Batch Gradient Descent	34
8.5	Choosing the Learning Rate in Practice	34
8.6	Comparison	35
8.7	Exam Toolkit	35
8.8	Practice Problems	35

9 Adaptive Learning Rates and Learning-Rate Schedules	37
9.1 AdaGrad	37
9.2 RMSProp	38
9.3 AdaDelta	38
9.4 Adam (Adaptive Moments)	38
9.5 AdaMax and NAdam	39
9.6 Learning-Rate Schedules	39
9.7 Comparison of Optimisers	40
9.8 Exam Toolkit	41
9.9 Practice Problems	41
10 Part I Integration: Master Problems and Rapid Revision	42
10.1 Master Integration Problems	42
10.2 Index of Course Paper Questions Solved in Part I	43
10.3 Part I Master Reference	43
II Representation, Regularisation and Better Training (Weeks 5–7)	46
Part II Road Map	47
11 Autoencoders and Their Relation to PCA	48
11.1 Working Principle and Intuition	48
11.2 Formal Definition	48
11.3 Linear Autoencoders Learn PCA	49
11.4 Regularisation in Autoencoders	50
11.5 Comparison of Autoencoder Variants	52
11.6 Exam Toolkit	52
11.7 Concept Gauntlet	52
11.8 Practice Problems	53
12 Regularisation: Bias and Variance	54
12.1 Bias and Variance	54
12.2 L_2 Regularisation (Weight Decay)	55
12.3 Dataset Augmentation	56
12.4 Parameter Sharing and Tying	56
12.5 Injecting Noise	56
12.6 Early Stopping	56
12.7 Ensemble Methods and Bagging	57
12.8 Dropout	57
12.9 Comparison of Regularisers	58
12.10 Exam Toolkit	58
12.11 Concept Gauntlet	59
12.12 Practice Problems	59

13 Training Deep Networks	60
13.1 Greedy Layer-Wise Unsupervised Pre-training	60
13.2 Better Activation Functions	60
13.3 Better Weight Initialisation	62
13.4 Batch Normalisation	63
13.5 Exam Toolkit	64
13.6 Concept Gauntlet	64
13.7 Practice Problems	64
14 Part II Integration and Revision	65
14.1 Master Integration Problems	65
14.2 Week 4 Carry-overs from Quiz 2	66
14.3 Index of Course Paper Questions Solved in Part II	66
14.4 Part II Master Reference	66
III Representations of Words and Images (Weeks 8–9)	68
15 Learning Vectorial Representations of Words	69
15.1 One-Hot Vectors and Their Limits	69
15.2 Count-Based Representations	69
15.3 Prediction-Based Models	69
15.4 GloVe	70
15.5 Walkthrough: Building a Co-occurrence Matrix by Hand	71
15.6 CBOW in Detail	71
15.7 Negative Sampling: Full Gradients	72
15.8 Hierarchical Softmax Walkthrough	72
15.9 GloVe: Derivation Sketch and Weighting	73
15.10 Evaluating Embeddings: Similarity and Analogy	73
15.11 Scenario Bank	73
15.12 Practice Problems	73
15.13 Concept Gauntlet	74
16 Convolutional Neural Networks	75
16.1 The Convolution Operation	75
16.2 Output Size and Parameters	75
16.3 Why Convolution? Sparse Connectivity and Sharing	76
16.4 Backpropagation through Convolution and Pooling	76
16.5 Landmark Architectures	76
16.6 Walkthroughs: Padding, Stride and Channels	77
16.7 Receptive Field Arithmetic	77
16.8 Pooling in Depth	78
16.9 Architecture Walkthroughs	78
16.10 Scenario Bank	79
16.11 Practice Problems	79

16.12	Concept Gauntlet	79
17	Visualising and Fooling Convolutional Networks	80
17.1	Receptive Fields and Patches	80
17.2	Visualising Filters and Occlusion	80
17.3	Gradients with Respect to the Input	80
17.4	Optimising the Input	80
17.5	Fooling CNNs: Adversarial Examples	80
17.6	Walkthrough: Three Backward Rules at a ReLU	81
17.7	Occlusion Walkthrough	81
17.8	Style Transfer in Numbers	81
17.9	Adversarial Walkthrough	82
17.10	Scenario Bank	82
17.11	Concept Gauntlet	82
IV	Sequences, Gates, Attention and Transformers (Weeks 10–12)	83
18	Recurrent Neural Networks and BPTT	84
18.1	The RNN Model	84
18.2	Backpropagation Through Time	84
18.3	Picturing the Unrolled RNN	85
18.4	BPTT Walkthrough with Numbers	85
18.5	Counting Parameters	85
18.6	Remedies in Numbers	85
18.7	Choosing the Sequence Mode	86
18.8	Scenario Bank	86
18.9	Practice Problems	86
18.10	Concept Gauntlet	86
19	Gated Recurrent Units and LSTMs	87
19.1	LSTM	87
19.2	Why LSTMs Solve Vanishing Gradients	87
19.3	GRU	87
19.4	Picturing the LSTM Cell	88
19.5	Two-Step Walkthrough	88
19.6	How Long Can Each Model Remember?	88
19.7	Designing with Gates: Scenarios	88
19.8	Practice Problems	89
19.9	Concept Gauntlet	89
20	Encoder–Decoder Models and Attention	90
20.1	Encoder–Decoder	90
20.2	Attention	90
20.3	Attention over Images and Hierarchical Attention	90
20.4	Picturing Attention	90

20.5 Scoring Functions Compared	91
20.6 Training and Decoding	91
20.7 Attention over Images: Full Walkthrough	91
20.8 Hierarchical Attention in Equations	92
20.9 Scenario Bank	92
20.10 Concept Gauntlet	92
21 Transformers	93
21.1 Scaled Dot-Product Self-Attention	93
21.2 Multi-Head Attention	93
21.3 The Transformer Block	93
21.4 Positional Encoding	94
21.5 The Architecture at a Glance	94
21.6 From Tokens to Vectors	94
21.7 Notation Bridge: Row vs Column Convention	94
21.8 Walkthrough: One Encoder Layer by Hand	95
21.9 Masked Self-Attention: Walkthrough	95
21.10 Cross-Attention and Shapes	96
21.11 Multi-Head Attention: Shape Walkthrough	96
21.12 Counting a Full Transformer	96
21.13 Why Scale by $\sqrt{d_k}$: in Numbers	96
21.14 Positional Encoding: Properties	96
21.15 Transformer vs RNN vs CNN	97
21.16 Training and Inference	97
21.17 The Transformer Family	97
21.18 Scenario Bank	97
21.19 Practice Problems	98
21.20 Concept Gauntlet	98
22 Final Master Reference (Weeks 8–12)	99
22.1 Formula Sheet	99
22.2 Recognition Rules (Weeks 8–12)	99
22.3 Index of Course Paper Questions (Parts III–IV)	100

How to Use This Book

This book has one job: to take you from a shaky first acquaintance with neural networks to the point where you can solve an unseen, multi-step examination problem calmly and quickly. It follows the course syllabus week by week, but the order inside a week follows *dependency*, not the lecture slide order. Whenever a later idea needs an earlier one, the earlier one is taught first and the connection is stated explicitly.

The narrative of every major topic. Each concept moves through the same arc: *working principle* → *intuition* → *formal definition* → *mathematics and derivation* → *algorithm* → *worked example* → *deeper example* → *failure case* → *exam perspective*. You will meet the same five “example worlds” (below) again and again; each time they return, they are one notch harder.

Box legend. Every box carries a printed label, so the book reads correctly in grayscale.

Box	What it signals
Definition	The precise, examinable statement of a concept.
Key Idea / Intuition / Insight	The mental model you should carry into the exam hall.
Important	A result you must know without hesitation.
Warning / Common Trap	Mistakes that look reasonable and cost marks.
Exam Tip	How the idea is typically tested and the fastest correct route.
Why-Chain	An examiner–student dialogue that exposes the reasoning underneath a rule.
Worked Example	Problem → Given → Required → Strategy → Solution → Verification → Interpretation → Exam Shortcut.
Course Paper Question	A question from the course’s own quiz / end-term papers, solved and linked back to the theory.
Beyond the Core Syllabus	Useful depth that is not required, clearly fenced off.

Difficulty ladder. Problems are tagged **[FOUNDATION]**, **[INTERMEDIATE]**, **[ADVANCED]**, **[EXAM]**, **[CHALLENGE]**. Inside chapters, examples climb Level 0 (intuition) through Level 5 (integrated).

Time-pressure training. Every chapter ends with an *Exam Toolkit* containing *Recognition Rules* (“if you see X, think Y”). Read those rules until they are reflexes: in a timed paper the hardest step is usually *recognising* which tool applies, not executing it.

Sources. Conventions (symbols, sign rules, update formulas) follow the course lectures and the course’s own notes and papers. Where textbooks differ from the course (for example in how the momentum update is written) both forms are shown and the course form is used in solutions.

Notation Guide

The notation below is fixed for the *entire* book, including later parts. Vectors are bold lower case, matrices are upper case, scalars are italic.

Symbol	Meaning
$\mathbf{x} = [x_1, \dots, x_n]^\top$	input vector with n features; in perceptron chapters we often prepend $x_0 = 1$ (augmented input).
$y, \hat{y}$	true target, model prediction. For classification, ℓ denotes the index of the true class.
$\mathbf{e}_\ell$	one-hot vector with 1 in position ℓ .
θ (MP / perceptron)	firing threshold. <i>Careful:</i> in optimisation chapters θ denotes <i>all parameters</i> ; context makes this unambiguous.
$w_0 = -\theta$	bias written as a weight on the constant input $x_0 = 1$.
$\mathbf{w}, b$	weight vector and bias of a single neuron.
L	number of layers of a feedforward network: $L - 1$ hidden layers and one output layer.
$W_i \in \mathbb{R}^{n_i \times n_{i-1}}, \mathbf{b}_i \in \mathbb{R}^{n_i}$	weight matrix and bias vector of layer i ; row j of W_i feeds neuron j of layer i .
$\mathbf{a}_i = \mathbf{b}_i + W_i \mathbf{h}_{i-1}$	pre-activation of layer i .
$\mathbf{h}_i = g(\mathbf{a}_i)$	activation of layer i (g applied element-wise); $\mathbf{h}_0 = \mathbf{x}$.
$\hat{\mathbf{y}} = O(\mathbf{a}_L)$	network output; O is the output function (linear or softmax).
$\sigma(z) = \frac{1}{1+e^{-z}}$	logistic sigmoid; $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
$\mathcal{L}(\theta)$	loss (objective) as a function of all parameters θ .
$\nabla_{\mathbf{w}} \mathcal{L}, \nabla w_t$	gradient of the loss w.r.t. $\mathbf{w}$; ∇w_t is shorthand for the gradient at step t .
η	learning rate (step size).
β, β_1, β_2	decay factors: momentum / RMSProp use β ; Adam uses β_1 (first moment) and β_2 (second moment).
$\mathbf{u}_t$	momentum “velocity” (history of gradients).
$\mathbf{m}_t, \mathbf{v}_t$	first- and second-moment accumulators; $\hat{\mathbf{m}}_t, \hat{\mathbf{v}}_t$ bias-corrected.
ϵ	tiny constant (e.g. 10^{-8}) preventing division by zero.
N, B	number of training examples, mini-batch size.
$\odot$	element-wise (Hadamard) product.
$\ \cdot\ $	Euclidean norm.
$\mathbf{1}[P]$	indicator: 1 if statement P is true, else 0.

Warning: Two meanings of θ

The course uses θ for the MP/perceptron threshold in Week 1 and for “all trainable parameters” from Week 2 onward ($\mathcal{L}(\theta)$). We keep both uses because the exam papers do; just read the chapter context.

The Example Universe

Rather than disconnected toy problems, this book uses five recurring *worlds*. Each world is introduced simply and returns later with more variables, uncertainty and constraints. Recognising the same world at a harder level trains the skill examiners actually test: transferring a known structure to an unfamiliar surface.

World	Story	Concepts it carries
SENTINEL	An automated greenhouse. Binary sensors (soil dry, daylight, tank level, rain) decide whether to open the irrigation valve.	MP neuron, thresholds, inhibition, Boolean functions, XOR, networks of perceptrons.
LUMEN	A Mars rover scoring rocks by two spectrometer features and deciding <i>sample</i> (+1) or <i>skip</i> (−1).	Perceptron, geometry of $\mathbf{w}$, PLA, convergence, margins.
PULSE	A hospital triage scorer mapping a vital-sign index to a risk probability.	Sigmoid neuron, squared-error loss, gradient descent, chain rule, vanishing gradients.
ATLAS	A parcel hub routing packages into 3 lanes (express / standard / freight) from weight and distance.	Feedforward networks, softmax, cross-entropy, forward pass, back-propagation, parameter counts.
ODYSSEY	A spacecraft tuning thruster trims by minimising a fuel-error bowl that is steep in one direction and flat in another.	GD stability, momentum, NAG, SGD/mini-batches, AdaGrad, RMSProp, AdaDelta, Adam, schedules.

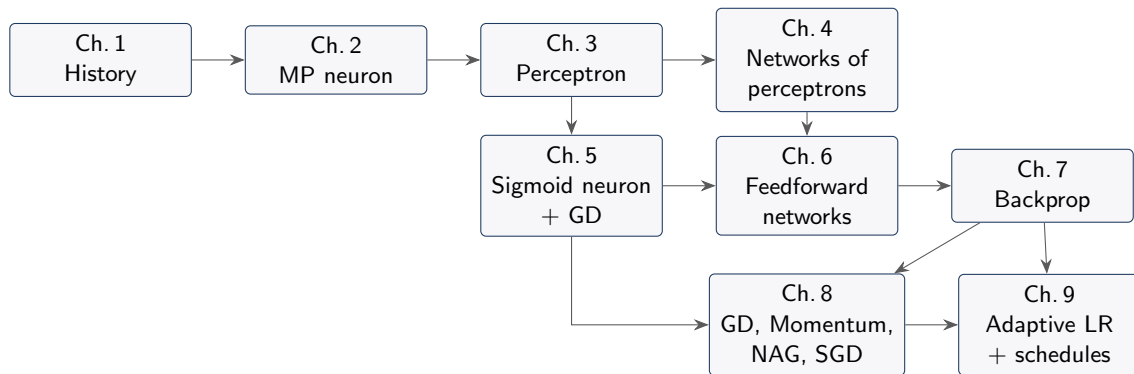


Figure 1: Dependency map of Part I. Arrows mean “is needed to understand”. Chapter 10 integrates all of them.

Part I

Foundations: Neurons, Networks and Learning (Weeks 1–4)

CHAPTER 1

A Short History of Deep Learning

Chapter Overview

The history is examined lightly (usually as one-mark ordering or attribution questions), but it also explains *why* the technical ideas of Weeks 1–4 appear in the order they do: each wave of progress fixed a specific limitation of the previous one.

Learning objectives. After this chapter you should be able to (i) place the key milestones in order; (ii) state which limitation each milestone addressed; (iii) connect the history to the chapters that follow.

1.1 From Biological Neurons to Threshold Logic

Biological neurons receive signals through *dendrites*, integrate them in the *soma*, and “fire” along the *axon* when the integrated signal is strong enough. The late-19th-century neuron doctrine (Cajal) established that the nervous system is made of discrete cells. In **1943**, **McCulloch and Pitts** abstracted this into a mathematical unit: binary inputs, a sum, and a threshold (Chapter 2). This is the first artificial neuron, and it could implement logic gates, but it could not *learn*.

1.2 The Perceptron, the First Winter, and Backpropagation

- **1957–58, Rosenblatt’s perceptron:** numeric weights that are *learned* from data by a simple error-correcting rule (Chapter 3).
- **1969, Minsky and Papert, *Perceptrons*:** a single perceptron cannot represent non-linearly-separable functions such as XOR. Although multilayer networks *can* (Chapter 4), there was no known way to train them, and funding collapsed: the first “AI winter”.
- **1986, Rumelhart, Hinton and Williams** popularised **backpropagation** (the idea itself appeared earlier, e.g. Werbos 1974), making multilayer networks trainable by gradient descent (Chapters 5–7).
- **1989:** the **universal approximation** results (Cybenko; Hornik et al.) showed that one hidden layer of sigmoid units can approximate any continuous function arbitrarily well (Section 5.6).

1.3 The Deep Revival

Deep networks were still hard to train (vanishing gradients, Section 7.6). The revival came from three sources: better training tricks (unsupervised layer-wise pre-training, 2006; better activations and initialisation; Part II), **data** (ImageNet, 2009), and **compute** (GPUs). **AlexNet (2012)** won the ImageNet challenge by a large margin and triggered the modern era: deeper CNNs (Part III), sequence models and attention, and Transformers (2017; Part IV). In parallel, *optimisation* improved: momentum (Polyak 1964), Nesterov (1983), AdaGrad (2011), RMSProp (2012), Adam (2014) — the subject of Chapters 8 and 9.

Year	Milestone	Limitation it fixed / created
1943	MP neuron	First formal neuron; no learning, unit weights.
1958	Perceptron + learning rule	Learns weights; only linearly separable functions.
1969	Minsky–Papert critique	Exposed XOR limitation; triggered AI winter.
1986	Backpropagation popularised	Multilayer networks become trainable.
1989	Universal approximation	One hidden layer suffices in principle.
2006	Layer-wise pre-training	Made deep nets trainable again (Part II).
2012	AlexNet	GPUs + data + ReLU: deep learning goes mainstream.
2011–14	AdaGrad, RMSProp, Adam	Adaptive per-parameter learning rates.
2017	Transformer	Attention-only sequence models (Part IV).

Quick Recall

Threshold logic (1943) → learning weights (1958) → XOR wall (1969) → backprop makes depth trainable (1986) → universal approximation (1989) → data + GPUs + tricks (2006–2012) → better optimisers (2011–2014). Every chapter in Part I corresponds to one arrow.

CHAPTER 2

The McCulloch–Pitts Neuron and Thresholding Logic

Chapter Overview

The McCulloch–Pitts (MP) neuron is the simplest possible model of a neuron: add up binary inputs and compare with a threshold. It introduces three ideas that will survive into every later model: (1) a neuron *aggregates* its inputs, (2) it *decides* by comparing the aggregate with a threshold, and (3) its decision boundary is a *hyperplane*. Its limitations (no learning, equal weights, only linearly separable functions) motivate everything in Chapters 3 and 4.

Learning objectives. Compute MP outputs including inhibitory inputs; choose thresholds for AND, OR, NOR, NOT and “at least k of n ”; interpret the neuron geometrically; prove that XOR is impossible; decide exam true/false statements about MP neurons.

Prerequisites. Boolean logic (AND, OR, NOT), equation of a line/plane.

2.1 Working Principle and Intuition

Imagine the SENTINEL greenhouse controller. Three sensors each report 1 (“yes”) or 0 (“no”): soil is dry (x_1), daylight is high (x_2), water tank is adequate (x_3). A fourth sensor reports rain (x_4). The rule the gardener wants is: *open the valve if at least two of the three conditions hold, but never when it is raining*.

The MP neuron implements exactly this kind of rule: count the “yes” votes, compare with a quota (the threshold), and let certain inputs act as an absolute veto (inhibitory inputs).

2.2 Formal Definition

Definition: McCulloch–Pitts neuron

Inputs $x_1, \dots, x_n \in \{0, 1\}$ are split into *excitatory* and *inhibitory* inputs. The neuron computes the aggregate

$$g(\mathbf{x}) = \sum_{i=1}^n x_i \quad (\text{over excitatory inputs}),$$

and outputs

$$y = f(g(\mathbf{x})) = \begin{cases} 1 & \text{if } g(\mathbf{x}) \geq \theta \text{ and every inhibitory input is 0,} \\ 0 & \text{otherwise.} \end{cases}$$

The threshold θ is fixed by hand; nothing is learned. If *any* inhibitory input is 1, the output is 0 regardless of the others (*absolute inhibition*).

2.3 Implementing Boolean Functions

Because inputs are binary and weights are all 1, $g(\mathbf{x})$ simply *counts* active excitatory inputs. So an MP neuron always answers the question “are at least θ of my excitatory inputs on (and no inhibitor on)?” Every thresholded function is of this “quota” form.

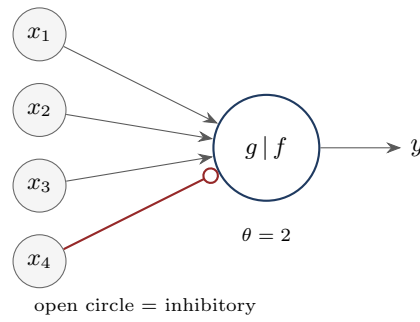


Figure 2.1: The SENTINEL valve neuron: three excitatory sensors, one inhibitory rain sensor, threshold 2.

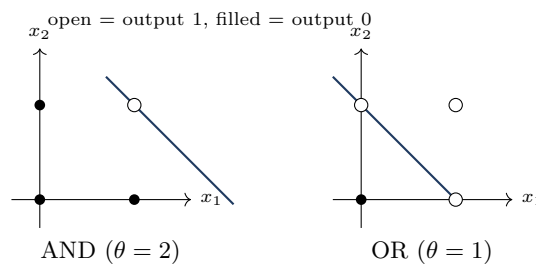
Function (n inputs)	Setting	Reason
AND	$\theta = n$	fires only if all n inputs are 1.
OR	$\theta = 1$	fires if at least one input is 1.
at least k of n	$\theta = k$	the quota itself.
NOR	all inputs inhibitory, $\theta = 0$	$g = 0 \geq 0$ always, so it fires unless some input (inhibitor) is 1.
NOT x_1	x_1 inhibitory, $\theta = 0$	special case of NOR with $n = 1$.
$x_1 \wedge \neg x_2$	x_1 excitatory, x_2 inhibitory, $\theta = 1$	needs $x_1 = 1$ and vetoed by x_2 .
Tautology (always 1)	$\theta = 0$, no inhibitors	$g \geq 0$ always.

2.4 Geometry: the Decision Boundary Is a Hyperplane

For $n = 2$ excitatory inputs, the neuron fires when $x_1 + x_2 \geq \theta$. The equation $x_1 + x_2 = \theta$ is a *line*; points on or above it output 1. In n dimensions, $\sum_i x_i = \theta$ is a *hyperplane*. So:

Important: Linear separability

An MP neuron (without inhibition) partitions the Boolean cube $\{0,1\}^n$ by the hyperplane $\sum_i x_i = \theta$. Every function it computes is **linearly separable**: its 1-points and 0-points can be separated by a hyperplane.

Figure 2.2: AND and OR as lines $x_1 + x_2 = \theta$ over the Boolean square.

2.5 Worked Examples

Worked Example 2.1: Sentinel valve, Level 1

Problem. For the neuron of Figure 2.1 (excitatory x_1, x_2, x_3 , inhibitory x_4 , $\theta = 2$), find the output for $(1, 0, 1, 0)$, $(1, 1, 1, 1)$ and $(0, 0, 1, 0)$, and count how many of the 16 inputs open the valve.

Given. $g = x_1 + x_2 + x_3$; fire iff $g \geq 2$ and $x_4 = 0$.

Strategy. Check the veto first (it overrides everything), then the quota.

Solution. $(1, 0, 1, 0)$: no rain, $g = 2 \geq 2 \Rightarrow y = 1$. $(1, 1, 1, 1)$: rain $\Rightarrow y = 0$ even though $g = 3$. $(0, 0, 1, 0)$: $g = 1 < 2 \Rightarrow y = 0$. Counting: need $x_4 = 0$ and at least 2 of 3 excitatory on: $\binom{3}{2} + \binom{3}{3} = 3 + 1 = 4$ inputs out of 16.

Verification. Listing $x_4 = 0$ rows with $g \geq 2$: 110, 101, 011, 111 — four. ✓

Interpretation. The inhibitory input is a hard safety interlock: no amount of excitation overrides it.

Exam Shortcut. “Fires” count = $\sum_{k \geq \theta} \binom{n_{\text{exc}}}{k}$ (with all inhibitors off).

Course Paper Question: Quiz 1, MP neuron with inhibition

Inputs x_1, x_2 excitatory, x_3 inhibitory, $\theta = 2$. Which are correct? (a) $(1, 1, 0) \mapsto 1$; (b) $(1, 1, 1) \mapsto 1$; (c) $(1, 0, 0) \mapsto 0$; (d) raising θ to 3 changes no output.

Solution. (a) $g = 2 \geq 2$, no inhibition: **true**. (b) $x_3 = 1$ vetoes: output 0, **false**. (c) $g = 1 < 2$: **true**. (d) With $\theta = 3$, $(1, 1, 0)$ has $g = 2 < 3$ and switches from 1 to 0: **false**. (In fact with only two excitatory inputs, $\theta = 3$ makes the neuron *never* fire.)

Principle used. Veto first, quota second; a threshold above the number of excitatory inputs gives the constant-0 function.

Worked Example 2.2: How many functions can one MP neuron realise? Level 3

Problem. A 3-input MP neuron has no inhibitory inputs. As θ ranges over all integers, how many *distinct* Boolean functions can it compute?

Strategy. The output depends on θ only through “how many inputs must be on”. Group thresholds that behave identically.

Solution. $g \in \{0, 1, 2, 3\}$. $\theta \leq 0$: always 1. $\theta = 1$: OR. $\theta = 2$: majority. $\theta = 3$: AND. $\theta \geq 4$: always 0. Distinct functions: $5 = n + 2$.

Verification. Out of $2^3 = 256$ Boolean functions of 3 inputs, one fixed wiring of an MP neuron reaches only 5. Even allowing each input to be excitatory or inhibitory adds only a few more.

Interpretation. Unit weights make the MP neuron *symmetric*: it cannot tell which inputs are on, only how many. That is the root of its weakness.

2.6 Failure Cases: What an MP Neuron Cannot Do

2.6.1 XOR

Proposition 2.1. No MP neuron computes $XOR(x_1, x_2)$.

Proof. Neither input can be inhibitory: $(1, 0) \mapsto 1$ rules out x_1 inhibitory and $(0, 1) \mapsto 1$ rules out x_2 . With both excitatory, $(1, 0) \mapsto 1$ requires $1 \geq \theta$, while $(1, 1) \mapsto 0$ requires $2 < \theta$. Both cannot hold. □

Geometrically, XOR’s 1-points $(1, 0), (0, 1)$ sit on one diagonal of the square and its 0-points $(0, 0), (1, 1)$ on the other; no single line separates the diagonals.

2.6.2 A linearly separable function that an MP neuron still cannot compute

This subtle case explains why the perceptron needed *weights*.

Worked Example 2.3: The weighted-vote gap, Level 4

Problem. Show that $f = x_1 \vee (x_2 \wedge x_3)$ is linearly separable but no single MP neuron computes it.

Solution. *Separable:* $2x_1 + x_2 + x_3 \geq 2$ works: $x_1 = 1$ gives at least 2; with $x_1 = 0$ we need $x_2 = x_3 = 1$. ✓

Not MP: $f(1, 0, 0) = 1$ forbids x_1 from being inhibitory (it is on and the output is 1), and $f(0, 1, 1) = 1$ forbids x_2 and x_3 from being inhibitory. So all are excitatory. Then $f(1, 0, 0) = 1$ needs $\theta \leq 1$, but $f(0, 1, 0) = 0$ needs $\theta > 1$. Contradiction.

Interpretation. The function needs x_1 to count *double*. Unit weights cannot express unequal importance. Rosenblatt’s weights fix exactly this.

Course Paper Question: Quiz 1, two statements about MP neurons

Statement I: a single MP neuron can represent *all* linearly separable Boolean functions. Statement II: every linearly separable Boolean function can be represented using at least one MP neuron. **Key: only II is true.**

Reasoning. Statement I is false for the plain reason that one neuron with a fixed threshold computes *one* function, not all of them. Statement II is the course’s intended “yes, MP neurons can implement linearly separable functions” reading, where “at least one MP neuron” allows several neurons to be combined.

Beyond the Core Syllabus: A careful reading of Statement II

Read literally with *one* neuron and unit weights, the previous worked example is a counterexample. With *several* MP neurons combined in a network (e.g. an AND neuron feeding an OR neuron: $x_1 \vee (x_2 \wedge x_3)$), every Boolean function becomes representable, which is why the key marks II as true. In the exam, choose the key’s answer, but know why the phrase “at least one” matters.

Course Paper Question: Quiz 1, true/false

“The decision boundary learned by a neural network is always non-linear.” **False.** A single neuron (MP, perceptron or sigmoid) has a linear (hyperplane) boundary, and a network with only linear activations is equivalent to a linear model (Section 6.5). Non-linearity is a *capability*, not a guarantee.

2.7 Why-Chain

Why-Chain: Why is the MP neuron a linear classifier?

Examiner: Why can’t the MP neuron compute XOR?

Student: Because its firing condition $\sum x_i \geq \theta$ is a half-space: the boundary is a hyperplane, and XOR’s classes are not separable by any hyperplane.

Examiner: But inhibitory inputs are a non-linear “veto”. Doesn’t that escape the hyperplane?

Student: Only partially. Inhibition carves out the face $x_j = 0$ of the cube, which is itself a half-space; intersecting half-spaces gives a convex region. XOR’s 1-points are not a convex subset of the cube’s vertices separated from its 0-points, so vetoes don’t help.

Examiner: So what is the deeper limitation?

Student: Two limitations: *geometric* (one hyperplane) and *expressive* (equal weights). The perceptron removes the second, and networks remove the first.

2.8 Exam Toolkit

Must know: MP definition with absolute inhibition; thresholds for AND (n), OR (1), NOR/NOT (inhibitory, $\theta = 0$). **Must understand:** the boundary $\sum x_i = \theta$ is a hyperplane; unit weights $\Rightarrow$ only “count” matters. **Must practise:** output tables with inhibitors; “what changes if θ changes”. **Numerical pattern:** count firing inputs with $\sum_{k \geq \theta} \binom{n}{k}$.

Exam Tip: Recognition rules

- See an inhibitory input set to 1 $\Rightarrow$ output is 0, stop computing.
- See “ θ larger than number of excitatory inputs” $\Rightarrow$ constant-0 function.
- See XOR / XNOR / “exactly one” / parity $\Rightarrow$ not linearly separable; a single neuron cannot do it.
- See “one neuron represents all functions” $\Rightarrow$ false (one neuron = one function).

Quick Recall: MP neuron

$y = \mathbf{1}[\sum_{\text{exc}} x_i \geq \theta] \cdot \prod_{\text{inh}} (1 - x_j)$. No learning, unit weights, binary inputs. Boundary: hyperplane. Computes only linearly separable (and in fact only “quota”) functions. Cannot do XOR.

2.9 Practice Problems

1. **[FOUNDATION]** Design an MP neuron for “ $x_1 \wedge x_2 \wedge \neg x_3$ ”.
Answer: x_1, x_2 excitatory, x_3 inhibitory, $\theta = 2$.
2. **[INTERMEDIATE]** A 4-input MP neuron (no inhibition) fires on exactly 11 of the 16 inputs. Find θ .
Answer: $\binom{4}{2} + \binom{4}{3} + \binom{4}{4} = 6 + 4 + 1 = 11 \Rightarrow \theta = 2$.
3. **[EXAM]** True or false: “Changing a single input from excitatory to inhibitory can never increase the number of inputs on which an MP neuron fires.”
Answer: True: inhibition only adds a veto and removes that input from the count, and removing a vote can only make the quota harder to reach.
4. **[ADVANCED]** Show that the 3-input majority function is computable by an MP neuron but “exactly two of three” is not.
Hint: Exactly-two fires on $g = 2$ but not $g = 3$; a threshold rule is monotone in g .

CHAPTER 3

The Perceptron and the Perceptron Learning Algorithm

Chapter Overview

The perceptron keeps the MP neuron’s “sum and threshold” decision but adds two things: **real-valued weights** (inputs can matter unequally, and inputs themselves can be real) and a **learning rule** that finds the weights from labelled data. This chapter builds the geometric picture that makes the learning rule obvious, runs the algorithm by hand, and proves that it always terminates on linearly separable data.

Learning objectives. Write the perceptron with the bias trick; explain the role of $\mathbf{w}$ as the normal to the boundary; execute the Perceptron Learning Algorithm (PLA) by hand, including tie cases; compute angles before/after updates; state and prove the convergence theorem; recognise when PLA can never converge.

Prerequisites. Chapter 2; dot product $\mathbf{w}^\top \mathbf{x} = \|\mathbf{w}\| \|\mathbf{x}\| \cos \alpha$.

3.1 Working Principle and Intuition

The LUMEN rover measures two features of each rock: an iron index x_1 and a porosity index x_2 . Scientists have labelled some rocks *sample* (+1) and others *skip* (−1). The rover must find a straight line in the (x_1, x_2) plane with all *sample* rocks on one side. The MP neuron could only draw lines like $x_1 + x_2 = \theta$ (slope −1); the perceptron can draw *any* line $w_1 x_1 + w_2 x_2 = \theta$, and it can *discover* the line by nudging it every time it misclassifies a rock.

3.2 Formal Definition

Definition: Perceptron

Given real inputs $x_1, \dots, x_n$, weights $w_1, \dots, w_n$ and threshold θ ,

$$y = \begin{cases} 1 & \text{if } \sum_{i=1}^n w_i x_i \geq \theta, \\ 0 & \text{otherwise.} \end{cases}$$

Bias trick. Move θ to the left: set $x_0 = 1$ and $w_0 = -\theta$. Then

$$y = \mathbf{1} \left[\sum_{i=0}^n w_i x_i \geq 0 \right] = \mathbf{1} [\mathbf{w}^\top \mathbf{x} \geq 0], \quad \mathbf{w} = [w_0, \dots, w_n]^\top, \quad \mathbf{x} = [1, x_1, \dots, x_n]^\top.$$

w_0 is called the *bias*: it encodes the neuron’s prior tendency (“prejudice”) to fire before seeing any evidence.

Many papers instead keep the bias separate and use labels ± 1 : predict +1 if $\mathbf{w}^\top \mathbf{x} + b \geq 0$, else −1. Both are the same model; exam questions use both, so be fluent in each.

3.3 Geometry: Why $\mathbf{w}$ Is the Key

The boundary is $\mathbf{w}^\top \mathbf{x} = 0$. For any point $\mathbf{x}$ on the boundary, $\cos \alpha = \frac{\mathbf{w}^\top \mathbf{x}}{\|\mathbf{w}\| \|\mathbf{x}\|} = 0$, so $\alpha = 90^\circ$: **$\mathbf{w}$ is perpendicular to the boundary**. Points with $\mathbf{w}^\top \mathbf{x} > 0$ make an *acute* angle with $\mathbf{w}$ (they lie on $\mathbf{w}$ ’s side); points with $\mathbf{w}^\top \mathbf{x} < 0$ make an *obtuse* angle.

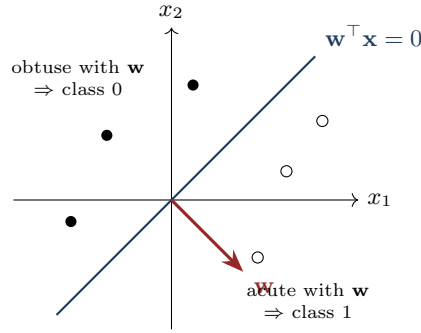


Figure 3.1: The weight vector is normal to the decision boundary and points into the positive half-space.

3.4 The Perceptron Learning Algorithm

3.4.1 Core idea

If a positive point $\mathbf{x}$ is misclassified, then $\mathbf{w}^\top \mathbf{x} < 0$: the angle between $\mathbf{w}$ and $\mathbf{x}$ is obtuse. We want to *rotate* $\mathbf{w}$ towards $\mathbf{x}$. The simplest rotation is $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{x}$. Symmetrically, for a misclassified negative point we rotate $\mathbf{w}$ away: $\mathbf{w} \leftarrow \mathbf{w} - \mathbf{x}$.

3.4.2 Why the update helps (derivation)

For a misclassified positive $\mathbf{x}$:

$$\mathbf{w}_{\text{new}}^\top \mathbf{x} = (\mathbf{w} + \mathbf{x})^\top \mathbf{x} = \mathbf{w}^\top \mathbf{x} + \|\mathbf{x}\|^2 > \mathbf{w}^\top \mathbf{x}.$$

Since $\cos \alpha_{\text{new}} \propto \mathbf{w}_{\text{new}}^\top \mathbf{x}$ (with a positive proportionality factor), the score strictly increases by $\|\mathbf{x}\|^2$, i.e. $\mathbf{w}$ turns towards $\mathbf{x}$. For a negative $\mathbf{x}$: $(\mathbf{w} - \mathbf{x})^\top \mathbf{x} = \mathbf{w}^\top \mathbf{x} - \|\mathbf{x}\|^2$, strictly decreasing. One update may not fix the point completely; repeated visits will.

Algorithm 1 Perceptron Learning Algorithm (course form)

```

1:  $P \leftarrow$  inputs with label 1;  $N \leftarrow$  inputs with label 0 ▷ inputs augmented with  $x_0 = 1$ 
2: initialise  $\mathbf{w}$  (randomly, or  $\mathbf{0}$ )
3: while not converged do
4:   pick  $\mathbf{x} \in P \cup N$  ▷ randomly or cyclically
5:   if  $\mathbf{x} \in P$  and  $\mathbf{w}^\top \mathbf{x} < 0$  then
6:      $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{x}$ 
7:   end if
8:   if  $\mathbf{x} \in N$  and  $\mathbf{w}^\top \mathbf{x} \geq 0$  then
9:      $\mathbf{w} \leftarrow \mathbf{w} - \mathbf{x}$ 
10:  end if
11: end while ▷ converged = a full pass with no update

```

Equivalent ± 1 form. With $y \in \{+1, -1\}$: if $\mathbf{x}$ is misclassified, $\mathbf{w} \leftarrow \mathbf{w} + y\mathbf{x}$ (and $b \leftarrow b + y$ if the bias is kept separate). With a learning rate, $\mathbf{w} \leftarrow \mathbf{w} + \eta y\mathbf{x}$; for PLA started at $\mathbf{0}$, η only rescales $\mathbf{w}$ and does not change the predictions.

Common Trap: The tie at exactly zero

The rule is “predict positive if $\mathbf{w}^\top \mathbf{x} \geq 0$ ”. A score of exactly 0 is a *positive* prediction. So a positive point with score 0 is *correct* (no update), but a negative point with score 0 is *misclassified* (update). Starting from $\mathbf{w} = \mathbf{0}$, the first negative point visited is always an update; the first positive point never is. Many hand-run answers go wrong right here.

3.5 Worked Examples

Worked Example 3.1: Lumen learns a rock boundary, Level 2

Problem. Run PLA from $\mathbf{w} = \mathbf{0}$ (augmented, $\mathbf{w} = [w_0, w_1, w_2]$) visiting the rocks cyclically in the order R1, R2, R3, R4 until a full pass has no mistakes. Report the number of updates and the final boundary.

Rock	(x_1, x_2)	label	augmented $\mathbf{x}$
R1	(2, 1)	+1	[1, 2, 1]
R2	(0, 2)	-1	[1, 0, 2]
R3	(3, 3)	+1	[1, 3, 3]
R4	(1, 3)	-1	[1, 1, 3]

Strategy. For each visit: compute score $s = \mathbf{w}^\top \mathbf{x}$; predict +1 iff $s \geq 0$; if wrong, add $y\mathbf{x}$.

Solution.

Epoch	Rock	$\mathbf{w}$ before	score s	pred	action / new $\mathbf{w}$
1	R1	[0, 0, 0]	0	+1	correct
1	R2	[0, 0, 0]	0	+1	wrong $\Rightarrow$ [-1, 0, -2]
1	R3	[-1, 0, -2]	-7	-1	wrong $\Rightarrow$ [0, 3, 1]
1	R4	[0, 3, 1]	6	+1	wrong $\Rightarrow$ [-1, 2, -2]
2	R1	[-1, 2, -2]	1	+1	correct
2	R2	[-1, 2, -2]	-5	-1	correct
2	R3	[-1, 2, -2]	-1	-1	wrong $\Rightarrow$ [0, 5, 1]
2	R4	[0, 5, 1]	8	+1	wrong $\Rightarrow$ [-1, 4, -2]
3	all	[-1, 4, -2]	5, -5, 5, -3		all correct: stop

Total updates: 5. Final boundary: $-1 + 4x_1 - 2x_2 = 0$, i.e. $x_2 = 2x_1 - \frac{1}{2}$.

Verification. R1: $-1 + 8 - 2 = 5 > 0$ ✓; R2: $-1 + 0 - 4 = -5 < 0$ ✓; R3: $-1 + 12 - 6 = 5$ ✓; R4: $-1 + 4 - 6 = -3$ ✓.

Interpretation. Rocks *below* the line $x_2 = 2x_1 - \frac{1}{2}$ (high iron relative to porosity) are sampled. Notice R3 was “fixed” in epoch 1 and then broken again by the R4 update — updates for one point can undo another. Convergence is guaranteed overall, not point by point.

Exam Shortcut. Track only the score column; every update changes the score of the *same* point by $\pm \|\mathbf{x}\|^2$ (e.g. R3: $-7 + 19 = 12$ if re-evaluated immediately).

Course Paper Question: Quiz 1, angle after an update

$\mathbf{w} = [3, 4]^\top$, misclassified $\mathbf{x} = [4, 3]^\top$ with $y = -1$, update $\mathbf{w}_{\text{new}} = \mathbf{w} - \mathbf{x}$. Find the angle between $\mathbf{w}_{\text{new}}$ and $\mathbf{x}$.

Solution. $\mathbf{w}_{\text{new}} = [-1, 1]^\top$. $\mathbf{w}_{\text{new}}^\top \mathbf{x} = -4 + 3 = -1$. $\|\mathbf{w}_{\text{new}}\| = \sqrt{2}$, $\|\mathbf{x}\| = 5$. So $\theta_{\text{new}} = \cos^{-1}\left(\frac{-1}{5\sqrt{2}}\right)$ (just over 90°).

Check with the theory. Before: $\mathbf{w}^\top \mathbf{x} = 24$ (angle acute, wrong for a negative point). The update subtracts $\|\mathbf{x}\|^2 = 25$: $24 - 25 = -1$ ✓. The point is now (barely) on the correct side.

Course Paper Question: Quiz 1, separate bias, four points

$\mathbf{w} = [2, -1]^\top$, $b = 0$, predict +1 iff $\mathbf{w}^\top \mathbf{x} + b \geq 0$; update $\mathbf{w} \leftarrow \mathbf{w} + y\mathbf{x}$, $b \leftarrow b + y$. Points A(3, 1, +1), B(2, 0, -1), C(1, 2, -1), D(4, 4, +1) once in order.

Solution. A: $6 - 1 = 5 \geq 0 \Rightarrow +1$ correct. B: $4 \geq 0 \Rightarrow +1$, wrong: $\mathbf{w} = [0, -1]$, $b = -1$. C: $0 - 2 - 1 = -3 < 0 \Rightarrow -1$ correct. D: $0 - 4 - 1 = -5 < 0 \Rightarrow -1$, wrong: $\mathbf{w} = [4, 3]$, $b = 0$.

Answer. First update after B ✓; second after D (not C); final bias 0 (not -1); exactly two updates ✓.

3.6 Convergence of PLA

3.6.1 Statement

Definition: Linear separability

Finite sets $P, N \subset \mathbb{R}^n$ are (absolutely) linearly separable if there exist $w_0, \dots, w_n$ with $\sum_{i \geq 1} w_i x_i \geq w_0$ for all $\mathbf{x} \in P$ and $\sum_{i \geq 1} w_i x_i < w_0$ for all $\mathbf{x} \in N$.

Theorem 3.1 (Perceptron convergence). *If P and N are finite and linearly separable, PLA makes only a finite number of updates, whatever order the points are visited in; after the last update, $\mathbf{w}$ separates P from N .*

3.6.2 Proof (course version)

Step 1: one set instead of two. If $\mathbf{x} \in N$, then “ $\mathbf{w}^\top \mathbf{x} < 0$ is wanted” is the same as “ $\mathbf{w}^\top (-\mathbf{x}) > 0$ is wanted”. Replace every negative point by its negation; call the combined set P' . Now PLA is: if $\mathbf{w}^\top \mathbf{p} < 0$ for some $\mathbf{p} \in P'$, set $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{p}$.

Step 2: normalise. Scale every $\mathbf{p}$ to $\|\mathbf{p}\| = 1$ (this does not change the sign of $\mathbf{w}^\top \mathbf{p}$). Let $\mathbf{w}^*$ be a *unit* separating vector and define the margin-like quantity $\delta = \min_{\mathbf{p} \in P'} \mathbf{w}^{*\top} \mathbf{p} > 0$.

Step 3: the numerator grows linearly. Suppose k corrections have been made, starting from $\mathbf{w}_0$. Each correction adds some $\mathbf{p}$ with $\mathbf{w}^{*\top} \mathbf{p} \geq \delta$:

$$\mathbf{w}^{*\top} \mathbf{w}_k = \mathbf{w}^{*\top} \mathbf{w}_{k-1} + \mathbf{w}^{*\top} \mathbf{p} \geq \mathbf{w}^{*\top} \mathbf{w}_{k-1} + \delta \geq \dots \geq \mathbf{w}^{*\top} \mathbf{w}_0 + k\delta.$$

Step 4: the norm grows only like $\sqrt{k}$. A correction happens only when $\mathbf{w}_{k-1}^\top \mathbf{p} < 0$, so

$$\|\mathbf{w}_k\|^2 = \|\mathbf{w}_{k-1}\|^2 + 2\mathbf{w}_{k-1}^\top \mathbf{p} + \|\mathbf{p}\|^2 \leq \|\mathbf{w}_{k-1}\|^2 + 1 \leq \dots \leq \|\mathbf{w}_0\|^2 + k.$$

Step 5: combine. With β the angle between $\mathbf{w}^*$ and $\mathbf{w}_k$,

$$\cos \beta = \frac{\mathbf{w}^{*\top} \mathbf{w}_k}{\|\mathbf{w}_k\|} \geq \frac{\mathbf{w}^{*\top} \mathbf{w}_0 + k\delta}{\sqrt{\|\mathbf{w}_0\|^2 + k}}.$$

The right side grows like $\sqrt{k}\delta$. Since $\cos \beta \leq 1$, k cannot grow without bound. With $\mathbf{w}_0 = \mathbf{0}$ this gives $\sqrt{k}\delta \leq 1$, i.e.

$$k \leq 1/\delta^2.$$

□

Intuition: What the proof really says

Every mistake pushes $\mathbf{w}$ a fixed amount *towards* the true direction $\mathbf{w}^*$ (linear growth), but can only make $\mathbf{w}$ longer by a bounded amount (square-root growth). Alignment therefore keeps improving, and alignment cannot exceed perfect. Small δ (points nearly on the boundary) means potentially many mistakes.

Beyond the Core Syllabus: Novikoff's bound with unnormalised data

If $\|\mathbf{x}_i\| \leq R$ for all i and some unit $\mathbf{w}^*$ satisfies $y_i \mathbf{w}^{*\top} \mathbf{x}_i \geq \gamma > 0$, then PLA from $\mathbf{0}$ makes at most $(R/\gamma)^2$ mistakes. The course proof is the special case $R = 1$, $\gamma = \delta$. The bound is independent of the number of points and of the dimension.

3.7 Failure Cases

- **Non-separable data** (e.g. XOR, or any data with label noise): Theorem 3.1 does not apply. PLA cycles forever; the weights keep changing. In practice this is how non-separability is “detected”: no pass is ever mistake-free.
- **Non-uniqueness:** many separating lines exist; which one PLA returns depends on initialisation and visiting order. PLA does *not* seek the “best” (maximum-margin) line.
- **Hard decisions:** the step function is flat almost everywhere, so its derivative is 0 (or undefined at the jump). We cannot measure “how wrong” a prediction is, so gradient-based learning is impossible. This limitation is removed by the sigmoid neuron (Chapter 5).

3.8 Comparison: MP Neuron vs Perceptron

Property	MP neuron	Perceptron
Inputs	binary	real
Weights	all 1 (plus absolute inhibition)	real, learned
Threshold	hand-set θ	learned as bias $w_0 = -\theta$
Learning	none	PLA (error-driven)
Boundary	hyperplane $\sum x_i = \theta$	any hyperplane $\mathbf{w}^\top \mathbf{x} = 0$
Can do $x_1 \vee (x_2 \wedge x_3)$?	no	yes
Can do XOR?	no	no

Why-Chain: Why does PLA add the input vector itself?

Examiner: Why $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{x}$ and not some other correction?

Student: Because the score is $\mathbf{w}^\top \mathbf{x}$, and adding $\mathbf{x}$ changes it by exactly $\|\mathbf{x}\|^2 > 0$ — the largest increase per unit of change in the direction that matters.

Examiner: Couldn't the update ruin other points?

Student: Yes, it can (R3 in the LUMEN run). PLA is not a per-point guarantee; the convergence proof tracks a global quantity, the alignment with $\mathbf{w}^*$.

Examiner: What if no $\mathbf{w}^*$ exists?

Student: Then Step 3 of the proof fails — there is no direction in which we make steady progress — and PLA can cycle forever.

Examiner: So what is the deeper principle?

Student: Learning = moving parameters in a direction that provably reduces a measure of error. PLA uses a crude measure (mistakes); gradient descent will use a smooth one (loss).

3.9 Exam Toolkit

Exam Tip: Recognition rules

- “Misclassified, $y = -1$ ” $\Rightarrow$ subtract $\mathbf{x}$; new score = old score $-\|\mathbf{x}\|^2$.
- Angle questions after update: compute $\mathbf{w}_{\text{new}}$, then $\cos^{-1} \frac{\mathbf{w}_{\text{new}}^\top \mathbf{x}}{\|\mathbf{w}_{\text{new}}\| \|\mathbf{x}\|}$; sanity-check the sign of the dot product.
- Score exactly 0 $\Rightarrow$ predicted positive (read the rule in the question!).
- Separate bias $b \Rightarrow$ update it with $+y$ whenever $\mathbf{w}$ is updated.
- “Will PLA converge?” $\Rightarrow$ only if the data are linearly separable; check XOR-like patterns.

Quick Recall: Perceptron

$\hat{y} = \mathbf{1}[\mathbf{w}^\top \mathbf{x} \geq 0]$ with $x_0 = 1$, $w_0 = -\theta$. $\mathbf{w} \perp$ boundary. PLA: on a mistake $\mathbf{w} \leftarrow \mathbf{w} + y\mathbf{x}$. Score changes by $\pm \|\mathbf{x}\|^2$. Converges in $\leq 1/\delta^2$ (normalised) or $(R/\gamma)^2$ updates on separable data; cycles otherwise.

3.10 Practice Problems

1. **[FOUNDATION]** Implement OR with a perceptron using $w_0 = -\theta$.
Answer: $w_1 = w_2 = 1$, $w_0 = -1$. (Any $w_0 \in [-1, 0)$ works: input $(0, 0)$ needs $w_0 < 0$; input $(1, 0)$ needs $1 + w_0 \geq 0$.)
2. **[INTERMEDIATE]** Re-run the LUMEN example visiting the rocks in order R4, R3, R2, R1. Is the final $\mathbf{w}$ the same?
Hint: Different order, generally different final vector; both separate the data.
3. **[EXAM]** $\mathbf{w} = [1, -2]^\top$, positive point $\mathbf{x} = [2, 2]^\top$ misclassified. After the update, is it correctly classified?
Answer: Score $2 - 4 = -2$; update adds $\|\mathbf{x}\|^2 = 8$: new score $6 \geq 0$, yes.
4. **[ADVANCED]** Normalised data has $\delta = 0.2$. Upper-bound the number of PLA corrections from $\mathbf{w}_0 = \mathbf{0}$.

Answer: $1/\delta^2 = 25$.

5. **[CHALLENGE]** Explain why, starting from $\mathbf{w}_0 = \mathbf{0}$ with $\eta > 0$, the learning rate in $\mathbf{w} \leftarrow \mathbf{w} + \eta y \mathbf{x}$ does not affect the sequence of predictions.

Hint: By induction, every $\mathbf{w}_k$ is η times the $\eta = 1$ weight vector; sign of the score is unchanged.

CHAPTER 4

Networks of Perceptrons and Representation Power

Chapter Overview

A single perceptron draws one hyperplane. By *combining* perceptrons in layers we can carve the input space into arbitrary regions. This chapter shows how to build XOR, then proves the course's central Week 2 theorem: **any Boolean function of n inputs can be represented exactly by a network with one hidden layer of 2^n perceptrons and one output perceptron.**

Learning objectives. Build XOR from perceptrons in two different ways; state and prove the 2^n representation theorem; derive the output-weight conditions for a target function; count Boolean functions; judge statements about the representation power of MLPs.

4.1 Counting Boolean Functions

A Boolean function of n inputs is a truth table with 2^n rows, each row independently 0 or 1. Hence there are

$$\boxed{2^{2^n}} \text{ Boolean functions of } n \text{ inputs.}$$

For $n = 2$ there are 16; exactly **14** are linearly separable. The two that are not are XOR and XNOR (“!XOR”). As n grows, the linearly separable fraction becomes vanishingly small, so single perceptrons are hopeless for general Boolean logic.

4.2 XOR by Composition

Worked Example 4.1: Sentinel disagreement alarm, Level 2

Problem. The greenhouse has two humidity sensors $x_1, x_2 \in \{0, 1\}$. Raise an alarm if they *disagree*: $y = x_1 \oplus x_2$. Build it from perceptrons (predict 1 iff score ≥ 0).

Strategy. $x_1 \oplus x_2 = (x_1 \vee x_2) \wedge \neg(x_1 \wedge x_2) = \text{OR AND NAND}$. Each of OR, NAND, AND is linearly separable, so each is one perceptron.

Solution. $h_1 = \text{OR}$: $x_1 + x_2 - 0.5 \geq 0$. $h_2 = \text{NAND}$: $-x_1 - x_2 + 1.5 \geq 0$. $y = \text{AND}(h_1, h_2)$: $h_1 + h_2 - 1.5 \geq 0$.

x_1	x_2	h_1	h_2	y
0	0	0	1	0
0	1	1	1	1
1	0	1	1	1
1	1	1	0	0

Interpretation. The hidden layer *re-represents* the input: in (h_1, h_2) coordinates, the XOR-positive points collapse onto one point $(1, 1)$, which is linearly separable from $(0, 1)$ and $(1, 0)$. **Hidden layers change the geometry so that the next layer's linear boundary suffices.** This sentence is the whole idea of deep learning.

4.3 The 2^n Construction (Course Theorem)

4.3.1 Setup

Use inputs in $\{-1, +1\}$ (True = +1, False = -1). For $n = 2$ build 4 hidden perceptrons, one for each input pattern $\mathbf{p} \in \{-1, +1\}^2$. Hidden perceptron j has **weights equal to its pattern $\mathbf{p}_j$** and **bias -2** (in general $-n$):

$$h_j = \mathbf{1}[\mathbf{p}_j^\top \mathbf{x} - n \geq 0].$$

Claim: h_j fires only when $\mathbf{x} = \mathbf{p}_j$. Indeed $\mathbf{p}_j^\top \mathbf{x} = \sum_i p_{ji}x_i$ where each term is +1 if $x_i = p_{ji}$ and -1 otherwise. So $\mathbf{p}_j^\top \mathbf{x} = n$ exactly when every coordinate matches, and $\leq n - 2$ otherwise. With bias $-n$, only the perfect match reaches ≥ 0 .

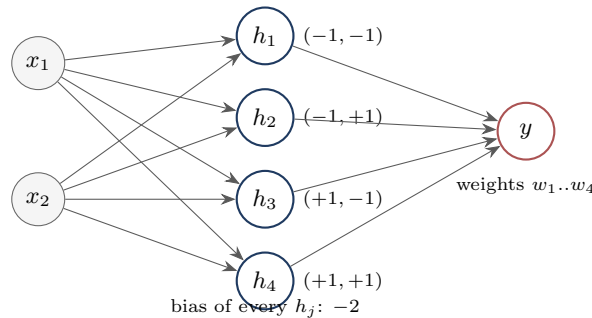


Figure 4.1: The course network for any 2-input Boolean function. Each hidden unit is a “detector” for one input pattern.

4.3.2 Choosing output weights

Exactly one hidden unit is active for each input. If the output perceptron fires when $\sum_j w_j h_j \geq w_0$, then on input $\mathbf{p}_j$ the output is simply $\mathbf{1}[w_j \geq w_0]$. So we need, for each pattern, one inequality:

$$w_j \geq w_0 \text{ if } f(\mathbf{p}_j) = 1, \quad w_j < w_0 \text{ if } f(\mathbf{p}_j) = 0.$$

These 2^n inequalities involve *distinct* variables, so they never conflict: **every** truth table is achievable.

Worked Example 4.2: XOR and its conditions, Level 3

Problem. With patterns ordered $(-1, -1), (-1, +1), (+1, -1), (+1, +1)$, write the output conditions for XOR and give one solution.

Solution. XOR is 0,1,1,0 on those patterns: $w_1 < w_0, w_2 \geq w_0, w_3 \geq w_0, w_4 < w_0$. One solution: $w_0 = 1, \mathbf{w} = (0, 1, 1, 0)$.

Verification. Input $(+1, -1)$: $h_3 = 1$ $((+1)(+1) + (-1)(-1) - 2 = 0 \geq 0)$, others: h_1 : $-1 + 1 - 2 = -2$, h_2 : $-1 - 1 - 2 = -4$, h_4 : $1 - 1 - 2 = -2$. Output: $w_3 = 1 \geq 1 \Rightarrow 1 \checkmark$.

Theorem 4.1 (Representation power of a network of perceptrons). *Any Boolean function of n inputs can be represented exactly by a network with one hidden layer of 2^n perceptrons (each detecting one input pattern, bias $-n$) and one output perceptron.*

Common Trap: Sufficient, not necessary

The theorem says 2^n hidden units are *enough*, not that they are *needed*. XOR needs only 2 hidden units (previous section). What the theorem does warn us about is growth: a general-purpose construction can need exponentially many neurons (for $n = 20$, over a million), which is one motivation for *deeper* networks that reuse intermediate features.

Course Paper Question: Quiz 1, perceptrons vs sigmoid neurons (multi-select)

Which are correct? (a) Perceptron output changes abruptly at the threshold, sigmoid output smoothly. (b) Sigmoid is differentiable, suited to gradient learning. (c) With large enough weights a sigmoid neuron

approximates a perceptron. (d) A one-hidden-layer network of perceptrons can represent any Boolean function exactly. (e) A one-hidden-layer network of sigmoid neurons can approximate any continuous function arbitrarily well given enough hidden neurons. (f) A single sigmoid neuron can represent every non-linear boundary in $\mathbb{R}^n$.

Answer. (a)–(e) true; (f) false. (d) is Theorem 4.1; (e) is the universal approximation theorem (Section 5.6); (c) is shown in Section 5.2.3; (f) fails because a single sigmoid neuron’s level sets $\{\sigma(\mathbf{w}^\top \mathbf{x} + b) = c\}$ are hyperplanes.

4.4 Why-Chain

Why-Chain: Why does one hidden layer suffice for Boolean functions?

Examiner: Why can one hidden layer represent every Boolean function?

Student: Because a Boolean input space is finite. We can dedicate one hidden unit to each of the 2^n inputs, like a lookup table, and let the output unit read off the answer.

Examiner: Does the same trick work for real-valued inputs?

Student: Not exactly: there are infinitely many real inputs, so we can’t dedicate a unit to each. We need *approximation*: cover the input space with small “bumps” — that is the sigmoid tower construction (Section 5.6).

Examiner: Why can’t we train the perceptron network with PLA?

Student: PLA needs to know the correct output of each neuron; hidden neurons have no target. And the step function has zero derivative, so errors cannot be passed backwards. Smooth neurons plus backpropagation solve both problems.

4.5 Exam Toolkit

Exam Tip: Recognition rules

- “Number of Boolean functions of n inputs” $\Rightarrow 2^{2^n}$. “Linearly separable 2-input” $\Rightarrow 14$.
- “Hidden neurons needed in the course construction” $\Rightarrow 2^n$, bias $-n$ (inputs ± 1).
- “Output conditions” $\Rightarrow$ one independent inequality $w_j \gtrless w_0$ per row of the truth table.
- “Always / any / every” statements $\Rightarrow$ distinguish exact representation (Boolean, perceptron nets) from approximation (continuous, sigmoid nets).

Quick Recall: Networks of perceptrons

Hidden layers re-represent inputs so a linear output can decide. One hidden layer of 2^n pattern detectors represents any Boolean function exactly; this is sufficient, not necessary, and grows exponentially.

4.6 Practice Problems

1. **[FOUNDATION]** How many Boolean functions have 3 inputs?
Answer: $2^8 = 256$.
2. **[INTERMEDIATE]** In the 2^n network for $n = 3$, what is the bias of each hidden neuron, and what is the largest score a non-matching input can give?
Answer: Bias -3 ; non-matching dot product ≤ 1 , score ≤ -2 .
3. **[EXAM]** Write the output conditions (pattern order as in the text) for XNOR.
Answer: $w_1 \geq w_0$, $w_2 < w_0$, $w_3 < w_0$, $w_4 \geq w_0$.
4. **[ADVANCED]** Build 3-input parity ($x_1 \oplus x_2 \oplus x_3$, inputs 0/1) with two layers of perceptrons using fewer than 8 hidden units.
Hint: Hidden units “ $\sum x_i \geq 1$ ”, “ $\sum x_i \geq 2$ ”, “ $\sum x_i \geq 3$ ”; output $h_1 - h_2 + h_3 \geq 1$.

CHAPTER 5

Sigmoid Neurons and Learning by Gradient Descent

Chapter Overview

The perceptron’s hard step makes learning with calculus impossible: nudging a weight slightly either changes nothing or flips the output. The **sigmoid neuron** replaces the step with a smooth S-curve, which makes the output a differentiable function of the weights. Once the output is differentiable, the *loss* is differentiable, and we can learn by **gradient descent**. This chapter derives gradient descent from first principles (a Taylor expansion), applies it to a single sigmoid neuron, and explains why networks of sigmoid neurons can approximate any continuous function.

Learning objectives. Differentiate the sigmoid and use $\sigma' = \sigma(1 - \sigma)$; linearise σ near 0; set up the five components of supervised learning; derive the gradient descent rule and explain why $-\nabla$ is the steepest-descent direction; compute gradients of the squared-error loss by hand; explain saturation; state the universal approximation theorem and the tower construction.

Prerequisites. Chapter 3; derivatives, chain rule, first-order Taylor series.

5.1 Working Principle and Intuition

The PULSE triage system reads a normalised vital-sign index x (e.g. lactate level) and must output a *risk*. A perceptron would say “critical” for $x = 1.001$ and “fine” for $x = 0.999$ — absurd for medicine. We want the output to rise smoothly: low risk for low x , high risk for high x , and a gentle transition in between whose steepness and position are learnable.

5.2 The Sigmoid Neuron

Definition: Sigmoid (logistic) neuron

$$\hat{y} = f(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \in (0, 1).$$

The weights $\mathbf{w}$ control the steepness (and direction) of the transition, the bias b its position. The output can be read as a probability.

5.2.1 Properties you must know

1. $\sigma(0) = \frac{1}{2}$; $\sigma(z) \rightarrow 1$ as $z \rightarrow \infty$; $\sigma(z) \rightarrow 0$ as $z \rightarrow -\infty$.
2. Symmetry: $\sigma(-z) = 1 - \sigma(z)$.
3. **Derivative.** Write $\sigma = (1 + e^{-z})^{-1}$. Then

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \underbrace{\frac{1}{1 + e^{-z}}}_{\sigma(z)} \cdot \underbrace{\frac{e^{-z}}{1 + e^{-z}}}_{1 - \sigma(z)} = \sigma(z)(1 - \sigma(z)).$$

4. Maximum slope: $\sigma(1 - \sigma)$ is largest when $\sigma = \frac{1}{2}$, i.e. at $z = 0$, giving $\sigma'(0) = \frac{1}{4}$. So $0 < \sigma'(z) \leq \frac{1}{4}$ everywhere. (This bound drives the vanishing-gradient story in Section 7.6.)

5.2.2 Linearisation near zero

Using $e^{-z} \approx 1 - z$ for small z :

$$\sigma(z) \approx \frac{1}{2-z} = \frac{1}{2} \cdot \frac{1}{1-z/2} \approx \frac{1}{2} \left(1 + \frac{z}{2}\right) = \frac{1}{2} + \frac{z}{4}.$$

This agrees with the Taylor expansion $\sigma(0) + \sigma'(0)z = \frac{1}{2} + \frac{z}{4}$. The approximation is excellent for $|z| \lesssim 1$ and increasingly wrong as $|z|$ grows, because the true sigmoid saturates while the line does not.

Course Paper Question: Quiz 1, sigmoid and its Taylor approximation

Which are true? (a) With $e^{-z} \approx 1 - z$, $\sigma(z) \approx \frac{1}{2} + \frac{z}{4}$ near 0. (b) $\sigma'(0) = \frac{1}{4}$. (c) The first-order approximation implies σ is exactly linear for all z . (d) For large $|z|$ the linear approximation becomes increasingly inaccurate. (e) Smooth differentiability enables gradient-based learning, unlike the hard-threshold perceptron.

Answer. (a), (b), (d), (e) true; (c) false — a local approximation says nothing about global shape.

5.2.3 Sigmoid versus step

Scaling the weights, $\sigma(k(\mathbf{w}^\top \mathbf{x} + b))$ with $k \rightarrow \infty$, sharpens the transition until it becomes the perceptron's step (except exactly on the boundary, where it stays $\frac{1}{2}$). So a sigmoid neuron *contains* the perceptron as a limiting case.

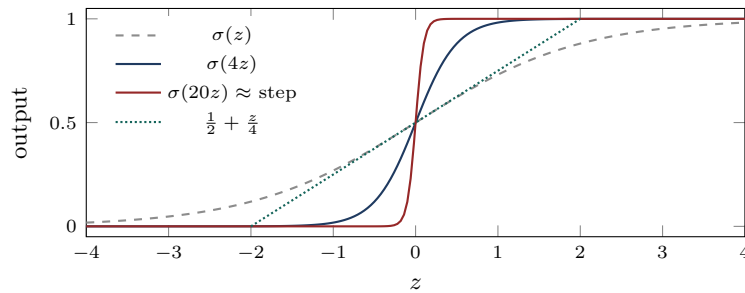


Figure 5.1: Sigmoid with increasing steepness, and its linearisation at 0.

5.3 The Supervised Learning Setup

Every learning problem in this course has five ingredients:

1. **Data** $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$.
2. **Model**: our hypothesis of how y depends on $\mathbf{x}$, e.g. $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$.
3. **Parameters to learn**: $\theta = (\mathbf{w}, b)$.
4. **Learning algorithm**: how the parameters are changed, e.g. gradient descent.
5. **Objective (loss)**: what “good” means, e.g. squared error

$$\mathcal{L}(\theta) = \frac{1}{2} \sum_{i=1}^N (f(\mathbf{x}_i) - y_i)^2.$$

The factor $\frac{1}{2}$ only cancels the 2 from differentiation; it does not move the minimum.

5.4 Gradient Descent: Derivation from First Principles

5.4.1 Which direction should we move?

Let θ be the current parameter vector and $\mathbf{u}$ a unit direction. Moving a small step η in direction $\mathbf{u}$ changes the loss by, to first order (Taylor),

$$\mathcal{L}(\theta + \eta \mathbf{u}) \approx \mathcal{L}(\theta) + \eta \mathbf{u}^\top \nabla_{\theta} \mathcal{L}(\theta).$$

The loss decreases when $\mathbf{u}^\top \nabla_\theta \mathcal{L} < 0$. Writing $\mathbf{u}^\top \nabla_\theta \mathcal{L} = \|\mathbf{u}\| \|\nabla_\theta \mathcal{L}\| \cos \beta$, the change is most negative when $\cos \beta = -1$, i.e. $\beta = 180^\circ$:

Important: Gradient descent rule

The direction of steepest local decrease is *opposite* to the gradient, giving

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t).$$

Assumption: the first-order approximation is valid only for *small* η . If η is too large, the neglected second-order terms dominate and the loss can *increase* (Chapter 8).

5.4.2 Gradients for one sigmoid neuron

Take a single example (x, y) , $f = \sigma(wx + b)$, $\mathcal{L} = \frac{1}{2}(f - y)^2$. By the chain rule,

$$\frac{\partial \mathcal{L}}{\partial w} = \underbrace{(f - y)}_{\partial \mathcal{L} / \partial f} \cdot \underbrace{f(1 - f)}_{\partial f / \partial z} \cdot \underbrace{x}_{\partial z / \partial w}, \quad \frac{\partial \mathcal{L}}{\partial b} = (f - y) f(1 - f).$$

For N examples, sum over the examples. Read each factor: *error* $\times$ *local slope of the neuron* $\times$ *input*. If any factor is 0, the weight does not move.

Algorithm 2 Gradient descent for a single sigmoid neuron (batch)

```

1: initialise  $w, b$ ; choose  $\eta$  and number of epochs
2: for epoch = 1 to max_epochs do
3:    $dw \leftarrow 0, db \leftarrow 0$ 
4:   for each  $(x, y)$  in data do
5:      $f \leftarrow \sigma(wx + b)$ 
6:      $dw \leftarrow dw + (f - y)f(1 - f)x; \quad db \leftarrow db + (f - y)f(1 - f)$ 
7:   end for
8:    $w \leftarrow w - \eta dw; \quad b \leftarrow b - \eta db$  ▷ one update per pass over data
9: end for

```

5.5 Worked Examples

Course Paper Question: Quiz 1, gradient of a sigmoid neuron

$f(x) = \sigma(wx + b)$, example $(x, y) = (2, 0.8)$, $w = b = 0$, $\mathcal{L} = \frac{1}{2}(f - y)^2$. Find $\nabla_w \mathcal{L}$.

Solution. $z = 0 \Rightarrow f = \frac{1}{2}$, $f(1 - f) = \frac{1}{4}$. $\nabla_w \mathcal{L} = (0.5 - 0.8) \cdot 0.25 \cdot 2 = -0.15$.

Interpretation. Negative gradient $\Rightarrow$ GD *increases* w , which pushes $f(2)$ up towards 0.8. ✓

Worked Example 5.1: Pulse takes one full batch step, Level 2

Problem. Data: $(x, y) = (1, 0.9)$ (sick patient) and $(-1, 0.2)$ (healthy patient). Start $w = b = 0$, $\eta = 1$. Perform one batch GD step and verify that the loss decreases.

Given. $\mathcal{L} = \frac{1}{2} \sum_i (f_i - y_i)^2$, $f = \sigma(wx + b)$.

Strategy. Forward pass for both points $\rightarrow$ per-point factors $(f - y)f(1 - f) \rightarrow$ sum with x for w , without x for $b \rightarrow$ update $\rightarrow$ re-evaluate.

Solution. Both $f_i = \frac{1}{2}$, $f(1 - f) = \frac{1}{4}$. Errors: $0.5 - 0.9 = -0.4$ and $0.5 - 0.2 = 0.3$.

$$\frac{\partial \mathcal{L}}{\partial w} = (-0.4)(0.25)(1) + (0.3)(0.25)(-1) = -0.1 - 0.075 = -0.175, \quad \frac{\partial \mathcal{L}}{\partial b} = -0.1 + 0.075 = -0.025.$$

Update: $w = 0 + 0.175 = 0.175$, $b = 0 + 0.025 = 0.025$.

Verification. Before: $\mathcal{L} = \frac{1}{2}(0.16 + 0.09) = 0.125$. After: $f_1 = \sigma(0.2) = 0.5498$, $f_2 = \sigma(-0.15) = 0.4626$; $\mathcal{L} = \frac{1}{2}((0.5498 - 0.9)^2 + (0.4626 - 0.2)^2) = \frac{1}{2}(0.1226 + 0.0690) = 0.0958 < 0.125$ ✓.

Interpretation. Both points “pulled” w upward (the sick patient because its prediction was too low at $x > 0$; the healthy one because its prediction was too high at $x < 0$), so they agree on w . On b they

disagree (one wants up, one down) and nearly cancel.

Exam Shortcut. At $w = b = 0$ every $f = \frac{1}{2}$ and every slope $= \frac{1}{4}$: $\nabla_w \mathcal{L} = \frac{1}{4} \sum_i (\frac{1}{2} - y_i) x_i$.

Worked Example 5.2: Saturation: when a huge error gives a tiny gradient, Level 4

Problem. PULSE has drifted to $w = 5$, $b = 0$. For a patient with $x = 2$ and true risk $y = 0$, compute $\nabla_w \mathcal{L}$ (squared error).

Solution. $z = 10$, $f = \sigma(10) = 0.99995$, $f(1 - f) \approx 4.54 \times 10^{-5}$. $\nabla_w \mathcal{L} = (0.99995)(4.54 \times 10^{-5})(2) \approx 9.1 \times 10^{-5}$.

Interpretation. The prediction is almost maximally wrong, yet the gradient is almost zero, because the neuron sits on the flat tail of the sigmoid. The error surface has **plateaus**; GD crawls across them. This *saturation* problem returns as the vanishing-gradient problem (Section 7.6) and is one reason for cross-entropy losses and ReLU-type activations (Part II).

5.5.1 The learning rate controls stability

Course Paper Question: Quiz 1, two learning rates on $\mathcal{L}(w) = w^2$

From $w_0 = 2$, graph X decreases rapidly to 0, graph Y increases explosively. Which is consistent: X uses $\eta = 0.3$, Y uses $\eta = 1.1$, or the reverse?

Solution. $\mathcal{L}'(w) = 2w$, so $w_{t+1} = w_t - 2\eta w_t = (1 - 2\eta)w_t$. $\eta = 0.3$: factor 0.4 — converges monotonically (X). $\eta = 1.1$: factor -1.2 — $|w|$ grows by 20% per step with alternating sign, and $\mathcal{L} = w^2$ grows by $1.44\times$ per step (Y). **X: 0.3, Y: 1.1.**

General rule. On $\mathcal{L} = \frac{\lambda}{2}w^2$, GD converges iff $0 < \eta < 2/\lambda$ (derived in Section 8.1).

5.6 Representation Power of Sigmoid Networks

5.6.1 The tower construction

Subtract two steep sigmoids with shifted centres:

$$\text{tower}(x) = \sigma(k(x - a)) - \sigma(k(x - c)), \quad a < c, \quad k \text{ large.}$$

For $x < a$ both terms are ≈ 0 ; for $a < x < c$ the first is ≈ 1 and the second ≈ 0 ; for $x > c$ both are ≈ 1 . The difference is a **rectangular bump** of height 1 on $[a, c]$. Scaling bumps by heights and adding many narrow bumps side by side reproduces any continuous function to any accuracy — exactly like a histogram / Riemann sum.

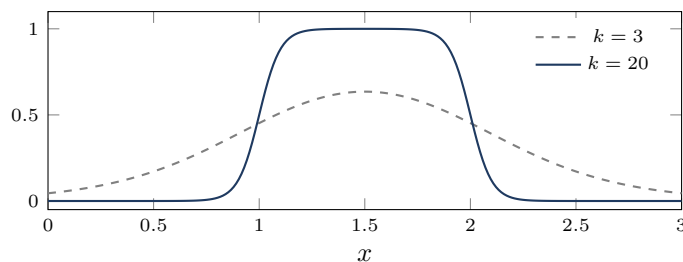


Figure 5.2: A tower on $[1, 2]$ from two sigmoids. Larger k gives a sharper bump.

Theorem 5.1 (Universal approximation, one hidden layer). *A feedforward network with one hidden layer of sigmoid neurons and a linear output can approximate any continuous function on a compact (closed, bounded) domain to any desired precision, provided the hidden layer has enough neurons.*

In n input dimensions a tower needs more sigmoids (in 2D, e.g., two “walls” in each direction combined and sharpened by a further sigmoid), and the number of towers needed grows rapidly with dimension.

Common Trap: What the theorem does not say

It guarantees that a good network *exists*; it says nothing about how many neurons are needed, whether gradient descent will *find* the weights, or how well the network generalises. “Approximate” is also not “represent exactly”.

5.7 Why-Chain

Why-Chain: Why the negative gradient?

Examiner: Why do we move along $-\nabla\mathcal{L}$?

Student: The first-order change is $\eta \mathbf{u}^\top \nabla\mathcal{L} = \eta \|\nabla\mathcal{L}\| \cos\beta$, minimised at $\beta = 180^\circ$.

Examiner: So a bigger step is always better?

Student: No. The formula is only a first-order approximation; for large η the curvature term (second order) takes over and the loss can grow, as graph Y shows.

Examiner: Is the gradient direction the fastest way to the minimum?

Student: It is the steepest *locally*. On a long narrow valley it points mostly across the valley rather than along it, which causes zig-zagging. Fixing that is the job of momentum and adaptive methods.

Examiner: What is the deeper principle?

Student: Gradient descent trusts a local linear model of the loss; every improvement in Chapters 8–9 is a smarter way to decide how far to trust it, per direction and over time.

5.8 Exam Toolkit

Exam Tip: Recognition rules

- $w = b = 0$ (or pre-activation 0) $\Rightarrow \sigma = \frac{1}{2}$, $\sigma' = \frac{1}{4}$: arithmetic becomes trivial.
- “Gradient of squared error of a sigmoid neuron” $\Rightarrow (f - y)f(1 - f)x$ — three factors.
- Very large $|z| \Rightarrow$ saturation, gradient ≈ 0 .
- $\mathcal{L} = \frac{\lambda}{2}w^2$ or w^2 with given $\eta \Rightarrow$ iterate factor $(1 - \eta\lambda)$; converge iff $|1 - \eta\lambda| < 1$.
- “Any continuous function, one hidden layer” $\Rightarrow$ universal approximation (approximate, enough neurons).

Quick Recall: Sigmoid neurons and GD

$\sigma' = \sigma(1 - \sigma) \leq \frac{1}{4}$; $\sigma \approx \frac{1}{2} + \frac{z}{4}$ near 0. GD: $\theta \leftarrow \theta - \eta \nabla\mathcal{L}$, justified by first-order Taylor. Sigmoid neuron gradient: error $\times$ slope $\times$ input. Saturation kills gradients. Towers $\Rightarrow$ universal approximation.

5.9 Practice Problems

1. **[FOUNDATION]** Compute $\sigma'(\ln 3)$.
 Answer: $\sigma(\ln 3) = \frac{3}{4}$, so $\sigma' = \frac{3}{4} \cdot \frac{1}{4} = \frac{3}{16}$.
2. **[INTERMEDIATE]** For $(x, y) = (-1, 1)$, $w = b = 0$, squared error: find $\nabla_w \mathcal{L}$ and $\nabla_b \mathcal{L}$.
 Answer: $(0.5 - 1)(0.25)(-1) = 0.125$; $\nabla_b = -0.125$.
3. **[EXAM]** $\mathcal{L}(w) = 3w^2$. For which η does GD converge? Oscillate while converging?
 Answer: $w_{t+1} = (1 - 6\eta)w_t$: converge iff $0 < \eta < \frac{1}{3}$; oscillating convergence for $\frac{1}{6} < \eta < \frac{1}{3}$.
4. **[ADVANCED]** Show that $\frac{d}{dz} \ln \sigma(z) = 1 - \sigma(z)$ and explain why this makes cross-entropy immune to saturation when the prediction is wrong.
 Hint: Differentiate; for a wrong confident prediction $\sigma \rightarrow 0$ so the gradient $\rightarrow 1$, not 0.

CHAPTER 6

Feedforward Neural Networks

Chapter Overview

We now stack sigmoid-type neurons into layers. A **feedforward neural network (FFNN)** is a composition of simple functions: affine map, non-linearity, affine map, non-linearity, $\dots$, output function. This chapter fixes the notation used for the rest of the course, shows how to choose output functions and losses, teaches parameter counting and the forward pass, and proves that without non-linear activations depth is useless.

Learning objectives. Write the layer equations and matrix shapes; count parameters; run a forward pass by hand; choose (output function, loss) pairs for regression and classification; derive cross-entropy as negative log-likelihood; use softmax identities; explain the linear-collapse result.

6.1 Architecture and Notation

Definition: Feedforward network with L layers

Input $\mathbf{x} \in \mathbb{R}^n$, $L - 1$ hidden layers and one output layer. With $\mathbf{h}_0 = \mathbf{x}$,

$$\mathbf{a}_i = \mathbf{b}_i + W_i \mathbf{h}_{i-1}, \quad \mathbf{h}_i = g(\mathbf{a}_i) \quad (i = 1, \dots, L - 1),$$

$$\mathbf{a}_L = \mathbf{b}_L + W_L \mathbf{h}_{L-1}, \quad \hat{\mathbf{y}} = f(\mathbf{x}) = O(\mathbf{a}_L).$$

$W_i \in \mathbb{R}^{n_i \times n_{i-1}}$ and $\mathbf{b}_i \in \mathbb{R}^{n_i}$, where n_i is the width of layer i . g is the (element-wise) activation and O the output function. All parameters together: $\theta = \{W_1, \dots, W_L, \mathbf{b}_1, \dots, \mathbf{b}_L\}$.

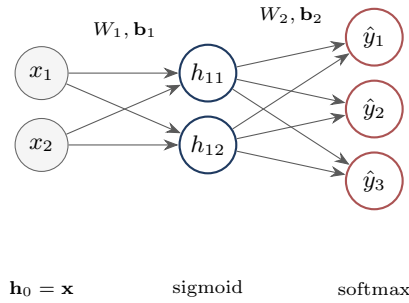


Figure 6.1: The ATLAS lane router: 2 inputs $\rightarrow$ 2 sigmoid hidden units $\rightarrow$ 3 softmax outputs.

Common Trap: Shapes

W_i has shape $(\text{outputs} \times \text{inputs}) = n_i \times n_{i-1}$, so that $W_i \mathbf{h}_{i-1}$ is defined. Row j of W_i holds the incoming weights of neuron j in layer i ; $W_{i,jk}$ is the weight from unit k of layer $i - 1$ to unit j of layer i . Questions sometimes define the transpose convention — always read the definition given.

6.2 Counting Parameters

Layer i contributes $n_i n_{i-1}$ weights and n_i biases. Total:

$$\#\theta = \sum_{i=1}^L (n_i n_{i-1} + n_i) = \sum_{i=1}^L n_i (n_{i-1} + 1).$$

Worked Example 6.1: Course-style parameter count, Level 1

Problem. A product classifier: 50 features $\rightarrow$ 64 sigmoid $\rightarrow$ 32 sigmoid $\rightarrow$ 10 softmax. Count trainable parameters.

Solution. $64(50 + 1) + 32(64 + 1) + 10(32 + 1) = 3264 + 2080 + 330 = \mathbf{5674}$.

Exam Shortcut. “(fan-in + 1) $\times$ fan-out”, layer by layer. Activation functions (sigmoid, softmax) have no parameters.

6.3 Output Functions and Loss Functions

The output function and loss are chosen *together*, according to what y is.

Task	Output function O	Loss	Gradient $\nabla_{\mathbf{a}_L} \mathcal{L}$
Regression (real y)	linear: $\hat{\mathbf{y}} = \mathbf{a}_L$	squared error $\frac{1}{2} \ \hat{\mathbf{y}} - \mathbf{y}\ ^2$	$\hat{\mathbf{y}} - \mathbf{y}$
Multi-class (k classes)	softmax	cross-entropy $-\log \hat{y}_\ell$	$\hat{\mathbf{y}} - \mathbf{e}_\ell$
Binary	sigmoid	binary cross-entropy	$\hat{y} - y$

(The gradients are derived in Chapter 7; the fact that all three have the same “prediction minus target” form is not a coincidence — each pair is a matched “canonical” pair.)

6.3.1 Softmax

$$\hat{y}_j = \text{softmax}(\mathbf{a}_L)_j = \frac{e^{a_{L,j}}}{\sum_{i=1}^k e^{a_{L,i}}}.$$

Properties: (i) every $\hat{y}_j > 0$ and $\sum_j \hat{y}_j = 1$ — a probability distribution; (ii) **shift invariance**: $\text{softmax}(\mathbf{a} + c\mathbf{1}) = \text{softmax}(\mathbf{a})$, since e^c cancels; (iii) it preserves order: larger pre-activation, larger probability.

6.3.2 Cross-entropy as negative log-likelihood

With true class ℓ , the target distribution is $\mathbf{e}_\ell$. The cross-entropy between target $\mathbf{y}$ and prediction $\hat{\mathbf{y}}$ is $-\sum_j y_j \log \hat{y}_j$; with $\mathbf{y} = \mathbf{e}_\ell$ only one term survives:

$$\mathcal{L}(\theta) = -\log \hat{y}_\ell.$$

Minimising it maximises the probability the model assigns to the correct class (maximum likelihood). $\mathcal{L} = 0$ iff $\hat{y}_\ell = 1$; $\mathcal{L} \rightarrow \infty$ as $\hat{y}_\ell \rightarrow 0$, so confident mistakes are punished heavily.

Course Paper Question: Quiz 1, inverting softmax

Three classes, $\mathbf{a}_L = [a_1, 0, 0]$. Find a_1 such that $\hat{y}_1 = 0.5$.

Solution. $\frac{e^{a_1}}{e^{a_1} + 1 + 1} = \frac{1}{2} \Rightarrow 2e^{a_1} = e^{a_1} + 2 \Rightarrow e^{a_1} = 2 \Rightarrow a_1 = \ln 2 \approx \mathbf{0.69}$.

General pattern. If the other $k - 1$ pre-activations are 0 and $\hat{y}_1 = p$, then $e^{a_1} = \frac{p(k-1)}{1-p}$.

6.4 The Forward Pass

Worked Example 6.2: Atlas routes one parcel, Level 2

Problem. Parcel features $\mathbf{x} = [1, 2]^\top$ (weight, distance; normalised). Network of Figure 6.1 with

$$W_1 = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}, \quad \mathbf{b}_1 = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad W_2 = \begin{bmatrix} 2 & 0 \\ 0 & 2 \\ -1 & -1 \end{bmatrix}, \quad \mathbf{b}_2 = \mathbf{0}.$$

The correct lane is *standard* ($\ell = 2$). Compute $\hat{\mathbf{y}}$ and the cross-entropy loss.

Strategy. Layer by layer: pre-activation, activation; softmax; $-\log$ of the correct entry. Keep 4–5 significant figures.

Solution. $\mathbf{a}_1 = W_1 \mathbf{x} + \mathbf{b}_1 = [1, -1 + 2 - 1]^\top = [1, 0]^\top$, so $\mathbf{h}_1 = [\sigma(1), \sigma(0)]^\top = [0.7311, 0.5]^\top$.

$\mathbf{a}_2 = W_2 \mathbf{h}_1 = [1.4621, 1.0000, -1.2311]^\top$.

Exponentials: $e^{1.4621} = 4.3150$, $e^1 = 2.7183$, $e^{-1.2311} = 0.2920$; sum = 7.3253.

$\hat{\mathbf{y}} = [0.5891, 0.3711, 0.0399]^\top$; $\mathcal{L} = -\ln 0.3711 = \mathbf{0.991}$.

Verification. Entries sum to $1.0001 \approx 1$ ✓. The largest pre-activation (lane 1) has the largest probability ✓.

Interpretation. The network currently prefers *express* (59%) for a parcel that belongs in *standard*; the loss 0.99 is moderate. Chapter 7 will compute how to change every weight to fix this.

Course Paper Question: Quiz 1, house-price network at initialisation

Inputs $(x_1, x_2) = (8, 3000)$, 3 sigmoid hidden units, linear output. $W_1 = 0$, $W_2 = [100, 100, 100]$, all biases 0. Initial prediction?

Solution. $\mathbf{a}_1 = W_1 \mathbf{x} + \mathbf{b}_1 = \mathbf{0}$ whatever $\mathbf{x}$ is, so $\mathbf{h}_1 = [\frac{1}{2}, \frac{1}{2}, \frac{1}{2}]$ and $\hat{y} = 100 \cdot \frac{3}{2} = \mathbf{150}$.

Trap avoided. Do not multiply by 3000! Zero first-layer weights make the network ignore its input entirely.

6.5 Linear Activations Collapse the Network

Proposition 6.1. *If every hidden activation is the identity ($g(z) = z$), an L -layer network computes $O(W' \mathbf{x} + \mathbf{b}')$ for a single matrix $W' = W_L \cdots W_2 W_1$ and a vector $\mathbf{b}'$, regardless of L and of the hidden widths.*

Proof. $\mathbf{h}_1 = W_1 \mathbf{x} + \mathbf{b}_1$; $\mathbf{h}_2 = W_2(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (W_2 W_1) \mathbf{x} + (W_2 \mathbf{b}_1 + \mathbf{b}_2)$. By induction every $\mathbf{h}_i$ is an affine function of $\mathbf{x}$, so $\mathbf{a}_L = W' \mathbf{x} + \mathbf{b}'$ with $W' = W_L \cdots W_1$. $\square$

Consequences: (i) the decision boundaries are exactly those of a single-layer model (softmax regression, or linear regression); (ii) extra layers add parameters but **no representational power**; (iii) such a network still *trains* perfectly well — gradients flow fine through linear layers — it is just not more expressive. A narrow hidden layer can even *reduce* power, since W' then has low rank.

Course Paper Question: Quiz 1 and End-Term: identity activations

$50 \rightarrow 64 \rightarrow 32 \rightarrow 10$ (softmax) with identity hidden activations. **True:** “equivalent to a single softmax regression $O(W' \mathbf{x} + \mathbf{b}')$ with $W' = W_3 W_2 W_1$.” **False:** “can learn non-linear boundaries because softmax is non-linear” (softmax is monotone in each $a_{L,j}$; the class boundaries $a_{L,i} = a_{L,j}$ are hyperplanes); “fails to train because gradients cannot flow” (they flow); “training forces W_i to identity matrices” (no such mechanism). End-term variant: “adding identity layers increases capacity” — false; “replacing identity by sigmoid enables non-linear boundaries” — true.

6.6 Representation Power of FFNNs

By the universal approximation theorem (Section 5.6), one hidden layer with enough sigmoid units approximates any continuous function; for classification, deep networks can approximate any decision region. Two practical caveats: the required width may be enormous, and existence says nothing about learnability. Depth helps by *reusing* features, so the same function often needs far fewer neurons when spread across several layers.

Why-Chain: Why do we need non-linearity between layers?

Examiner: The output softmax is non-linear. Why isn't that enough?

Student: Softmax is applied after all mixing is done. The class scores are still affine in $\mathbf{x}$, so the boundary between any two classes, $a_{L,i} = a_{L,j}$, is a hyperplane.

Examiner: What exactly does a hidden non-linearity buy us?

Student: It lets the next layer's *affine* map act on a *bent* version of the input space. As with XOR in Chapter 4, the hidden layer re-represents the data so that a linear boundary suffices.

Examiner: So is depth or non-linearity the source of power?

Student: Non-linearity. Depth without it is a single matrix; one non-linear hidden layer is already universal. Depth makes the representation *efficient*.

6.7 Exam Toolkit

Exam Tip: Recognition rules

- “Count parameters” $\Rightarrow \sum n_i(n_{i-1} + 1)$.
- “Weights of first layer are zero” $\Rightarrow$ hidden output independent of input; sigmoid gives $\frac{1}{2}$.
- “Softmax with other logits equal” $\Rightarrow$ solve $\frac{e^a}{e^a + (k-1)e^c} = p$.
- “Identity / linear hidden activations” $\Rightarrow$ collapse to one affine map; no extra power.
- “Real-valued target” $\Rightarrow$ linear output + squared error; “class label” $\Rightarrow$ softmax + cross-entropy.

Quick Recall: Feedforward networks

$\mathbf{a}_i = \mathbf{b}_i + W_i \mathbf{h}_{i-1}$, $\mathbf{h}_i = g(\mathbf{a}_i)$, $\hat{\mathbf{y}} = O(\mathbf{a}_L)$; $W_i \in \mathbb{R}^{n_i \times n_{i-1}}$. Params $\sum n_i(n_{i-1} + 1)$. Softmax + CE $\Rightarrow \mathcal{L} = -\log \hat{y}_\ell$. Linear activations collapse depth.

6.8 Practice Problems

1. **[FOUNDATION]** Count parameters of $784 \rightarrow 128 \rightarrow 10$.
Answer: $128 \cdot 785 + 10 \cdot 129 = 100480 + 1290 = 101770$.
2. **[INTERMEDIATE]** Softmax over $[2, 1, 0]$: compute $\hat{y}$.
Answer: $[0.665, 0.245, 0.090]$.
3. **[EXAM]** In the ATLAS forward pass, what is the loss if the correct lane were *express* ($\ell = 1$)?
Answer: $-\ln 0.5891 = 0.529$.
4. **[ADVANCED]** Show that a network $3 \rightarrow 1 \rightarrow 3$ with identity activations cannot represent the identity map $\mathbb{R}^3 \rightarrow \mathbb{R}^3$.
Answer: $W' = W_2 W_1$ has rank ≤ 1 , but the identity has rank 3.

CHAPTER 7

Backpropagation

Chapter Overview

Gradient descent needs $\partial\mathcal{L}/\partial w$ for *every* weight in the network. Backpropagation is simply the chain rule, organised so that each intermediate derivative is computed once and reused. We derive it in three stages — “talk to the output layer”, “talk to the hidden layers”, “talk to the weights” — then run it by hand on the ATLAS network and verify the result numerically.

Learning objectives. Derive $\nabla_{\mathbf{a}_L}\mathcal{L} = \hat{\mathbf{y}} - \mathbf{e}_\ell$ for softmax + cross-entropy; derive the hidden-layer recursion $\nabla_{\mathbf{h}_i}\mathcal{L} = W_{i+1}^\top \nabla_{\mathbf{a}_{i+1}}\mathcal{L}$ and $\nabla_{\mathbf{a}_i}\mathcal{L} = \nabla_{\mathbf{h}_i}\mathcal{L} \odot g'(\mathbf{a}_i)$; obtain $\nabla_{W_i}\mathcal{L} = \nabla_{\mathbf{a}_i}\mathcal{L} \mathbf{h}_{i-1}^\top$; execute the algorithm by hand; explain symmetry breaking and vanishing/exploding gradients.

7.1 Working Principle: the Chain Rule on a Graph

A weight deep in the network affects the loss only through the chain of quantities that follow it. If $\mathcal{L}$ depends on z through several intermediate quantities $q_1, \dots, q_m$, the multivariable chain rule sums over paths:

$$\frac{\partial\mathcal{L}}{\partial z} = \sum_m \frac{\partial\mathcal{L}}{\partial q_m} \frac{\partial q_m}{\partial z}.$$

Backpropagation evaluates these sums from the output backwards, so that the derivative with respect to each layer is built from the derivative with respect to the layer above it.

7.2 Warm-up: a Two-Neuron Chain

Worked Example 7.1: Pulse two-stage scorer, Level 2

Problem. $x \rightarrow h = \sigma(w_1 x) \rightarrow \hat{y} = \sigma(w_2 h)$, $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$. With $x = 2$, $w_1 = 0$, $w_2 = 2$, $y = 1$, compute $\partial\mathcal{L}/\partial w_2$ and $\partial\mathcal{L}/\partial w_1$.

Strategy. Forward pass, then define $\delta_2 = \partial\mathcal{L}/\partial a_2$ and pass it backwards.

Solution. Forward: $a_1 = 0$, $h = 0.5$; $a_2 = 1$, $\hat{y} = \sigma(1) = 0.7311$.

Output: $\delta_2 = (\hat{y} - y) \hat{y}(1 - \hat{y}) = (-0.2689)(0.1966) = -0.05288$.

$\frac{\partial\mathcal{L}}{\partial w_2} = \delta_2 h = -0.02644$.

Back to hidden: $\frac{\partial\mathcal{L}}{\partial h} = \delta_2 w_2 = -0.1058$; $\delta_1 = \frac{\partial\mathcal{L}}{\partial h} h(1 - h) = -0.1058 \times 0.25 = -0.02644$.

$\frac{\partial\mathcal{L}}{\partial w_1} = \delta_1 x = -0.05288$.

Interpretation. Each layer backwards multiplies the signal by (weight) $\times$ (sigmoid slope $\leq \frac{1}{4}$). Here the hidden slope $\frac{1}{4}$ and weight 2 shrink it by half. With many layers these factors compound — the seed of the vanishing-gradient problem.

7.3 Derivation for a General Network

Assume softmax output and cross-entropy loss $\mathcal{L} = -\log \hat{y}_\ell$.

7.3.1 Talk to the output layer

First, $\frac{\partial\mathcal{L}}{\partial \hat{y}_i} = -\frac{1}{\hat{y}_i} \mathbf{1}[i = \ell]$. Each $\hat{y}_\ell$ depends on *every* $a_{L,i}$ through the softmax denominator. Differentiating $\hat{y}_\ell = e^{a_{L,\ell}} / \sum_j e^{a_{L,j}}$:

$$\frac{\partial \hat{y}_\ell}{\partial a_{L,i}} = \begin{cases} \hat{y}_\ell(1 - \hat{y}_\ell) & i = \ell, \\ -\hat{y}_\ell \hat{y}_i & i \neq \ell, \end{cases} \quad \text{i.e.} \quad \frac{\partial \hat{y}_\ell}{\partial a_{L,i}} = \hat{y}_\ell (\mathbf{1}[i = \ell] - \hat{y}_i).$$

Therefore

$$\frac{\partial \mathcal{L}}{\partial a_{L,i}} = -\frac{1}{\hat{y}_\ell} \hat{y}_\ell (\mathbf{1}[i = \ell] - \hat{y}_i) = \hat{y}_i - \mathbf{1}[i = \ell], \quad \boxed{\nabla_{\mathbf{a}_L} \mathcal{L} = \hat{\mathbf{y}} - \mathbf{e}_\ell.}$$

Reading it: the correct class gets gradient $\hat{y}_\ell - 1 \leq 0$ (push its score up); every wrong class gets $\hat{y}_i \geq 0$ (push its score down, in proportion to how much probability it stole). The entries always sum to zero.

Course Paper Question: Quiz 1, output gradient

Softmax output $\hat{y}_\ell = 0.99$ for the true class. $\partial \mathcal{L} / \partial a_{L,\ell} = ?$

Solution. $\hat{y}_\ell - 1 = -0.01$. A nearly-correct prediction produces a nearly-zero push.

7.3.2 Talk to the hidden layers

Unit j of layer i feeds every unit m of layer $i + 1$ via $a_{i+1,m} = b_{i+1,m} + \sum_j W_{i+1,mj} h_{i,j}$. Summing over these paths,

$$\frac{\partial \mathcal{L}}{\partial h_{i,j}} = \sum_m \frac{\partial \mathcal{L}}{\partial a_{i+1,m}} W_{i+1,mj} \implies \boxed{\nabla_{\mathbf{h}_i} \mathcal{L} = W_{i+1}^\top \nabla_{\mathbf{a}_{i+1}} \mathcal{L}.}$$

Then through the element-wise activation, $h_{i,j} = g(a_{i,j})$:

$$\boxed{\nabla_{\mathbf{a}_i} \mathcal{L} = \nabla_{\mathbf{h}_i} \mathcal{L} \odot g'(\mathbf{a}_i)} \quad g' = \sigma(1 - \sigma) \text{ (sigmoid)}, \quad g' = 1 - \tanh^2 \text{ (tanh)}.$$

7.3.3 Talk to the weights

$a_{i,j} = b_{i,j} + \sum_k W_{i,jk} h_{i-1,k}$, so $\partial a_{i,j} / \partial W_{i,jk} = h_{i-1,k}$ and $\partial a_{i,j} / \partial b_{i,j} = 1$:

$$\boxed{\nabla_{W_i} \mathcal{L} = \nabla_{\mathbf{a}_i} \mathcal{L} \mathbf{h}_{i-1}^\top, \quad \nabla_{\mathbf{b}_i} \mathcal{L} = \nabla_{\mathbf{a}_i} \mathcal{L}.}$$

Shape check: $(n_i \times 1)(1 \times n_{i-1}) = n_i \times n_{i-1}$, the shape of W_i ✓. In words: *gradient of a weight = (error signal at its destination) × (activation at its source)*.

Algorithm 3 Forward and backward propagation (one example)

```

1: Forward:  $\mathbf{h}_0 \leftarrow \mathbf{x}$ 
2: for  $i = 1$  to  $L - 1$  do
3:    $\mathbf{a}_i \leftarrow \mathbf{b}_i + W_i \mathbf{h}_{i-1}$ ;  $\mathbf{h}_i \leftarrow g(\mathbf{a}_i)$ 
4: end for
5:  $\mathbf{a}_L \leftarrow \mathbf{b}_L + W_L \mathbf{h}_{L-1}$ ;  $\hat{\mathbf{y}} \leftarrow O(\mathbf{a}_L)$ 
6: Backward:  $\nabla_{\mathbf{a}_L} \mathcal{L} \leftarrow \hat{\mathbf{y}} - \mathbf{e}_\ell$  ▷ or  $\hat{\mathbf{y}} - \mathbf{y}$  for linear + squared error
7: for  $k = L$  down to 1 do
8:    $\nabla_{W_k} \mathcal{L} \leftarrow \nabla_{\mathbf{a}_k} \mathcal{L} \mathbf{h}_{k-1}^\top$ ;  $\nabla_{\mathbf{b}_k} \mathcal{L} \leftarrow \nabla_{\mathbf{a}_k} \mathcal{L}$ 
9:   if  $k > 1$  then
10:     $\nabla_{\mathbf{h}_{k-1}} \mathcal{L} \leftarrow W_k^\top \nabla_{\mathbf{a}_k} \mathcal{L}$ ;  $\nabla_{\mathbf{a}_{k-1}} \mathcal{L} \leftarrow \nabla_{\mathbf{h}_{k-1}} \mathcal{L} \odot g'(\mathbf{a}_{k-1})$ 
11:   end if
12: end for

```

Cost. Each backward step does one matrix–vector product with $W_k^\top$ and one outer product, i.e. $O(n_k n_{k-1})$ work — the same order as the forward pass. Total cost $O(\#\theta)$ per example, instead of $O(\#\theta^2)$ for perturbing each weight separately. That efficiency is why backprop made deep learning possible.

7.4 Worked Example: Backprop Through Atlas

Worked Example 7.2: Full gradient of the Atlas network, Level 3

Problem. Using the forward pass of the previous chapter ($\mathbf{x} = [1, 2]^\top$, $\mathbf{h}_1 = [0.7311, 0.5]^\top$, $\hat{\mathbf{y}} = [0.5891, 0.3711, 0.0399]^\top$, $\ell = 2$), compute all gradients.

Solution. *Output:* $\nabla_{\mathbf{a}_2} \mathcal{L} = \hat{\mathbf{y}} - \mathbf{e}_2 = [0.5891, -0.6289, 0.0399]^\top$.

$$\text{Output weights: } \nabla_{W_2} \mathcal{L} = \nabla_{\mathbf{a}_2} \mathcal{L} \mathbf{h}_1^\top = \begin{bmatrix} 0.4306 & 0.2945 \\ -0.4598 & -0.3145 \\ 0.0291 & 0.0199 \end{bmatrix}, \nabla_{\mathbf{b}_2} \mathcal{L} = \nabla_{\mathbf{a}_2} \mathcal{L}.$$

$$\text{Hidden: } \nabla_{\mathbf{h}_1} \mathcal{L} = W_2^\top \nabla_{\mathbf{a}_2} \mathcal{L} = \begin{bmatrix} 2 & 0 & -1 \\ 0 & 2 & -1 \end{bmatrix} \begin{bmatrix} 0.5891 \\ -0.6289 \\ 0.0399 \end{bmatrix} = \begin{bmatrix} 1.1383 \\ -1.2977 \end{bmatrix}.$$

Slopes: $\mathbf{h}_1 \odot (1 - \mathbf{h}_1) = [0.1966, 0.25]^\top$, so $\nabla_{\mathbf{a}_1} \mathcal{L} = [0.2238, -0.3244]^\top$.

$$\text{First-layer weights: } \nabla_{W_1} \mathcal{L} = \nabla_{\mathbf{a}_1} \mathcal{L} \mathbf{x}^\top = \begin{bmatrix} 0.2238 & 0.4476 \\ -0.3244 & -0.6489 \end{bmatrix}, \nabla_{\mathbf{b}_1} \mathcal{L} = [0.2238, -0.3244]^\top.$$

Verification. (1) $\nabla_{\mathbf{a}_2} \mathcal{L}$ sums to 0 ✓. (2) *Finite difference*: perturbing $W_{1,11}$ by $\pm 10^{-6}$ and recomputing the loss gives slope 0.22380, matching 0.2238 ✓. (3) One GD step with $\eta = 0.5$ lowers the loss from 0.991 to 0.252 (Master Integration Problem 1, Chapter 10).

Interpretation. Hidden unit 2's error signal is negative: increasing h_{12} helps, because h_{12} feeds the correct lane with weight +2. Row 2 of ∇_{W_1} is twice as large in column 2 as in column 1 because $x_2 = 2x_1$: *bigger inputs receive bigger updates*.

Exam Shortcut. For a single target entry: $\frac{\partial \mathcal{L}}{\partial W_{i,jk}} = \delta_{i,j} h_{i-1,k}$ — compute only the needed δ path.

7.5 Symmetry: Why Weights Must Not Start Equal

Course Paper Question: Quiz 1, identical initialisation

A hidden layer of n sigmoid neurons is initialised with identical weight vectors and identical biases. Which are true? (a) After one GD step the neurons have different weights. (b) For any input all n neurons give the same output. (c) The gradients of all neurons are always zero. (d) The layer is functionally equivalent to a single neuron.

Answer. (b) and (d) true; (a) and (c) false.

Reasoning. Identical incoming weights $\Rightarrow$ identical $a_{i,j}$ and $h_{i,j}$ (b). In backprop, unit j 's error is $\sum_m W_{i+1,mj} \delta_{i+1,m}$; if the outgoing weights are also equal (the usual meaning of “identical initialisation” of the whole network) these errors are equal, so all units receive *identical, generally non-zero* gradients (c false) and stay identical forever (a false). The layer behaves like one neuron (d). Random initialisation *breaks the symmetry*.

7.6 Vanishing and Exploding Gradients

Unroll the recursion for a chain of scalar layers: $\delta_i = \delta_{i+1} w_{i+1} \sigma'(a_i)$. Going back k layers multiplies the error by $\prod(w \sigma')$. Because $\sigma' \leq \frac{1}{4}$:

- if $|w| \leq 1$, the factor is at most 4^{-k} — after 10 layers, below 10^{-6} : early layers barely learn (**vanishing**);
- if $|w|$ is large (and units are not saturated), the product can grow geometrically (**exploding**), giving huge, unstable updates.

Course Paper Question: End-Term, backprop with sigmoid and cross-entropy (multi-select)

(a) The gradient at each layer depends on the gradient from the layer above ✓. (b) Since $\sigma' \leq 0.25$, repeated multiplication can shrink gradients toward zero ✓. (c) A very large gradient can cause excessively large updates and unstable training ✓. (d) “Small gradients always cause the loss to decrease” — **false**: a small gradient means small steps (possibly stuck on a plateau), and even small steps along a noisy/mini-batch gradient need not decrease the loss.

Remedies (ReLU, careful initialisation, batch normalisation, residual connections, LSTMs) come in Parts II–IV; the diagnosis starts here.

7.7 Common Failure Modes

Mistake	Why it looks right / why it fails / fix
Using W_{i+1} instead of $W_{i+1}^\top$	Looks symmetric; fails the shape check whenever widths differ. Always check shapes.
Forgetting $g'(\mathbf{a}_i)$	Treats hidden layer as linear; gradient too large. Apply $\odot g'$ at every hidden layer, never at a softmax output (already inside $\hat{\mathbf{y}} - \mathbf{e}_\ell$).
Applying softmax derivative again	$\hat{\mathbf{y}} - \mathbf{e}_\ell$ <i>already</i> includes softmax; multiplying by $\hat{y}(1 - \hat{y})$ double-counts.
g' evaluated at $\mathbf{h}$ instead of $\mathbf{a}$	For sigmoid $g'(\mathbf{a}) = \mathbf{h} \odot (1 - \mathbf{h})$ uses $\mathbf{h}$ — correct — but $\sigma'(\mathbf{h})$ is wrong.
Sign errors in the update	Gradient <i>descent</i> subtracts: $\theta \leftarrow \theta - \eta \nabla \mathcal{L}$.

Why-Chain: Why is backprop efficient?

Examiner: Why not just compute each partial derivative separately?

Student: Each derivative shares the entire path from its layer to the loss. Computing separately repeats that work for every weight.

Examiner: How does backprop avoid the repetition?

Student: It stores $\nabla_{\mathbf{a}_i} \mathcal{L}$ for each layer; every weight in layer i reuses it. That is dynamic programming on the computation graph.

Examiner: What does it cost in memory?

Student: All forward activations $\mathbf{h}_i$ (and $\mathbf{a}_i$) must be kept until the backward pass uses them — the price of speed.

7.8 Exam Toolkit

Exam Tip: Recognition rules

- “ $\partial \mathcal{L} / \partial a_{L,\ell}$ with softmax + CE” $\Rightarrow \hat{y}_\ell - 1$; other classes $\Rightarrow \hat{y}_i$.
- “Gradient w.r.t. a weight” $\Rightarrow (\text{downstream } \delta) \times (\text{upstream activation})$.
- “Identical weights” $\Rightarrow$ symmetry: same outputs, same gradients, never diverge.
- “Deep sigmoid network, early layers learn slowly” $\Rightarrow$ vanishing gradient via $\sigma' \leq \frac{1}{4}$.
- “Linear output + squared error” $\Rightarrow \nabla_{\mathbf{a}_L} \mathcal{L} = \hat{\mathbf{y}} - \mathbf{y}$.

Quick Recall: Backpropagation

$\nabla_{\mathbf{a}_L} \mathcal{L} = \hat{\mathbf{y}} - \mathbf{e}_\ell$; $\nabla_{\mathbf{h}_i} \mathcal{L} = W_{i+1}^\top \nabla_{\mathbf{a}_{i+1}} \mathcal{L}$; $\nabla_{\mathbf{a}_i} \mathcal{L} = \nabla_{\mathbf{h}_i} \mathcal{L} \odot g'(\mathbf{a}_i)$; $\nabla_{W_i} \mathcal{L} = \nabla_{\mathbf{a}_i} \mathcal{L} \mathbf{h}_{i-1}^\top$; $\nabla_{\mathbf{b}_i} \mathcal{L} = \nabla_{\mathbf{a}_i} \mathcal{L}$. Cost $\approx$ forward pass. Symmetric init never breaks; $\sigma' \leq \frac{1}{4}$ vanishes.

7.9 Practice Problems

1. **[FOUNDATION]** $\hat{\mathbf{y}} = [0.2, 0.7, 0.1]$, true class 3. Give $\nabla_{\mathbf{a}_L} \mathcal{L}$.
Answer: $[0.2, 0.7, -0.9]$.
2. **[INTERMEDIATE]** In the ATLAS network, what is $\partial \mathcal{L} / \partial W_{2,21}$?
Answer: $-0.6289 \times 0.7311 = -0.4598$.
3. **[EXAM]** A 5-layer chain has each $w = 1$ and each pre-activation 0. By what factor is the error signal multiplied between the output and the first layer (4 backward hops through sigmoids)?
Answer: $(1 \cdot \frac{1}{4})^4 = \frac{1}{256}$.
4. **[ADVANCED]** Derive $\nabla_{\mathbf{a}_L} \mathcal{L}$ for a sigmoid output with binary cross-entropy $-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$.
Answer: $\hat{y} - y$.
5. **[CHALLENGE]** Explain why, with identical initialisation of *only* the incoming weights (outgoing weights random), hidden units do become different after one step.
Hint: Their error signals $\sum_m W_{i+1,mj} \delta_m$ differ.

CHAPTER 8

Gradient Descent and Its Accelerations: Momentum, NAG, SGD

Chapter Overview

Backpropagation tells us the gradient; this chapter and the next are about *what to do with it*. We first analyse plain gradient descent exactly on quadratic bowls — the local shape of every smooth loss near a minimum — which explains when it converges, oscillates or diverges, and why it is slow in narrow valleys. Momentum and Nesterov’s accelerated gradient fix the slowness by accumulating history; stochastic and mini-batch variants fix the cost of computing the full gradient.

Learning objectives. Derive the GD iteration factor $1 - \eta\lambda$ and the stability bound $\eta < 2/\lambda_{\max}$; explain zig-zagging in ill-conditioned valleys; write and execute momentum and NAG updates by hand; relate the momentum coefficient β to overshoot and oscillation; count updates per epoch for batch, stochastic and mini-batch GD; explain the variance–frequency trade-off of batch size.

The Odyssey world. The spacecraft’s trim controller minimises a fuel-error “bowl”. Along one control axis the bowl is steep (small changes waste much fuel); along another it is flat. Every optimiser in these two chapters is a different pilot trying to reach the bottom quickly without crashing into the walls.

8.1 Gradient Descent on a Quadratic: an Exact Analysis

8.1.1 One dimension

Near any minimum w^* , a smooth loss looks like $\mathcal{L}(w) \approx \frac{\lambda}{2}(w - w^*)^2$ with curvature $\lambda = \mathcal{L}''(w^*) > 0$. Then $\mathcal{L}'(w) = \lambda(w - w^*)$ and GD gives, for the error $e_t = w_t - w^*$,

$$e_{t+1} = e_t - \eta\lambda e_t = (1 - \eta\lambda) e_t \implies e_t = (1 - \eta\lambda)^t e_0.$$

Everything is controlled by the **iteration factor** $r = 1 - \eta\lambda$:

Range of η	factor r	behaviour
$0 < \eta < 1/\lambda$	$0 < r < 1$	monotone convergence
$\eta = 1/\lambda$	$r = 0$	exact minimum in one step
$1/\lambda < \eta < 2/\lambda$	$-1 < r < 0$	oscillating (sign-alternating) convergence
$\eta = 2/\lambda$	$r = -1$	bounces forever between $\pm e_0$
$\eta > 2/\lambda$	$r < -1$	oscillating divergence

The loss behaves like r^{2t} , so it decreases monotonically in every convergent regime, even when w itself oscillates.

Important: Stability rule

GD on a quadratic with curvature λ converges iff $0 < \eta < 2/\lambda$. For $\mathcal{L} = cw^2$, $\lambda = 2c$, so $\eta < 1/c$.

8.1.2 Several dimensions: ill-conditioning

For a separable bowl $\mathcal{L} = \frac{1}{2} \sum_i \lambda_i (w_i - w_i^*)^2$, each coordinate evolves independently with its own factor $1 - \eta\lambda_i$. One global η must satisfy $\eta < 2/\lambda_{\max}$ (the steepest direction), which makes the flattest direction crawl with factor $1 - \eta\lambda_{\min} \approx 1$. The ratio $\kappa = \lambda_{\max}/\lambda_{\min}$ (condition number) measures how bad this is; large κ produces the familiar **zig-zag**: bouncing across the steep walls while inching along the valley floor.

Worked Example 8.1: Odyssey trim bowl (course end-term), Level 3

Problem. Minimise $f(x, y) = \frac{1}{2}(25x^2 + y^2)$ from $(x_0, y_0) = (1, 1)$ with vanilla GD, $\eta = 0.1$. (a) Find $x_1 + y_1$. (b) Along which coordinate does GD fail to converge? (c) What is the largest η that converges?

Given. $\nabla f = (25x, y)$; curvatures $\lambda_x = 25$, $\lambda_y = 1$.

Solution. (a) $\nabla f(1, 1) = (25, 1)$. $x_1 = 1 - 0.1 \cdot 25 = -1.5$, $y_1 = 1 - 0.1 = 0.9$. $x_1 + y_1 = -0.6$.

(b) Factors: $r_x = 1 - 0.1 \cdot 25 = -1.5$ ($|r_x| > 1$: diverges, x : $1, -1.5, 2.25, -3.375, \dots$); $r_y = 0.9$ (converges).

The x -coordinate diverges.

(c) Need $\eta < 2/\lambda_{\max} = 2/25 = 0.08$.

Interpretation. Even at the best safe rate, say $\eta = 0.07$, $r_y = 0.93$: after 40 steps $y \approx 0.055$ still, while x has long converged ($r_x = -0.75$). One learning rate cannot suit both directions. Adaptive methods (Chapter 9) give each coordinate its own rate.

Exam Shortcut. For diagonal quadratics, never iterate blindly: write each coordinate's factor $1 - \eta\lambda_i$ and read off the behaviour.

8.2 Momentum-Based Gradient Descent

8.2.1 Working principle

On gentle slopes or plateaus (e.g. saturated sigmoid regions), the gradient is small and GD crawls. A ball rolling downhill does not crawl: it *accumulates velocity*. Momentum keeps a running history of gradients and moves along it.

Definition: Momentum (course form)

$$\mathbf{u}_t = \beta \mathbf{u}_{t-1} + \nabla \mathbf{w}_t, \quad \mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{u}_t, \quad \mathbf{u}_{-1} = \mathbf{0}, \quad 0 \leq \beta < 1.$$

Unrolling the recursion shows what $\mathbf{u}_t$ is:

$$\mathbf{u}_t = \nabla \mathbf{w}_t + \beta \nabla \mathbf{w}_{t-1} + \beta^2 \nabla \mathbf{w}_{t-2} + \dots = \sum_{\tau=0}^t \beta^{t-\tau} \nabla \mathbf{w}_\tau,$$

an **exponentially weighted sum** of all past gradients. If the gradient is constant ($\mathbf{g}$), $\mathbf{u}_t \rightarrow \mathbf{g}/(1 - \beta)$: with $\beta = 0.9$ the effective step becomes $10\times$ larger on consistent slopes. Along oscillating directions, successive gradients have opposite signs and partially cancel.

Warning: Two equivalent notations

Many texts write $\text{update}_t = \gamma \text{update}_{t-1} + \eta \nabla \mathbf{w}_t$, $\mathbf{w}_{t+1} = \mathbf{w}_t - \text{update}_t$. With constant η this is identical to the course form ($\text{update}_t = \eta \mathbf{u}_t$). Use whichever the question uses.

Worked Example 8.2: Momentum overshoots the Odyssey minimum, Level 2

Problem. $\mathcal{L}(w) = w^2$, $w_0 = 1$, $\eta = 0.1$, $\beta = 0.9$. Run 4 momentum steps; compare with vanilla GD.

Solution. Gradient $2w$.

t	∇w_t	$u_t = 0.9u_{t-1} + \nabla w_t$	$w_{t+1} = w_t - 0.1u_t$	vanilla $w_{t+1} = 0.8w_t$
0	2.000	2.000	0.800	0.800
1	1.600	3.400	0.460	0.640
2	0.920	3.980	0.062	0.512
3	0.124	3.706	-0.309	0.410

Interpretation. Momentum reaches the neighbourhood of 0 in 3 steps where vanilla GD is still at 0.51 — but it arrives with large velocity (3.7) and **overshoots** to -0.31; it will oscillate around the minimum before settling. Fast, but with “U-turns”.

8.2.2 Why momentum oscillates: the heavy-ball equation

Beyond the Core Syllabus: Exact analysis on a quadratic

With $\mathcal{L} = \frac{\lambda}{2}(w - w^*)^2$ and $e_t = w_t - w^*$: $u_t = \beta u_{t-1} + \lambda e_t$ and $\eta u_{t-1} = e_{t-1} - e_t$. Eliminating u :

$$e_{t+1} = (1 + \beta - \eta\lambda) e_t - \beta e_{t-1}.$$

Solutions are $e_t \propto r^t$ with $r^2 - (1 + \beta - \eta\lambda)r + \beta = 0$. If $(1 + \beta - \eta\lambda)^2 < 4\beta$ the roots are complex with $|r| = \sqrt{\beta}$: *damped oscillation* whose decay rate per step is $\sqrt{\beta}$. Larger $\beta \Rightarrow$ slower decay of the oscillation. This single fact decodes curve-matching questions.

Course Paper Question: Quiz 1, matching momentum curves

$\mathcal{L}(w) = (w - 5)^2$, $w_0 = -5$, $\eta = 0.04$, $\beta \in \{0, 0.5, 0.9, 0.99\}$. Curve S decays smoothly without oscillation; T drops fastest, almost no oscillation; U oscillates a few times then settles; V oscillates for over 140 steps. Match.

Solution with the heavy-ball roots. $\lambda = 2$, $\eta\lambda = 0.08$.

β	roots r	$ r $	behaviour
0	0.92	0.92	smooth monotone (S)
0.5	0.774, 0.646 (real)	0.774	fastest, no oscillation (T)
0.9	complex	$\sqrt{0.9} = 0.949$	damped oscillation (U)
0.99	complex	$\sqrt{0.99} = 0.995$	long-lived oscillation (V)

S: 0, T: 0.5, U: 0.9, V: 0.99.

Fast exam reasoning (no algebra). $\beta = 0$ is plain GD $\Rightarrow$ no oscillation. More momentum $\Rightarrow$ more overshoot and longer ringing. So the most persistent oscillation is the largest β , and the moderate $\beta = 0.5$ is the fast, well-damped one.

8.3 Nesterov Accelerated Gradient (NAG)

8.3.1 Working principle: look before you leap

Momentum computes the gradient at the current point and then adds the (large) history step — by the time it notices the slope has reversed, it is already past the minimum. NAG first takes the history step *tentatively*, evaluates the gradient at that *look-ahead* point, and corrects:

Definition: Nesterov accelerated gradient (course form)

$$\mathbf{w}_{\text{look}} = \mathbf{w}_t - \eta\beta \mathbf{u}_{t-1}, \quad \mathbf{u}_t = \beta \mathbf{u}_{t-1} + \nabla \mathcal{L}(\mathbf{w}_{\text{look}}), \quad \mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{u}_t.$$

Worked Example 8.3: NAG versus momentum on the same bowl, Level 3

Problem. Repeat the previous example ($\mathcal{L} = w^2$, $w_0 = 1$, $\eta = 0.1$, $\beta = 0.9$) with NAG.

Solution.

t	$w_{\text{look}} = w_t - 0.09u_{t-1}$	$\nabla \mathcal{L}(w_{\text{look}})$	u_t	w_{t+1}
0	1.000	2.000	2.000	0.800
1	0.620	1.240	3.040	0.496
2	0.222	0.445	3.181	0.178
3	-0.108	-0.217	2.646	-0.087

Interpretation. At $t = 3$ the look-ahead point is already past 0, so NAG sees a *negative* gradient and brakes (u falls from 3.18 to 2.65). The overshoot is -0.087 instead of momentum's -0.309 : smaller U-turns, faster settling.

8.4 Stochastic and Mini-Batch Gradient Descent

8.4.1 The cost problem

The true gradient is a sum over all N examples, $\nabla \mathcal{L} = \sum_{i=1}^N \nabla \mathcal{L}_i$. With millions of examples, one exact step costs a full pass over the data.

Definition: Batch, stochastic and mini-batch GD

- **Batch (vanilla) GD:** one update per pass, using the exact gradient over all N points.
- **Stochastic GD (SGD):** update after *each* example using $\nabla \mathcal{L}_i$ only — N updates per pass.
- **Mini-batch GD:** update after each batch of B examples — $\lceil N/B \rceil$ updates per pass.

An **epoch** is one pass over the entire data; a **step** is one parameter update.

Method	examples per step	steps per epoch	gradient quality
Batch GD	N	1	exact
Mini-batch	B	$\lceil N/B \rceil$	noisy, variance $\propto 1/B$
SGD	1	N	very noisy

8.4.2 Why noise, and how much

The mini-batch gradient is an *estimate* of the true (average) gradient. It is unbiased (correct on average), and if per-example gradients have variance s^2 , the average of B independent ones has variance s^2/B . So larger batches give **smoother, more accurate** steps but **fewer** of them per epoch; smaller batches give more, noisier steps. The noise is not all bad — it can shake the parameters out of shallow minima and saddle regions — but it also makes the loss curve jagged, and a single stochastic step is *not* guaranteed to decrease the total loss.

Course Paper Question: Quiz 1, bird-audio classifier

$N = 100,000$ clips. (a) Updates per epoch with vanilla GD? (b) With mini-batches of $B = 50$?

Solution. (a) **1**. (b) $100000/50 = 2000$.

Course Paper Question: End-Term, batch size 32 $\rightarrow$ 128

Same data and learning rate. **Correct:** each update uses a gradient estimate with *lower* variance, and the number of updates per epoch *decreases* (by $4\times$). Incorrect: “higher variance”, “updates increase fourfold”, “guarantees a better minimum”.

Course Paper Question: Quiz 1, training statements (multi-select)

- (a) “SGD ($B = 1$) computes the true gradient of the total loss at each step” — **false** (it uses one example).
 (b) “Mini-batch: one epoch = $\lceil N/B \rceil$ updates” — **true**. (c) “Momentum can overshoot a narrow valley because accumulated velocity carries parameters past the minimum” — **true** (see the overshoot example).
 (d) “Vanilla batch GD decreases the loss each step for sufficiently small η on a smooth loss” — **true** (first-order Taylor; unless already at a stationary point).

Common Trap: Counting with a remainder

$N = 1000$, $B = 64$: $1000/64 = 15.625$, so an epoch has $\lceil 15.625 \rceil = 16$ updates (the last batch has 40 examples). Do not round down, and do not forget that SGD’s count equals N .

Momentum and NAG combine naturally with mini-batches: simply replace $\nabla \mathbf{w}_t$ by the mini-batch gradient. Accumulated history also *averages out* some of the stochastic noise.

8.5 Choosing the Learning Rate in Practice

- **Log-scale search:** try $\eta \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$, run a few epochs each, keep the one whose loss falls fastest without blowing up, then refine around it.

- **Line search:** at each step, try several η values along $-\nabla\mathcal{L}$ and keep the one giving the lowest loss. Each step costs several extra loss evaluations but moves as far as is safe.
- **Schedules:** large η early (fast progress), smaller later (fine convergence) — Chapter 9.

8.6 Comparison

Property	Vanilla GD	Momentum	NAG
Update	$\mathbf{w} - \eta \nabla \mathbf{w}_t$	$\mathbf{u}_t = \beta \mathbf{u}_{t-1} + \nabla \mathbf{w}_t$; $\mathbf{w} - \eta \mathbf{u}_t$	gradient at $\mathbf{w}_t - \eta \beta \mathbf{u}_{t-1}$
Memory	none	one velocity vector	one velocity vector
Plateaus	crawls	accelerates (up to $\frac{1}{1-\beta} \times$)	accelerates
Narrow valleys	zig-zags	overshoots, oscillates	smaller U-turns
Main trap	$\eta > 2/\lambda_{\max}$ diverges	large β rings for long	look-ahead uses $\eta \beta \mathbf{u}_{t-1}$, not $\eta \mathbf{u}_{t-1}$

Why-Chain: Why does momentum help if it also overshoots?

Examiner: Momentum overshoots. Why use it?

Student: Because most of the loss landscape is *not* the immediate neighbourhood of the minimum; it is long gentle slopes and plateaus where GD wastes thousands of steps. Momentum shortens those drastically.

Examiner: And in a zig-zag valley?

Student: The across-valley gradient components alternate in sign and partially cancel in $\mathbf{u}_t$, while the along-valley components add up. So momentum damps the zig-zag and accelerates along the floor.

Examiner: Then why ever choose a small β ?

Student: Because the oscillation near the minimum decays only like $(\sqrt{\beta})^t$. Very large β means very slow settling — curve V.

Examiner: Deeper principle?

Student: Momentum is a low-pass filter on the gradient sequence: it keeps persistent directions and suppresses alternating ones. β sets the filter's memory length $\approx \frac{1}{1-\beta}$ steps.

8.7 Exam Toolkit

Exam Tip: Recognition rules

- Quadratic loss + given $\eta \Rightarrow$ factor $1 - \eta\lambda$ per coordinate; $|\cdot| > 1$ diverges; negative oscillates.
- “Which coordinate diverges” $\Rightarrow$ the one with the largest curvature.
- “Match curves to β ” $\Rightarrow$ more ringing = larger β ; no ringing = $\beta = 0$ or small.
- “Updates per epoch” $\Rightarrow$ batch 1, SGD N , mini-batch $\lceil N/B \rceil$.
- “Larger batch” $\Rightarrow$ lower variance, fewer updates per epoch, no guarantee of better minimum.
- “NAG” $\Rightarrow$ gradient at the look-ahead point $\mathbf{w}_t - \eta \beta \mathbf{u}_{t-1}$.

Quick Recall: GD, momentum, NAG, SGD

Quadratic: $e_{t+1} = (1 - \eta\lambda)e_t$, stable iff $\eta < 2/\lambda_{\max}$. Momentum: $\mathbf{u}_t = \beta \mathbf{u}_{t-1} + \nabla \mathbf{w}_t$, EMA of gradients, speeds plateaus, overshoots; oscillation decays like $\sqrt{\beta}^t$. NAG: gradient at look-ahead, smaller overshoot. Mini-batch: $\lceil N/B \rceil$ steps/epoch, variance $\propto 1/B$.

8.8 Practice Problems

1. **[FOUNDATION]** $\mathcal{L} = (w - 3)^2$, $w_0 = 0$, $\eta = 0.25$. Find w_1, w_2 .
Answer: factor $1 - 0.5 = 0.5$ on the error: $w_1 = 1.5$, $w_2 = 2.25$.
2. **[INTERMEDIATE]** $f = \frac{1}{2}(4x^2 + y^2)$ from $(1, 1)$ with $\eta = 0.4$: which coordinate oscillates, and does it converge?
Answer: $r_x = 1 - 1.6 = -0.6$ (oscillates, converges); $r_y = 0.6$.

3. **[EXAM]** $N = 50,000$, $B = 128$. Updates in 3 epochs?

Answer: $\lceil 390.625 \rceil = 391$ per epoch; 1173.

4. **[ADVANCED]** Show that with constant gradient g momentum's velocity approaches $g/(1 - \beta)$.

Hint: Fixed point of $u = \beta u + g$.

5. **[CHALLENGE]** For $\mathcal{L} = \frac{\lambda}{2}w^2$ with momentum, find the condition on β for the iteration to have real (non-oscillating) roots when $\eta\lambda = 0.08$.

Answer: Need $(0.92 + \beta)^2 \geq 4\beta$, i.e. $\beta^2 - 2.16\beta + 0.8464 \geq 0$, whose roots are $\beta = \frac{2.16 \pm \sqrt{1.28}}{2} \approx 0.514, 1.646$.
With $0 \leq \beta < 1$: real roots iff $\beta \leq 0.514$ (consistent with $\beta = 0.5$ not oscillating and $\beta = 0.9$ oscillating).

Adaptive Learning Rates and Learning-Rate Schedules

Chapter Overview

Chapter 8 ended with a dilemma: one global learning rate cannot suit a steep direction and a flat direction at the same time. **Adaptive methods** give every parameter its own effective learning rate, computed from the history of *its own* gradients. We develop them as a chain of fixes — AdaGrad → RMSProp → AdaDelta → Adam → AdaMax, NAdam — where each method repairs a specific flaw of the previous one, and then study **schedules** that vary the learning rate over time.

Learning objectives. Write every update rule from memory; compute effective learning rates and the first few steps by hand; explain AdaGrad’s decay problem, RMSProp’s fix, AdaDelta’s removal of η , and Adam’s bias correction (with derivation); evaluate step, exponential, $1/t$, cyclical and cosine schedules; pick the right tool for saddle points and sparse features.

9.1 AdaGrad

9.1.1 Working principle

Suppose one feature is *sparse* (non-zero for only a few examples). Its weight receives a gradient only rarely, so under a global η it barely learns. Conversely, a direction with huge gradients (the steep wall of ODYSSEY) needs a *small* rate. AdaGrad’s rule: *decay each parameter’s learning rate in proportion to how much it has already been updated.*

Definition: AdaGrad

Per coordinate,

$$v_t = v_{t-1} + (\nabla w_t)^2, \quad w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla w_t, \quad v_{-1} = 0.$$

The *effective learning rate* is $\eta_t^{\text{eff}} = \eta / \sqrt{v_t + \epsilon}$.

Worked Example 9.1: AdaGrad on the Odyssey bowl (course end-term), Level 3

Problem. $f(x, y) = \frac{1}{2}(25x^2 + y^2)$, start $(1, 1)$, $\eta = 0.1$, $\epsilon = 10^{-8}$. Compute $\eta_x^{(1)} + \eta_y^{(1)}$ and describe how AdaGrad fixes the divergence of vanilla GD.

Solution. First gradient $(25, 1)$. $v_x = 625$, $v_y = 1$. $\eta_x = 0.1/\sqrt{625} = 0.004$, $\eta_y = 0.1/1 = 0.1$. Sum = **0.104**.

Along x the rate is reduced by a factor of $\sqrt{625} = \mathbf{25}$ (the course’s correct option: “reduced 25 times along the x -direction”); along y it is unchanged. New point: $x_1 = 1 - 0.004 \cdot 25 = 0.9$, $y_1 = 1 - 0.1 \cdot 1 = 0.9$.

Next steps. $(0.8331, 0.8331)$, then $(0.7805, 0.7805)$ — no divergence, and both coordinates move *identically*.

Interpretation. AdaGrad divides each gradient by the root of its accumulated square, so a coordinate that is uniformly $25\times$ steeper gets a $25\times$ smaller rate: the method is **invariant to per-coordinate rescaling**. That is exactly the cure for ill-conditioning along the axes.

Exam Shortcut. First AdaGrad step: $v = g^2$, so the step is $\eta g / |g| = \eta \text{sign}(g)$ in every coordinate.

Warning: AdaGrad’s flaw

v_t only grows. For frequently updated (dense) parameters the effective rate decays like $1/\sqrt{t}$ towards zero, and learning can stall before reaching the minimum.

9.2 RMSProp

Fix: replace the *sum* by an *exponentially decaying average*, so old gradients are forgotten.

Definition: RMSProp

$$v_t = \beta v_{t-1} + (1 - \beta)(\nabla w_t)^2, \quad w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla w_t.$$

Now the denominator tracks the *recent* gradient magnitude, so the effective rate can go down *or up*. AdaGrad's rate can only decrease.

Worked Example 9.2: RMSProp's oversized first step, Level 3

Problem. Same bowl, $\eta = 0.1$, $\beta = 0.9$. First step?

Solution. $v_1 = 0.1g^2 \Rightarrow \text{step} = \eta g / \sqrt{0.1g^2} = \eta / \sqrt{0.1} \cdot \text{sign } g \approx 3.16\eta \text{ sign } g$. So $x_1 = y_1 = 1 - 0.316 = 0.684$; following steps: 0.499, 0.369.

Interpretation. Because v starts at 0, early estimates of $\mathbb{E}[g^2]$ are *biased low* by the factor $(1 - \beta^t)$, inflating early steps. Adam will correct this bias.

9.3 AdaDelta

RMSProp still needs a hand-tuned η . AdaDelta replaces η by a second running average — of past *updates* — so the step has the right “units” automatically.

Definition: AdaDelta

$$v_t = \beta v_{t-1} + (1 - \beta)(\nabla w_t)^2, \quad \Delta w_t = -\frac{\sqrt{u_{t-1} + \epsilon}}{\sqrt{v_t + \epsilon}} \nabla w_t,$$

$$w_{t+1} = w_t + \Delta w_t, \quad u_t = \beta u_{t-1} + (1 - \beta)(\Delta w_t)^2.$$

The numerator u_{t-1} “runs one step behind” the denominator. First step (with $u_{-1} = 0$): $|\Delta w| \approx \sqrt{\epsilon} / \sqrt{1 - \beta}$ — with $\epsilon = 10^{-6}$, $\beta = 0.9$, about 0.0032: AdaDelta starts cautiously and *grows* its steps as update history accumulates. If gradients become small while past updates were large, the ratio $\sqrt{u}/\sqrt{v}$ increases, so the rate adapts upward.

9.4 Adam (Adaptive Moments)

9.4.1 Rule

Adam combines momentum (a running mean of gradients, the *first moment*) with RMSProp (a running mean of squared gradients, the *second moment*) and corrects both for their zero initialisation.

Definition: Adam

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla w_t, \quad \hat{m}_t = \frac{m_t}{1 - \beta_1^t},$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla w_t)^2, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t},$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t, \quad \text{typical } \beta_1 = 0.9, \beta_2 = 0.999, \eta = 10^{-3}.$$

Here $t = 1, 2, \dots$ counts steps, and $m_0 = v_0 = 0$.

9.4.2 Why bias correction? (derivation)

Unroll: $m_t = (1 - \beta_1) \sum_{\tau=1}^t \beta_1^{t-\tau} \nabla w_\tau$. If the gradients are drawn from a stationary distribution with mean $\mathbb{E}[g]$,

$$\mathbb{E}[m_t] = \mathbb{E}[g] (1 - \beta_1) \sum_{\tau=1}^t \beta_1^{t-\tau} = \mathbb{E}[g] (1 - \beta_1) \frac{1 - \beta_1^t}{1 - \beta_1} = (1 - \beta_1^t) \mathbb{E}[g].$$

So m_t underestimates the mean by exactly the factor $1 - \beta_1^t$; dividing by it gives an unbiased estimate. Same for v_t with β_2 . The correction matters most early (at $t = 1$ with $\beta_2 = 0.999$ the factor is 0.001!) and fades as $t \rightarrow \infty$.

Worked Example 9.3: Adam's first two steps, Level 3

Problem. $\mathcal{L} = w^2$, $w_0 = 1$, $\eta = 0.1$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon \approx 0$. Compute w_1, w_2 .

Solution. $t = 1$: $g = 2$; $m_1 = 0.2$, $v_1 = 0.004$; $\hat{m}_1 = 0.2/0.1 = 2$, $\hat{v}_1 = 0.004/0.001 = 4$; step $0.1 \cdot 2/2 = 0.1$; $w_1 = \mathbf{0.9}$.

$t = 2$: $g = 1.8$; $m_2 = 0.18 + 0.18 = 0.36$; $v_2 = 0.003996 + 0.00324 = 0.007236$; $\hat{m}_2 = 0.36/0.19 = 1.8947$; $\hat{v}_2 = 0.007236/0.001999 = 3.6198$, $\sqrt{\hat{v}_2} = 1.9026$; step $0.1 \cdot 1.8947/1.9026 = 0.0996$; $w_2 = \mathbf{0.8004}$.

Verification (why correction is needed). Without correction the first step would be $0.1 \cdot 0.2 / \sqrt{0.004} = 0.316$ — over three times too large, the same inflation as RMSPProp.

Interpretation. With bias correction the first Adam step is $\eta g / |g| = \eta \text{sign}(g)$: Adam's step size is roughly η independent of the gradient's scale. On the ODYSSEY bowl it moves $(1, 1) \rightarrow (0.9, 0.9) \rightarrow (0.800, 0.800) \rightarrow (0.702, 0.702)$.

9.5 AdaMax and NAdam

AdaMax. Adam's $\sqrt{v_t}$ is an (L^2 -type) norm of past gradients. Replacing it with the L^∞ norm (the maximum) gives

$$v_t = \max(\beta_2 v_{t-1}, |\nabla w_t|), \quad w_{t+1} = w_t - \frac{\eta}{v_t + \epsilon} \hat{m}_t.$$

No bias correction for v_t : a max is not dragged towards the zero initialisation. It is also robust when a parameter's gradient goes to zero for a while (sparse inputs): v_t decays only slowly, so the rate does not suddenly jump. On $\mathcal{L} = w^2$ (same settings) it gives $w_1 = 0.9$, $w_2 = 0.805$.

NAdam. Nesterov-style look-ahead inside Adam: use the momentum you *will* have, i.e. blend the corrected momentum with the current bias-corrected gradient,

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \left(\beta_1 \hat{m}_t + \frac{(1 - \beta_1) \nabla w_t}{1 - \beta_1^t} \right).$$

(Course notes write the indices of the bias-correction terms slightly differently; what is tested is the idea: *Adam + NAG look-ahead*.)

9.6 Learning-Rate Schedules

Adaptive methods adjust rates *per parameter*; schedules adjust the global η *over time*.

Schedule	Formula	Example ($\eta_0 = 0.1$)
Step decay	halve η every k epochs	$k = 10$: epoch 25 $\Rightarrow 0.1 \cdot 0.5^2 = 0.025$
Exponential	$\eta_t = \eta_0 e^{-kt}$	$k = 0.1, t = 10$: 0.0368
$1/t$ decay	$\eta_t = \eta_0 / (1 + kt)$	$k = 0.1, t = 10$: 0.05
Cyclical (triangular)	η rises linearly $\eta_{\min} \rightarrow \eta_{\max}$ over s steps, falls back over s steps	$s = 10, [0.001, 0.01]$: $t = 5 \Rightarrow 0.0055, t = 10 \Rightarrow 0.01, t = 20 \Rightarrow 0.001$
Cosine annealing with warm restarts	$\eta_t = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2} \left(1 + \cos \frac{\pi (t \bmod (T+1))}{T} \right)$	$\eta_{\max} = 0.1, \eta_{\min} = 0.001, T = 50$: $t = 0 \Rightarrow 0.1$; $t = 25 \Rightarrow 0.0505$; $t = 50 \Rightarrow 0.001$; $t = 51 \Rightarrow 0.1$ (restart)
Warm-up	start small and increase, e.g. $\eta_t = \eta t/k$ for $t \leq k$	helps early instability (large initial gradients)

Key Idea: Why cyclical rates help at saddle points

Much of the difficulty of training comes from *saddle points* and plateaus (gradient ≈ 0 but not a minimum), not from poor local minima. With a monotonically decreasing rate, by the time the parameters reach a saddle the rate is tiny and they cannot leave. Periodically raising η (cyclical, cosine restarts) supplies the “kick” to escape.

9.7 Comparison of Optimisers

Method	Key idea	Fixes	Weakness
GD / SGD	$-\eta \nabla$	–	one rate for all; noisy (SGD)
Momentum	EMA of gradients	plateaus, zig-zag	overshoot
NAG	gradient at look-ahead	overshoot	still one global rate
AdaGrad	divide by $\sqrt{\sum g^2}$	sparse features, ill-scaling	rate decays to 0
RMSProp	divide by $\sqrt{\text{EMA}(g^2)}$	AdaGrad’s decay	needs η ; biased early
AdaDelta	ratio of update-EMA to gradient-EMA	removes η	slow start, extra β
Adam	momentum + RMSProp + bias correction	all of the above	can still overshoot; default choice
AdaMax	L^∞ norm in denominator	robust to zero gradients	niche
NAdam	Adam + NAG	overshoot	slightly more complex

Why-Chain: Why does Adam need bias correction but AdaMax does not?

Examiner: Why correct m_t and v_t ?

Student: They start at 0 and are exponential averages, so early on they are shrunk by $1 - \beta^t$. Without correction, v is shrunk much more than m (since $\beta_2 > \beta_1$), making early steps too large.

Examiner: AdaMax also starts v at 0. Why no correction?

Student: Its $v_t = \max(\beta_2 v_{t-1}, |g|)$ equals $|g|$ immediately at $t = 1$; a maximum is not averaged with the zero initial value, so there is no shrinkage to undo.

Examiner: What is the common principle behind all adaptive methods?

Student: Normalise each coordinate’s step by an estimate of that coordinate’s gradient scale, so every direction moves at a comparable speed — a cheap, diagonal substitute for knowing the curvature.

9.8 Exam Toolkit

Exam Tip: Recognition rules

- “Effective learning rate after step 1” (AdaGrad) $\Rightarrow \eta / |g_1|$ per coordinate.
- “Reduced k times along x ” $\Rightarrow k = \sqrt{v_x}$; the steeper coordinate is reduced.
- First bias-corrected Adam step $\Rightarrow \eta \text{ sign}(g)$.
- “Rate can only decrease” $\Rightarrow$ AdaGrad; “can increase or decrease” $\Rightarrow$ RMSProp/AdaDelta.
- “No learning rate to set” $\Rightarrow$ AdaDelta. “ L^∞ / max” $\Rightarrow$ AdaMax.
- “Stuck at saddle point with decaying η ” $\Rightarrow$ cyclical / cosine with restarts.

Quick Recall: Adaptive methods

AdaGrad $v \leftarrow v + g^2$; RMSProp $v = \beta v + (1 - \beta)g^2$; AdaDelta uses $\sqrt{u_{t-1} + \epsilon} / \sqrt{v_t + \epsilon}$; Adam = EMA(g) / $\sqrt{\text{EMA}(g^2)}$ with $\div (1 - \beta^t)$; AdaMax uses max; NAdam adds look-ahead. Schedules: step, exp, $1/t$, cyclical, cosine restarts, warm-up.

9.9 Practice Problems

1. **[FOUNDATION]** AdaGrad, gradients 3 then 4, $\eta = 1$. Effective rates at steps 1 and 2?
Answer: $1/3$ and $1/5$.
2. **[INTERMEDIATE]** RMSProp with $\beta = 0.99$: how many times larger than η is the first step (no correction)?
Answer: $1/\sqrt{0.01} = 10$.
3. **[EXAM]** In Adam with $\beta_1 = 0.9$, what is the bias-correction divisor for m at $t = 3$?
Answer: $1 - 0.729 = 0.271$.
4. **[ADVANCED]** Cosine annealing, $\eta_{\max} = 0.2$, $\eta_{\min} = 0$, $T = 100$: find η at $t = 25$ and $t = 101$.
Answer: $0.1(1 + \cos \frac{\pi}{4}) = 0.1707$; restart: 0.2.
5. **[CHALLENGE]** Show that AdaGrad’s trajectory on $\frac{1}{2} \sum_i \lambda_i w_i^2$ from $w_i^{(0)} = c$ is identical in every coordinate.
Hint: Induction: $g_i = \lambda_i w_i$, and λ_i cancels between numerator and $\sqrt{\sum g_i^2}$.

Part I Integration: Master Problems and Rapid Revision

Chapter Overview

This chapter combines everything in Part I into multi-step problems of the kind that appear late in a paper, provides an index of every course paper question solved in this Part, and ends with the Part I master reference sheet for the night before the exam.

10.1 Master Integration Problems

Master Integration Problem 1: Atlas learns from one mistake

Problem. Take the ATLAS network and gradients of Chapters 6–7 ($\mathbf{x} = [1, 2]^\top$, $\ell = 2$, loss 0.991). (1) Perform one GD step with $\eta = 0.5$ on all parameters. (2) Recompute the loss. (3) State the cost of the step in multiply-adds. (4) Identify one limitation of this update.

Strategy. $\theta \leftarrow \theta - \eta \nabla \mathcal{L}$ using the stored gradients; then a fresh forward pass.

Solution. (1) $W_1 \leftarrow \begin{bmatrix} 0.8881 & -0.2238 \\ -0.8378 & 1.3244 \end{bmatrix}$, $\mathbf{b}_1 \leftarrow [-0.1119, -0.8378]^\top$; $W_2 \leftarrow \begin{bmatrix} 1.7847 & -0.1473 \\ 0.2299 & 2.1572 \\ -1.0146 & -1.0100 \end{bmatrix}$,

$\mathbf{b}_2 \leftarrow [-0.2945, 0.3145, -0.0199]^\top$.

(2) $\mathbf{a}_1 = [0.3286, 0.9733]^\top$, $\mathbf{h}_1 = [0.5814, 0.7258]^\top$, $\mathbf{a}_2 = [0.6362, 2.0138, -1.3428]^\top$, $\hat{\mathbf{y}} = [0.1960, \mathbf{0.7770}, 0.0271]^\top$. New loss $-\ln 0.7770 = \mathbf{0.252}$ (down from 0.991).

(3) Forward $\approx 2 \cdot 2 + 3 \cdot 2 = 10$ multiplies; backward about twice that; the network has $2 \cdot 3 + 3 \cdot 3 = 15$ parameters — cost $O(\#\theta)$.

(4) One example only: this is SGD with $B = 1$. The step may hurt other parcels; a mini-batch gradient would be a lower-variance estimate.

Verification. The correct lane's probability rose from 0.371 to 0.777 and the loss fell, as first-order Taylor predicts for a moderate η .

Master Integration Problem 2: The Odyssey optimiser race

Problem. On $f = \frac{1}{2}(25x^2 + y^2)$ from $(1, 1)$, compare three steps of: GD ($\eta = 0.1$), AdaGrad ($\eta = 0.1$), RMSProp ($\eta = 0.1, \beta = 0.9$), Adam ($\eta = 0.1$, default β 's). Explain the pattern and recommend a method.

Solution.

Method	step 1	step 2	step 3
GD	(-1.5, 0.9)	(2.25, 0.81)	(-3.375, 0.729)
AdaGrad	(0.900, 0.900)	(0.833, 0.833)	(0.780, 0.780)
RMSProp	(0.684, 0.684)	(0.499, 0.499)	(0.369, 0.369)
Adam	(0.900, 0.900)	(0.800, 0.800)	(0.702, 0.702)

Interpretation. GD diverges in x ($r_x = -1.5$). All adaptive methods normalise per coordinate, so x and y move in lock-step. AdaGrad slows fastest (its sum only grows). RMSProp moves fastest early because of its uncorrected (inflated) first steps. Adam moves about η per step. For this ill-conditioned bowl any adaptive method succeeds; if plain GD must be used, $\eta < 0.08$ is mandatory.

Complexity. Adaptive methods cost $O(\#\theta)$ extra memory (one or two accumulator vectors) and $O(\#\theta)$ extra arithmetic per step — negligible next to backprop.

Master Integration Problem 3: From Lumen's perceptron to a trainable network

Problem. The rover's perceptron ended with $\mathbf{w} = [-1, 4, -2]$ (Chapter 3). (1) Replace the step by a sigmoid, $\hat{y} = \sigma(k \mathbf{w}^\top \mathbf{x})$. For $k = 1$ give the outputs on R1–R4. (2) Why is k like a “confidence knob”? (3) Scientists add a new rock class, needing a network $2 \rightarrow 4 \rightarrow 2$ (sigmoid hidden, softmax output). Count its parameters. (4) A colleague initialises all hidden weights to 0.1. Predict what happens.

Solution. (1) Scores 5, -5 , 5, $-3 \Rightarrow \sigma$: 0.993, 0.007, 0.993, 0.047 — same decisions as the perceptron (threshold $0.5 \leftrightarrow$ score 0).

(2) $\sigma(kz) \rightarrow$ step as $k \rightarrow \infty$ (Section 5.2.3): larger k makes predictions more extreme and, when wrong, saturated (tiny gradients).

(3) $4(2 + 1) + 2(4 + 1) = 12 + 10 = 22$.

(4) If the output weights are also equal, all 4 hidden units compute the same function and receive identical gradients forever: the layer acts as one neuron and cannot represent the new boundary (symmetry, Chapter 7). Use random initialisation.

10.2 Index of Course Paper Questions Solved in Part I

Paper	Topic	Where solved
Quiz 1 Q2	“NN boundary always non-linear” (False)	Ch. 2
Quiz 1 Q3	MP neuron statements I/II	Ch. 2
Quiz 1 Q9	MP neuron with inhibition	Ch. 2
Quiz 1 Q6	angle after perceptron update	Ch. 3
Quiz 1 Q10	PLA with separate bias	Ch. 3
Quiz 1 Q11	perceptron vs sigmoid, representation power	Ch. 4
Quiz 1 Q12	sigmoid Taylor approximation	Ch. 5
Quiz 1 Q15	sigmoid neuron gradient (-0.15)	Ch. 5
Quiz 1 Q4	learning rates on w^2	Ch. 5
Quiz 1 Q13, Q14	house price 150; softmax $a_1 = \ln 2$	Ch. 6
Quiz 1 Q16; ET	identity activations collapse	Ch. 6
Quiz 1 Q17	$\partial \mathcal{L} / \partial a_{L,\ell} = -0.01$	Ch. 7
Quiz 1 Q8	identical initialisation	Ch. 7
End-Term	sigmoid backprop, vanishing/exploding	Ch. 7
Quiz 1 Q5	matching momentum curves	Ch. 8
Quiz 1 Q7, Q18, Q19; ET	SGD/mini-batch/epochs; batch $32 \rightarrow 128$	Ch. 8
End-Term	$\frac{1}{2}(25x^2 + y^2)$: GD step, divergence	Ch. 8
End-Term	AdaGrad effective rates 0.104; reduced $25\times$ along x	Ch. 9

10.3 Part I Master Reference

10.3.1 Formula sheet

Formula	Meaning	Use when / assumptions
$y = \mathbf{1}[\sum_{\text{exc}} x_i \geq \theta]$, any inhibitor $\Rightarrow 0$	MP neuron	binary inputs, hand-set θ
$\hat{y} = \mathbf{1}[\mathbf{w}^\top \mathbf{x} \geq 0]$, $x_0 = 1$, $w_0 = -\theta$	perceptron	real inputs, linear boundary
$\mathbf{w} \leftarrow \mathbf{w} + y\mathbf{x}$ on mistake	PLA	score changes by $\pm \ \mathbf{x}\ ^2$
$k \leq 1/\delta^2$; $(R/\gamma)^2$	PLA mistake bound	separable data, $\mathbf{w}_0 = \mathbf{0}$
2^{2^n} ; 2^n hidden units, bias $-\mathbf{n}$	Boolean functions; universal perceptron net	inputs ± 1

Formula	Meaning	Use when / assumptions
$\sigma' = \sigma(1 - \sigma) \leq \frac{1}{4}; \sigma \approx \frac{1}{2} + \frac{z}{4}$	sigmoid facts	linearisation valid near 0
$\theta \leftarrow \theta - \eta \nabla \mathcal{L}$	gradient descent	first-order Taylor, small η
$(f - y)f(1 - f)x$	sigmoid + squared-error gradient	single example; sum over data
$\mathbf{a}_i = \mathbf{b}_i + W_i \mathbf{h}_{i-1}, \mathbf{h}_i = g(\mathbf{a}_i)$	forward pass	$W_i \in \mathbb{R}^{n_i \times n_{i-1}}$
$\sum n_i(n_{i-1} + 1)$	parameter count	fully connected layers
$\mathcal{L} = -\log \hat{y}_\ell$	cross-entropy	softmax output
$\nabla_{\mathbf{a}_L} \mathcal{L} = \hat{\mathbf{y}} - \mathbf{e}_\ell$	output gradient	softmax + CE (also $\hat{\mathbf{y}} - \mathbf{y}$ for linear + MSE)
$\nabla_{\mathbf{h}_i} = W_{i+1}^\top \nabla_{\mathbf{a}_{i+1}}, \nabla_{\mathbf{a}_i} = \nabla_{\mathbf{h}_i} \odot g'(\mathbf{a}_i)$	backprop recursion	element-wise activations
$\nabla_{W_i} = \nabla_{\mathbf{a}_i} \mathbf{h}_{i-1}^\top, \nabla_{\mathbf{b}_i} = \nabla_{\mathbf{a}_i}$	weight gradients	shape check $n_i \times n_{i-1}$
$e_{t+1} = (1 - \eta\lambda)e_t; \eta < 2/\lambda_{\max}$	GD on quadratic	local model of any smooth minimum
$\mathbf{u}_t = \beta \mathbf{u}_{t-1} + \nabla \mathbf{w}_t; \mathbf{w} - \eta \mathbf{u}_t$	momentum	oscillation decays as $\sqrt{\beta}^t$
gradient at $\mathbf{w}_t - \eta \beta \mathbf{u}_{t-1}$	NAG	smaller overshoot
$\lceil N/B \rceil$ steps/epoch; $\text{var} \propto 1/B$	mini-batch	batch GD: 1; SGD: N
$v += g^2; \eta/\sqrt{v + \epsilon}$	AdaGrad	rate only decreases
$v = \beta v + (1 - \beta)g^2$	RMSProp	rate can go up or down
$\Delta w = -\frac{\sqrt{u_{t-1} + \epsilon}}{\sqrt{v_t + \epsilon}} g$	AdaDelta	no η
$\hat{m} = \frac{m}{1 - \beta_1^t}, \hat{v} = \frac{v}{1 - \beta_2^t}, \frac{\eta \hat{m}}{\sqrt{\hat{v} + \epsilon}}$	Adam	first step = $\eta \text{sign } g$
$v = \max(\beta_2 v, g)$	AdaMax	no correction of v
$\eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2} (1 + \cos \frac{\pi(t \bmod (T+1))}{T})$	cosine annealing, restarts	escaping saddles

10.3.2 Recognition rules (all of Part I)

1. Inhibitory input on $\Rightarrow$ MP output 0. XOR / parity $\Rightarrow$ not one neuron.
2. Score exactly 0 $\Rightarrow$ predicted positive; negative point with score 0 is a mistake.
3. Pre-activation 0 $\Rightarrow \sigma = \frac{1}{2}, \sigma' = \frac{1}{4}$.
4. First-layer weights 0 $\Rightarrow$ output independent of input.
5. Identity hidden activations $\Rightarrow$ one affine map; no extra power.
6. Softmax + CE $\Rightarrow \hat{\mathbf{y}} - \mathbf{e}_\ell$; true-class entry = $\hat{y}_\ell - 1$.
7. Identical weights $\Rightarrow$ symmetric forever; functionally one neuron.
8. Quadratic + $\eta \Rightarrow$ factors $1 - \eta\lambda_i$; largest curvature diverges first.
9. Most ringing curve $\Rightarrow$ largest β .
10. Updates/epoch $\Rightarrow 1, N, \lceil N/B \rceil$.
11. AdaGrad first step $\Rightarrow$ rate $\eta/|g|$; Adam first step $\Rightarrow \eta \text{sign } g$.
12. Saddle point + decaying $\eta \Rightarrow$ cyclical / cosine restarts.

10.3.3 Top traps

- Forgetting to update the separate bias in PLA; mishandling the tie at 0.
- Treating “ 2^n hidden units” as necessary; confusing exact representation with approximation.
- Using W_{i+1} instead of $W_{i+1}^\top$; multiplying softmax gradient by $\hat{y}(1 - \hat{y})$ again.
- Believing a smaller gradient always lowers the loss, or that a larger batch guarantees a better minimum.
- Rounding N/B down; forgetting that SGD makes N updates per epoch.
- Confusing AdaGrad (sum) with RMSProp (decaying average); omitting Adam’s bias correction.

10.3.4 Debugging checklist: if your answer looks wrong

1. **Definitions:** which prediction rule (≥ 0 or > 0)? Which label set (0/1 or ± 1)?
2. **Shapes:** does every matrix product conform? Does ∇_{W_i} have the shape of W_i ?
3. **Signs:** descent subtracts the gradient; negative-class PLA updates subtract $\mathbf{x}$.
4. **Sanity:** probabilities sum to 1; $\nabla_{\mathbf{a}_L}$ sums to 0; loss decreases after a small GD step.
5. **Indexing:** Adam's t starts at 1; momentum's $\mathbf{u}_{-1} = \mathbf{0}$.
6. **Rounding:** carry 4–5 significant figures; round only at the end to the requested decimals.

End of Part I. Part II continues with Weeks 5–7: autoencoders, regularisation, and better training of deep networks.

Part II

Representation, Regularisation and Better Training (Weeks 5–7)

Part II Road Map

Part I built networks and taught them to descend a loss. Part II asks three sharper questions.

1. **What does a network learn inside?** Autoencoders (Chapter 11) learn a representation with no labels at all, and in the linear case they rediscover PCA.
2. **How do we stop a network from memorising?** Regularisation (Chapter 12) trades a little bias for a large drop in variance.
3. **How do we make deep networks train at all?** Pre-training, better activations, careful initialisation and batch normalisation (Chapter 13) attack the vanishing-gradient and saturation problems diagnosed in Section 7.6.

The example universe, extended. The five worlds return, and one sub-system joins them:

ECHO	The ODYSSEY spacecraft’s telemetry compressor. It must squeeze sensor frames into a few numbers for the slow radio link and rebuild them on Earth. (Autoencoders, PCA, denoising.)
PULSE	Now with only a few dozen patients and many features: the textbook overfitting setting. (Bias–variance, L2, noise, early stopping, ensembles.)
LUMEN	The rover’s camera images: rotations and lighting changes that must not change the label. (Data augmentation, parameter sharing.)
ATLAS	The parcel router grows to many layers. (Dropout, activations, initialisation, batch normalisation.)

New feature: Concept Gauntlets. Every Part II chapter ends with a set of short, invented questions that are *simple to state but easy to get wrong*. They test whether you understand *why* a rule holds, not just the rule. Try each before reading its answer.

Autoencoders and Their Relation to PCA

Chapter Overview

An autoencoder is a network trained to *reproduce its own input*. That sounds pointless until we force the input through a narrow middle layer (or otherwise constrain the middle layer): to reconstruct well, the network must discover the few directions of variation that really matter. This chapter defines autoencoders, chooses losses for binary and real data, proves that a linear autoencoder with squared error learns exactly the PCA subspace, and studies four ways of regularising autoencoders: weight penalties/tying, denoising, sparsity and contraction.

Learning objectives. Write encoder and decoder equations; pick the decoder activation and loss from the data type; distinguish under- and overcomplete autoencoders; state and justify the PCA-equivalence conditions; compute reconstruction error from eigenvalues; compute denoising corruption statistics, the sparsity KL penalty and its gradient, and the contractive penalty.

Prerequisites. Chapters 6–7; eigenvalues/eigenvectors of a symmetric matrix; the idea of PCA as “directions of maximum variance”.

11.1 Working Principle and Intuition

ECHO receives 4-bit status frames from the spacecraft sensors but the radio link carries only 2 numbers per frame. If the four bits were independent, no 2-number code could preserve them. But real telemetry is *redundant*: some bits move together. An autoencoder learns a 2-number code $\mathbf{h}$ from which the 4 bits can be rebuilt. The code is useful precisely because it is forced to capture the structure of the data.

11.2 Formal Definition

Definition: Autoencoder

For input $\mathbf{x}_i \in \mathbb{R}^n$,

$$\mathbf{h} = g(W\mathbf{x}_i + \mathbf{b}) \quad (\text{encoder}), \quad \hat{\mathbf{x}}_i = f(W^*\mathbf{h} + \mathbf{c}) \quad (\text{decoder}),$$

with $W \in \mathbb{R}^{k \times n}$, $W^* \in \mathbb{R}^{n \times k}$, $\mathbf{b} \in \mathbb{R}^k$, $\mathbf{c} \in \mathbb{R}^n$. Training minimises a reconstruction loss between $\hat{\mathbf{x}}_i$ and $\mathbf{x}_i$ over the data. If $k < n$ the autoencoder is **undercomplete**; if $k \geq n$ it is **overcomplete**.

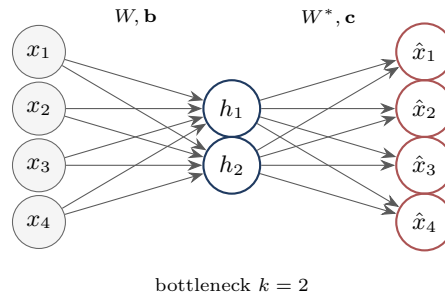


Figure 11.1: The ECHO undercomplete autoencoder: $4 \rightarrow 2 \rightarrow 4$.

11.2.1 Choosing the decoder and the loss

The pairing mirrors Chapter 6:

Input type	Decoder f	Loss for one example
Binary $x_j \in \{0, 1\}$	sigmoid (outputs in $(0, 1)$)	cross-entropy $-\sum_j [x_j \log \hat{x}_j + (1 - x_j) \log(1 - \hat{x}_j)]$
Real $x_j \in \mathbb{R}$	linear	squared error $\sum_j (\hat{x}_j - x_j)^2$

The encoder g is typically sigmoid, tanh or ReLU. Training is ordinary backpropagation: the “target” is the input itself.

11.2.2 Under- versus overcomplete

- **Undercomplete** ($k < n$): the bottleneck forces compression. If the network reconstructs well, $\mathbf{h}$ captures the important characteristics of the data — a learned dimensionality reduction.
- **Overcomplete** ($k \geq n$): nothing stops the network from learning the trivial identity (e.g. copy x_j into h_j and back). Perfect reconstruction, useless code. Overcomplete autoencoders are useful *only with regularisation* (Section 11.4).

Worked Example 11.1: Echo encodes one frame, Level 2

Problem. Frame $\mathbf{x} = [1, 0, 1, 1]^\top$. Encoder: sigmoid, $W = \begin{bmatrix} 1 & -1 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}$, $\mathbf{b} = [-1, -1]^\top$. Decoder: sigmoid with *tied weights* $W^* = W^\top$, $\mathbf{c} = \mathbf{0}$. Compute $\mathbf{h}$, $\hat{\mathbf{x}}$ and the cross-entropy reconstruction loss.

Strategy. Binary data $\Rightarrow$ sigmoid decoder and cross-entropy (table above). Tied weights: decoder matrix is the encoder’s transpose.

Solution. $\mathbf{a} = W\mathbf{x} + \mathbf{b} = [1 - 0 + 1 + 0 - 1, 0 + 0 + 1 + 1 - 1]^\top = [1, 1]^\top$, so $\mathbf{h} = [0.7311, 0.7311]^\top$. $W^\top \mathbf{h} = [0.7311, -0.7311, 1.4621, 0.7311]^\top$, so $\hat{\mathbf{x}} = \sigma(\cdot) = [0.675, 0.325, 0.812, 0.675]^\top$. Loss $= -[\ln 0.675 + \ln(1 - 0.325) + \ln 0.812 + \ln 0.675] = 0.393 + 0.393 + 0.208 + 0.393 = \mathbf{1.387}$.

Interpretation. Every bit is on the correct side of $\frac{1}{2}$ (bits 1, 3, 4 above, bit 2 below), so the frame is decoded correctly by thresholding, but the loss is still positive because the network is not yet confident. Training would push these probabilities towards 0/1.

Exam Shortcut. For a 0 bit, the CE term is $-\ln(1 - \hat{x}_j)$ — the most common slip is writing $-\ln \hat{x}_j$ for every bit.

11.3 Linear Autoencoders Learn PCA

11.3.1 The statement

Theorem 11.1 (Autoencoder–PCA equivalence). *Suppose (i) the encoder is linear, (ii) the decoder is linear, (iii) the loss is squared error, and (iv) the inputs are normalised,*

$$\hat{x}_{ij} = \frac{1}{\sqrt{m}} \left(x_{ij} - \frac{1}{m} \sum_{i'=1}^m x_{i'j} \right),$$

(m examples; i.e. each feature is centred and the data scaled by $1/\sqrt{m}$ so that $X^\top X$ is the covariance matrix). Then the optimal k -unit autoencoder projects onto the span of the top- k eigenvectors of $X^\top X$: it learns the same subspace as PCA.

11.3.2 Why it is true (proof sketch)

Stack the (normalised) examples as rows of $X \in \mathbb{R}^{m \times n}$ and the codes as rows of $H \in \mathbb{R}^{m \times k}$. With linear maps and no biases, the reconstruction is $\hat{X} = HW^{*\top}$, and the loss is $\|X - HW^{*\top}\|_F^2$.

1. $HW^{*\top}$ is a product through a k -dimensional layer, so its rank is at most k .

2. **Eckart–Young theorem:** among all matrices of rank $\leq k$, the closest to X in Frobenius norm is the truncated SVD $U_k \Sigma_k V_k^\top$, where V_k holds the top- k right singular vectors of X — which are the top- k eigenvectors of $X^\top X$.
3. This optimum is achievable: take the encoder $W = V_k^\top$ (so $H = X V_k$) and the decoder $W^* = V_k$; then $HW^*\top = X V_k V_k^\top = U_k \Sigma_k V_k^\top$.

□

Common Trap: Same subspace, not necessarily the same basis

For any invertible $A \in \mathbb{R}^{k \times k}$, encoder $A^{-1}V_k^\top$ and decoder $V_k A$ give the *same* reconstruction. So a trained linear autoencoder spans the principal subspace, but its individual hidden units need not be the principal components themselves (they need not be orthogonal or ordered by variance). “AE = PCA” is a statement about the subspace and the reconstruction error.

Worked Example 11.2: Echo discovers the principal direction, Level 3

Problem. Four centred 2-D sensor readings: $(2, 1), (-2, -1), (1, 2), (-1, -2)$. A linear autoencoder with one hidden unit and squared error is trained to optimality (no $1/\sqrt{m}$ scaling). (a) Which direction does it learn? (b) What is the minimum total reconstruction error? (c) Reconstruct $(2, 1)$.

Strategy. By Theorem 11.1, find the top eigenvector of $X^\top X$; the error equals the sum of the discarded eigenvalues.

Solution. $X^\top X = \begin{bmatrix} 4+4+1+1 & 2+2+2+2 \\ 8 & 1+1+4+4 \end{bmatrix} = \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}$. Eigenvalues 10 ± 8 : $\lambda_1 = 18$ with

$\mathbf{u}_1 = \frac{1}{\sqrt{2}}[1, 1]^\top$; $\lambda_2 = 2$ with $\mathbf{u}_2 = \frac{1}{\sqrt{2}}[1, -1]^\top$.

(a) Direction $\mathbf{u}_1$ (the diagonal $x_1 = x_2$).

(b) Error $= \lambda_2 = 2$.

(c) $h = \mathbf{u}_1^\top [2, 1]^\top = 3/\sqrt{2}$; $\hat{\mathbf{x}} = h\mathbf{u}_1 = [1.5, 1.5]^\top$, error $0.5^2 + 0.5^2 = 0.5$.

Verification. Each of the four points has error 0.5; total $2 = \lambda_2$ ✓.

Interpretation. The two sensors are correlated, so their *average* carries most information; the discarded coordinate is their disagreement. With the course normalisation ($1/\sqrt{m}$, $m = 4$), the matrix and the error are simply divided by 4.

Why-Chain: Why do all four conditions matter?

Examiner: Why must the *decoder* be linear?

Student: Eckart–Young concerns linear low-rank reconstructions. A non-linear decoder can reconstruct along curved surfaces and beat PCA; it is then non-linear dimensionality reduction.

Examiner: And the encoder — a sigmoid encoder with a linear decoder?

Student: The code is then a squashed version of the input; the optimal reconstruction is no longer the orthogonal projection in general. Near the origin, where σ is almost linear (Section 5.2.3), it behaves approximately like PCA.

Examiner: Why centre the data?

Student: PCA is about variance around the mean. Without centring (and without biases), the top eigenvector of $X^\top X$ tends to point at the mean itself rather than along the direction of spread.

Examiner: So what is the deeper principle?

Student: An autoencoder is PCA generalised: linear + squared error + centred data gives exactly PCA; relaxing any condition gives a strictly richer (but less analysable) model.

11.4 Regularisation in Autoencoders

The danger in every autoencoder — especially an overcomplete one — is learning to copy. Each regulariser below makes copying *costly* or *impossible*.

11.4.1 Weight decay and weight tying

Add $\lambda \|\theta\|^2$ to the loss (Chapter 12), and/or **tie** the weights: $W^* = W^\top$. Tying halves the weight count and restricts the model. For $n = 784$, $k = 64$: untied $784 \cdot 64 \cdot 2 + 64 + 784 = 101,200$ parameters; tied $784 \cdot 64 + 64 + 784 = 51,024$.

11.4.2 Denoising autoencoders

Definition: Denoising autoencoder

Corrupt each input by a stochastic process, e.g. *masking noise*:

$$P(\tilde{x}_{ij} = 0 \mid x_{ij}) = q, \quad P(\tilde{x}_{ij} = x_{ij} \mid x_{ij}) = 1 - q,$$

(or Gaussian noise $\tilde{\mathbf{x}} = \mathbf{x} + \epsilon$ for real data). Feed $\tilde{\mathbf{x}}$ to the encoder but compute the loss against the **clean** $\mathbf{x}$.

Copying is now useless: copying $\tilde{\mathbf{x}}$ reproduces the corruption, not $\mathbf{x}$. To succeed, the network must learn how features relate (e.g. “bit 3 is usually on when bit 4 is on”), i.e. the structure of the data distribution. Trained on images, denoising autoencoders learn stroke- and edge-like filters rather than pixel copies.

Common Trap: Loss against which input?

The loss is $\mathcal{L}(\hat{\mathbf{x}}, \mathbf{x})$ with the *clean* input. Using $\mathcal{L}(\hat{\mathbf{x}}, \tilde{\mathbf{x}})$ turns it back into an ordinary autoencoder on noisy data, which can again learn the identity. Also, masking noise with $q = 0$ is just a plain autoencoder.

11.4.3 Sparse autoencoders

A hidden sigmoid unit is “active” near 1 and “inactive” near 0. A sparse autoencoder wants each unit to be active only rarely, so that each unit specialises.

Definition: Sparsity penalty

The average activation of hidden unit l over m examples is $\hat{\rho}_l = \frac{1}{m} \sum_{i=1}^m h_l(\mathbf{x}_i)$. With target sparsity ρ (e.g. 0.005–0.05),

$$\Omega(\theta) = \sum_{l=1}^k \left[\rho \ln \frac{\rho}{\hat{\rho}_l} + (1 - \rho) \ln \frac{1 - \rho}{1 - \hat{\rho}_l} \right], \quad \hat{\mathcal{L}} = \mathcal{L} + \Omega.$$

This is the KL divergence between Bernoulli(ρ) and Bernoulli($\hat{\rho}_l$): zero iff $\hat{\rho}_l = \rho$, positive otherwise.

Its derivative is

$$\frac{\partial \Omega}{\partial \hat{\rho}_l} = -\frac{\rho}{\hat{\rho}_l} + \frac{1 - \rho}{1 - \hat{\rho}_l},$$

which is positive when $\hat{\rho}_l > \rho$ (pushes activation down) and negative when $\hat{\rho}_l < \rho$ (pushes it up). Via $\partial \hat{\rho}_l / \partial W_{lj} = \frac{1}{m} \sum_i h_l(1 - h_l) x_{ij}$ it enters backpropagation.

Worked Example 11.3: How much does a lazy unit pay? Level 2

Problem. Target $\rho = 0.05$; a hidden unit’s average activation is $\hat{\rho} = 0.2$. Compute its penalty and $\partial \Omega / \partial \hat{\rho}$.

Solution. $\Omega = 0.05 \ln \frac{0.05}{0.2} + 0.95 \ln \frac{0.95}{0.8} = 0.05(-1.3863) + 0.95(0.1719) = -0.0693 + 0.1633 = \mathbf{0.0939}$.
 $\frac{\partial \Omega}{\partial \hat{\rho}} = -\frac{0.05}{0.2} + \frac{0.95}{0.8} = -0.25 + 1.1875 = \mathbf{0.9375} > 0$.

Interpretation. The first term is negative — do not panic: KL is non-negative only as a whole. The positive derivative says “decrease $\hat{\rho}$ ”: the unit fires too often.

11.4.4 Contractive autoencoders

Definition: Contractive penalty

$$\Omega(\theta) = \|J_{\mathbf{x}}(\mathbf{h})\|_F^2 = \sum_{j=1}^n \sum_{l=1}^k \left(\frac{\partial h_l}{\partial x_j} \right)^2,$$

the squared Frobenius norm of the Jacobian of the code with respect to the input.

If $\partial h_l / \partial x_j = 0$, then h_l ignores x_j — small changes in the input do not change the code. **The tension:** the reconstruction loss wants $\mathbf{h}$ to be sensitive to the input (to capture it), the penalty wants it insensitive. The compromise: $\mathbf{h}$ becomes sensitive only along directions that actually vary in the training data (the data “manifold”) and insensitive to everything else — the same spirit as PCA keeping high-variance directions.

For a sigmoid encoder $h_l = \sigma(\sum_j W_{lj}x_j + b_l)$: $\frac{\partial h_l}{\partial x_j} = h_l(1 - h_l)W_{lj}$, so

$$\Omega = \sum_l [h_l(1 - h_l)]^2 \sum_j W_{lj}^2.$$

Worked Example 11.4: Contractive penalty for the Echo frame, Level 2

Problem. Using the ECHO worked example ($h_1 = h_2 = 0.7311$, rows of W : $[1, -1, 1, 0]$ and $[0, 0, 1, 1]$), compute Ω .

Solution. $h(1 - h) = 0.1966$, squared 0.03866. Row sums of squares: 3 and 2. $\Omega = 0.03866(3 + 2) = \mathbf{0.193}$.

Interpretation. The penalty grows with the weights and *shrinks when units saturate* ($h(1 - h) \rightarrow 0$) — a saturated code is insensitive by construction.

11.5 Comparison of Autoencoder Variants

Variant	How copying is prevented	Extra term / change	What $\mathbf{h}$ captures
Undercomplete	bottleneck $k < n$	none	main directions (PCA if linear)
Weight decay / tied	small or shared weights	$\lambda \ \theta\ ^2$; $W^* = W^\top$	smoother code
Denoising	input corrupted, target clean	loss $\mathcal{L}(\hat{\mathbf{x}}, \mathbf{x})$ with $\tilde{\mathbf{x}}$ in	dependencies between features
Sparse	few units active per input	$\sum_l \text{KL}(\rho \ \hat{\rho}_l)$	specialised, part-based units
Contractive	code insensitive to input	$\ J_{\mathbf{x}}(\mathbf{h})\ _F^2$	only data-manifold directions

11.6 Exam Toolkit

Exam Tip: Recognition rules

- Binary input $\Rightarrow$ sigmoid decoder + CE; real input $\Rightarrow$ linear decoder + squared error.
- “Linear encoder, linear decoder, squared error, normalised input” $\Rightarrow$ PCA subspace; error = $\sum$ discarded eigenvalues.
- “Overcomplete, no regularisation” $\Rightarrow$ can learn identity; useless code.
- “Denoising” $\Rightarrow$ corrupt input, compare with clean input; expected corrupted entries = nq .
- “ $\hat{\rho}$, ρ , KL” $\Rightarrow$ sparse AE; sign of $-\rho/\hat{\rho} + (1 - \rho)/(1 - \hat{\rho})$ tells the push.
- “Jacobian / Frobenius” $\Rightarrow$ contractive; for sigmoid, $h(1 - h)W_{lj}$.

Quick Recall: Autoencoders

$\mathbf{h} = g(W\mathbf{x} + \mathbf{b})$, $\hat{\mathbf{x}} = f(W^*\mathbf{h} + \mathbf{c})$. Linear + squared error + normalised input = PCA subspace (Eckart–Young). Regularise to prevent copying: tie/decay weights, denoise ($\tilde{\mathbf{x}}$ in, $\mathbf{x}$ as target), sparsify (KL to ρ), contract ($\|J\|_F^2$).

11.7 Concept Gauntlet

G1. A linear autoencoder with $k = n$ reaches zero training error. Has it “learned PCA”?

Answer: Trivially — the top- n subspace is all of $\mathbb{R}^n$. Nothing was compressed, so nothing was learned about the data's structure.

- G2.** Can a sigmoid-*decoder* autoencoder reach zero cross-entropy on binary data?

Answer: Only in the limit: σ never outputs exactly 0 or 1, so the loss approaches 0 as pre-activations go to $\pm\infty$ but never reaches it.

- G3.** In a denoising AE with $q = 0.3$ and $n = 100$ inputs, how many inputs are zeroed on average? Can a zeroed input stay “correct”?

Answer: 30. Yes — an input that was already 0 is unchanged by masking, so for sparse binary data far fewer than 30 bits actually flip.

- G4.** Two linear autoencoders trained on the same data have different hidden weights. Is one of them wrong?

Answer: Not necessarily: any invertible re-mixing A of the code gives the same reconstruction; compare their spanned subspaces instead.

- G5.** A sparse AE uses $\rho = 0.5$. What does the penalty now encourage?

Answer: Units active half the time on average — not sparsity at all. Sparsity needs small ρ .

- G6.** Why can't we apply the KL sparsity penalty to a ReLU (unbounded) code as written?

Answer: $\hat{\rho}_i$ must lie in $(0, 1)$ to be a Bernoulli parameter; ReLU averages can exceed 1. Use an L_1 activity penalty instead.

- G7.** The contractive penalty is zero. Name two very different ways this can happen.

Answer: All weights zero (code ignores input entirely — terrible reconstruction), or all units fully saturated. Hence the penalty only makes sense together with the reconstruction loss.

- G8.** Data are *not* centred and the linear AE has no biases. What does the top direction tend to become?

Answer: The direction of the data mean (it dominates $X^\top X$), not the direction of maximum spread.

11.8 Practice Problems

1. **[FOUNDATION]** Count parameters of an untied $100 \rightarrow 10 \rightarrow 100$ autoencoder.

Answer: $10 \cdot 101 + 100 \cdot 11 = 1010 + 1100 = 2110$.

2. **[INTERMEDIATE]** Covariance eigenvalues are 6, 3, 1. Reconstruction error (in the same units) of the best 1-unit and 2-unit linear AEs? Fraction of variance kept by 2 units?

Answer: 4 and 1; $9/10 = 90\%$.

3. **[EXAM]** $\rho = 0.1$, $\hat{\rho} = 0.05$. Sign of $\partial\Omega/\partial\hat{\rho}$ and its meaning.

Answer: $-2 + 0.9474 = -1.05 < 0$: increase activation.

4. **[ADVANCED]** Show that for a tanh encoder the contractive penalty is $\sum_l (1 - h_l^2)^2 \sum_j W_{lj}^2$.

5. **[CHALLENGE]** Explain why an overcomplete denoising autoencoder can learn a useful code even though $k > n$.

Hint: Identity mapping of $\tilde{\mathbf{x}}$ is penalised because the target is the clean $\mathbf{x}$.

Regularisation: Bias, Variance and the Art of Not Memorising

Chapter Overview

Modern networks often have more parameters than training examples. They can fit the training set perfectly — including its noise — and then fail on new data. This chapter makes the failure precise with the **bias–variance decomposition**, and then studies the course’s toolbox for controlling variance: L_2 regularisation, data augmentation, parameter sharing and tying, input noise, early stopping, ensembles and dropout. For each we ask the same question: *what does it cost in bias, and what does it buy in variance?*

Learning objectives. Define and decompose expected test error; predict how data size and model complexity move bias and variance; derive the L_2 update and its eigen-direction shrinkage; prove that input noise equals L_2 for a linear model; apply early stopping with patience; derive the ensemble error formula; compute dropout expectations and test-time scaling.

12.1 Bias and Variance

12.1.1 Intuition

The PULSE team fits risk curves on many different random samples of 30 patients. A *simple* model (a straight line) gives nearly the same line each time — but all of them miss the true curved relationship: **high bias, low variance**. A *complex* model (degree-25 polynomial) wiggles through every training patient; its average over samples is close to the truth, but each individual fit is wildly different: **low bias, high variance**.

12.1.2 Definitions

Let $y = f(x) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$. A model $\hat{f}$ trained on a random training set is itself random. At a fixed test point x :

$$\text{Bias } \hat{f}(x) = \mathbb{E}[\hat{f}(x)] - f(x), \quad \text{Var } \hat{f}(x) = \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2].$$

The expectation is over training sets.

12.1.3 The decomposition (derivation)

Write $\bar{f} = \mathbb{E}[\hat{f}(x)]$. Then $y - \hat{f} = (f - \bar{f}) + (\bar{f} - \hat{f}) + \varepsilon$. Squaring and taking expectations, every cross term vanishes: ε is independent of $\hat{f}$ and has mean 0, and $\mathbb{E}[\bar{f} - \hat{f}] = 0$. Hence

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(f - \bar{f})^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\bar{f} - \hat{f})^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{irreducible}}.$$

Worked Example 12.1: Pulse: measuring bias and variance, Level 2

Problem. The true risk at a test patient is $f(x_0) = 2.5$ and noise variance $\sigma^2 = 0.04$. Three models trained on three independent samples predict 2.0, 2.4, 1.6. Estimate bias², variance and expected squared error.

Solution. $\bar{f} = 2.0$. Bias = $2.0 - 2.5 = -0.5$, bias² = 0.25. Variance = $\frac{1}{3}(0^2 + 0.4^2 + 0.4^2) = 0.1067$. Expected error = $0.25 + 0.1067 + 0.04 = \mathbf{0.397}$.

Interpretation. Here bias dominates: the model family is too simple or too constrained. Regularising it harder would make things *worse*.

Course Paper Question: Quiz 2, more data for a degree-25 polynomial

Training data is increased from the same distribution; architecture unchanged. **Answer: variance tends to decrease, bias stays about the same.**

Why. Bias is a property of the model family (what it can represent on average); more samples do not change the family. Variance measures sensitivity to the particular sample; with more data, fits on different samples agree more. Neither becomes zero (bias is fixed; σ^2 is irreducible anyway).

12.1.4 Train error, test error and model complexity

As model complexity grows, training error keeps falling, but test error first falls (bias shrinking) and then rises (variance growing): the classic U-shaped test curve. Training error is an *optimistic* estimate of test error. For squared error one can show (via Stein's lemma)

$$\mathbb{E}[\text{test err}] = \mathbb{E}[\text{train err}] + \frac{2\sigma^2}{n} \sum_{i=1}^n \frac{\partial \hat{f}(x_i)}{\partial y_i},$$

so the optimism is proportional to how *sensitive* the fitted values are to their own training labels — exactly what complex models are. Regularisation therefore minimises $\mathcal{L}_{\text{train}}(\theta) + \Omega(\theta)$, where Ω is large for complex (sensitive) models.

12.2 L_2 Regularisation (Weight Decay)**Definition: L_2 regularisation**

$$\tilde{\mathcal{L}}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \frac{\alpha}{2} \|\mathbf{w}\|^2, \quad \nabla \tilde{\mathcal{L}} = \nabla \mathcal{L} + \alpha \mathbf{w},$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta(\nabla \mathcal{L}(\mathbf{w}_t) + \alpha \mathbf{w}_t) = (1 - \eta\alpha) \mathbf{w}_t - \eta \nabla \mathcal{L}(\mathbf{w}_t).$$

Each step first *decays* the weights by $(1 - \eta\alpha)$, then takes the usual step — hence “weight decay”. (Biases are usually not penalised.)

Course Paper Question: End-Term, one regularised step

$J(w) = L(w) + \frac{\lambda}{2} w^2$, $w_t = 3$, $\partial L / \partial w = 4$, $\lambda = 0.1$, $\eta = 0.05$. Find w_{t+1} .

Solution. $w_{t+1} = 3 - 0.05(4 + 0.1 \cdot 3) = 3 - 0.05(4.3) = \mathbf{2.785}$.

Check via decay form. $(1 - 0.005) \cdot 3 - 0.05 \cdot 4 = 2.985 - 0.2 = 2.785 \checkmark$.

12.2.1 What L_2 does geometrically

Approximate the unregularised loss near its minimum $\mathbf{w}^*$ by a quadratic with Hessian $H = Q\Lambda Q^\top$ (eigenvalues λ_i , eigenvectors in Q). Setting $\nabla \tilde{\mathcal{L}} = H(\mathbf{w} - \mathbf{w}^*) + \alpha \mathbf{w} = 0$ gives

$$\tilde{\mathbf{w}} = (H + \alpha I)^{-1} H \mathbf{w}^* = Q(\Lambda + \alpha I)^{-1} \Lambda Q^\top \mathbf{w}^*.$$

In the eigenbasis, component i is multiplied by

$$\frac{\lambda_i}{\lambda_i + \alpha} \quad \begin{cases} \approx 1 & \lambda_i \gg \alpha \text{ (important direction: kept)} \\ \approx 0 & \lambda_i \ll \alpha \text{ (unimportant direction: shrunk away)} \end{cases}$$

The quantity $\sum_i \lambda_i / (\lambda_i + \alpha)$ acts as the *effective number of parameters*.

Worked Example 12.2: Which weights survive? Level 3

Problem. In the Hessian eigenbasis, $\mathbf{w}^* = [3, 2]^\top$ with curvatures $\lambda = (9, 1)$. Find the L_2 -regularised optimum for $\alpha = 1$ and the effective number of parameters.

Solution. Factors $\frac{9}{10} = 0.9$ and $\frac{1}{2} = 0.5$: $\tilde{\mathbf{w}} = [2.7, 1.0]^\top$. Effective parameters $0.9 + 0.5 = 1.4$ (out of 2).

Interpretation. The loss is steep along direction 1 (moving it costs a lot of training loss), so L_2 barely touches it. Along the flat direction 2, the data hardly care, so the penalty wins and halves it. L_2 removes weight *where the data are not paying for it*.

12.3 Dataset Augmentation

Create new labelled examples by applying transformations that *do not change the label*: for LUMEN’s rock images, rotation, small shifts, scaling, blur, brightness changes, added noise. More (effective) data lowers variance. It works best for images and speech, where invariances are known.

Common Trap: Label-changing “augmentations”

Rotating a handwritten “6” by 180° gives a “9”; horizontally flipping “b” gives “d”; flipping a road sign with text breaks it. An augmentation is valid only if the task is invariant to it.

12.4 Parameter Sharing and Tying

- **Sharing:** the same weights are applied at many positions — a CNN filter slides over every image location (Part III). Far fewer parameters, and a built-in assumption (“a feature is the same feature wherever it appears”).
- **Tying:** two parts of a model are constrained to be equal or related, e.g. $W^* = W^\top$ in autoencoders (Section 11.4).

Both reduce the effective number of free parameters, hence variance.

12.5 Injecting Noise

12.5.1 Noise at the input = L_2 for a linear model

Let $\hat{y} = \sum_i w_i x_i$ and corrupt the input: $\tilde{x}_i = x_i + \varepsilon_i$ with independent $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$. Then $\tilde{y} = \hat{y} + \sum_i w_i \varepsilon_i$ and

$$\mathbb{E}[(\tilde{y} - y)^2] = \mathbb{E}[(\hat{y} - y)^2] + 2\mathbb{E}\left[(\hat{y} - y) \sum_i w_i \varepsilon_i\right] + \mathbb{E}\left[\left(\sum_i w_i \varepsilon_i\right)^2\right] = \mathbb{E}[(\hat{y} - y)^2] + \sigma^2 \sum_i w_i^2.$$

The middle term is 0 (noise independent, mean 0); in the last, cross terms vanish and $\mathbb{E}[\varepsilon_i^2] = \sigma^2$. So training on noisy inputs is minimising squared error + $\sigma^2 \|\mathbf{w}\|^2$: L_2 with $\alpha/2 = \sigma^2$.

Worked Example 12.3: Hidden penalty of noisy sensors, Level 2

Problem. A linear PULSE scorer has $\mathbf{w} = [2, -1]$ and its sensors add independent noise of variance 0.1. By how much does the expected squared error increase?

Solution. $\sigma^2 \|\mathbf{w}\|^2 = 0.1 \cdot (4 + 1) = \mathbf{0.5}$.

Interpretation. Large weights amplify sensor noise. A model trained on noisy inputs learns to keep weights small — exactly what L_2 does explicitly.

12.5.2 Noise at the output (label smoothing)

With k classes, replace the hard target $\mathbf{e}_\ell$ by $1 - \varepsilon$ on the true class and $\varepsilon/(k - 1)$ on the others. The network is no longer rewarded for pushing its logits to infinity, which curbs overconfidence (and saturation).

12.6 Early Stopping

Definition: Early stopping with patience p

Track validation error after each epoch (or step). If it has not improved for p consecutive checks, stop and return the parameters from the *best* check (i.e. p checks earlier).

Worked Example 12.4: When does Pulse stop? Level 2

Problem. Validation errors by epoch: 0.90, 0.70, 0.62, 0.60, 0.61, 0.63, 0.64, 0.58, ... Patience $p = 3$. At which epoch does training stop, and which model is returned?

Solution. Best so far: epoch 4 (0.60). Epochs 5, 6, 7 do not improve $\Rightarrow$ after epoch 7 the patience is exhausted: **stop at epoch 7, return epoch 4**. The later improvement at epoch 8 is never seen.

Interpretation. Early stopping is a heuristic; too small a patience can stop before a later dip. It costs nothing to combine with other regularisers and is used almost universally.

Beyond the Core Syllabus: Early stopping behaves like L_2

For GD from $\mathbf{w}_0 = \mathbf{0}$ on a quadratic, after t steps component i (eigenbasis) equals $[1 - (1 - \eta\lambda_i)^t]w_i^*$. High-curvature directions are learned in a few steps; low-curvature ones are still small when we stop. This matches L_2 's factor $\lambda_i/(\lambda_i + \alpha)$ with $\alpha \approx 1/(\eta t)$. E.g. $\eta = 0.1$, $t = 20$, $\lambda = (9, 1, 0.1)$: GD factors (1.00, 0.88, 0.18); L_2 factors with $\alpha = 0.5$: (0.95, 0.67, 0.17) — qualitatively the same. *Training time is a regulariser.*

12.7 Ensemble Methods and Bagging

12.7.1 Why averaging helps: the error formula

Let k models have errors $\varepsilon_1, \dots, \varepsilon_k$ at a test point with $\mathbb{E}[\varepsilon_i^2] = V$ and $\mathbb{E}[\varepsilon_i \varepsilon_j] = C$ ($i \neq j$). The averaged model's error is $\frac{1}{k} \sum_i \varepsilon_i$, so

$$\text{mse}_{\text{ens}} = \mathbb{E}\left[\left(\frac{1}{k} \sum_i \varepsilon_i\right)^2\right] = \frac{1}{k^2} (kV + k(k-1)C) = \boxed{\frac{1}{k}V + \frac{k-1}{k}C}.$$

- $C = V$ (perfectly correlated errors): $\text{mse} = V$ — no gain at all.
- $C = 0$ (uncorrelated): $\text{mse} = V/k$ — error falls like $1/k$.
- $k \rightarrow \infty$: $\text{mse} \rightarrow C$ — correlation sets the floor.

Bagging makes errors less correlated by training each model on a bootstrap sample (drawn with replacement). A given example is left out of a bootstrap sample of size N with probability $(1 - \frac{1}{N})^N \approx e^{-1} \approx 0.368$, so each model sees about 63% distinct examples.

Course Paper Question: Quiz 2, bagging covariance

$\text{mse} = \frac{1}{k}V + \frac{k-1}{k}C$, $V = 12$. (a) $k = 2$, $-V \leq C \leq V$: which C minimises the error? (b) $k \rightarrow \infty$ with $C = C_0 > 0$: limit?

Solution. (a) $\text{mse} = 6 + C/2$, increasing in C : minimised at $C = -V$ (giving 0 — the two errors cancel exactly). (b) C_0 .

Extension. $k = 4$, $C = 3$: $\text{mse} = 3 + \frac{3}{4} \cdot 3 = 5.25$ vs a single model's 12.

12.8 Dropout

12.8.1 Working principle

Ensembles work but training many large networks is expensive. Dropout trains an *implicit* ensemble of exponentially many “thinned” networks that share weights.

Definition: Dropout

During training, for each example (or mini-batch) retain each unit independently with probability p (typically $p = 0.5$ for hidden units, $p \approx 0.8$ for inputs) and remove the others with all their connections. Forward and backward passes use this thinned network; only the retained units' parameters receive gradients. At test time use the full network with each unit's outgoing weights multiplied by p .

12.8.2 Why scale by p at test time

A retained-or-not unit contributes mwh with $m \sim \text{Bernoulli}(p)$, so its expected contribution during training is pwh . Scaling weights by p at test time makes the full network's output equal to the *expected* output of the thinned networks. (Modern implementations use **inverted dropout**: divide by p during training and use raw weights at test time — same expectations.)

Worked Example 12.5: Atlas with dropout: train versus test, Level 2

Problem. Hidden activations $\mathbf{h} = [2, 4, 0, 6]$, next-layer weights all 1, $p = 0.5$. The sampled mask is $[1, 0, 1, 1]$. (a) Training-time input to the next unit? (b) Expected value over masks? (c) Test-time value with weight scaling? (d) Same three with inverted dropout.

Solution. (a) $2 + 0 + 0 + 6 = 8$. (b) $p \sum h = 0.5 \cdot 12 = 6$. (c) $0.5 \cdot 12 = 6$ — matches (b). (d) Training: $8/0.5 = 16$; expectation 12; test (raw weights): 12 — again consistent.

Interpretation. Any single training pass is noisy; the *expectation* is what test time reproduces.

Course Paper Question: Quiz 2, dropout counting

$n = 10$ units, retention $p = 0.7$. (a) Expected number of retained units? (b) Number of distinct thinned sub-networks?

Solution. (a) $np = 7$ (binomial mean). (b) Each unit is in or out: 2^{10} .

12.8.3 Why dropout regularises

- **Ensemble view:** 2^n thinned networks, each trained rarely, but weight sharing means every parameter is trained often. Test-time scaling approximates averaging them (exactly for a single softmax layer via the geometric mean).
- **Co-adaptation view:** a unit cannot rely on specific partners being present, so it must learn features useful on their own. Dropout applies *masking noise to hidden units* — the same idea as a denoising autoencoder, one level deeper.

12.9 Comparison of Regularisers

Method	Mechanism	Cost	Trap
L_2	shrink weights in flat directions by $\frac{\lambda_i}{\lambda_i + \alpha}$	one hyper-parameter	too large $\alpha \Rightarrow$ bias
Augmentation	more label-preserving data	domain knowledge	label-changing transforms
Sharing / tying	fewer free parameters	architectural	wrong invariance assumption
Input noise	$\equiv \sigma^2 \ \mathbf{w}\ ^2$ (linear)	none	only exactly L_2 for linear models
Early stopping	limit training time ($\alpha \approx 1/\eta t$)	validation set	patience too small
Bagging	average less-correlated models	$k \times$ training	correlated errors give no gain
Dropout	implicit ensemble of 2^n nets, anti-co-adaptation	slower convergence	forgetting test-time scaling

12.10 Exam Toolkit

Exam Tip: Recognition rules

- “More data, same model” $\Rightarrow$ variance $\downarrow$, bias $\approx$ same. “More complex model” $\Rightarrow$ bias $\downarrow$, variance $\uparrow$.
- “Regularised GD step” $\Rightarrow w - \eta(\nabla L + \lambda w)$.

- “Which eigen-directions shrink” $\Rightarrow$ factor $\lambda_i/(\lambda_i + \alpha)$; flat directions shrink most.
- “Gaussian input noise, linear model” $\Rightarrow$ extra $\sigma^2 \|\mathbf{w}\|^2$.
- “Ensemble with covariance” $\Rightarrow V/k + (k-1)C/k$; limit C .
- “Dropout counts” $\Rightarrow$ expected kept np ; sub-networks 2^n ; test weights $\times p$.
- “Patience” $\Rightarrow$ stop after p non-improving checks, return the best.

Quick Recall: Regularisation

MSE = bias² + variance + σ^2 . L_2 : $w \leftarrow (1 - \eta\alpha)w - \eta\nabla L$; keeps steep directions, kills flat ones. Noise $\equiv L_2$ (linear). Early stopping $\approx L_2$. Ensemble: $V/k + \frac{k-1}{k}C$. Dropout: retain w.p. p , 2^n nets, scale by p at test.

12.11 Concept Gauntlet

- G1.** Training error is 0.1% and validation error 25%. Is the fix more capacity or more regularisation?
Answer: More regularisation / data: this is the high-variance regime. More capacity helps when *both* errors are high.
- G2.** A model’s training and validation errors are both 30% and equal. Will dropout help?
Answer: Unlikely: no gap means no overfitting to cure; this is bias. Increase capacity or train longer.
- G3.** With L_2 , does a weight in a direction of *zero* curvature survive?
Answer: No: factor $0/(0 + \alpha) = 0$. L_2 selects the minimum-norm solution among equally good ones.
- G4.** Does L_2 with $\alpha > 0$ ever make a weight exactly zero (in GD)?
Answer: Generally no: it shrinks proportionally, approaching but not reaching 0 for weights the data need. Exact zeros are characteristic of L_1 .
- G5.** You train 10 bagged copies but use the *same* bootstrap sample and the same seed for all. Ensemble error?
Answer: Identical models: $C = V$, error = V — no benefit.
- G6.** Dropout with $p = 1$ is ...? With $p \rightarrow 0$?
Answer: $p = 1$: no dropout. $p \rightarrow 0$: almost every unit dropped, the network sees almost nothing and underfits badly.
- G7.** Why not also apply dropout at test time and average many passes?
Answer: You can (“Monte Carlo dropout”); weight scaling is a cheap one-pass approximation of that average.
- G8.** Adding Gaussian noise to the inputs of a *deep non-linear* network: exactly L_2 ?
Answer: No — the equivalence is exact only for the linear model; for non-linear nets it is an approximate, data-dependent regulariser.
- G9.** Early stopping uses the validation set to choose the epoch. Can you then report validation error as your test estimate?
Answer: Not without bias: the validation set has been used for model selection. Keep a separate test set.

12.12 Practice Problems

- [FOUNDATION]** $w = 1$, $\nabla L = 0$, $\lambda = 0.2$, $\eta = 0.5$. Value after 3 regularised GD steps?
Answer: $(1 - 0.1)^3 = 0.729$.
- [INTERMEDIATE]** Eigenvalues $(4, 0.25)$, $\alpha = 0.25$. Shrink factors and effective parameters?
Answer: 0.941, 0.5; 1.44.
- [EXAM]** $V = 8$, $k = 4$, $C = 2$. Ensemble mse and its limit as $k \rightarrow \infty$.
Answer: $2 + 1.5 = 3.5$; limit 2.
- [EXAM]** A layer of 20 units uses dropout $p = 0.8$. Expected dropped units, and the test-time weight multiplier.
Answer: 4; 0.8.
- [ADVANCED]** Show that for L_2 -regularised GD from w_0 with $\nabla L \equiv 0$, $w_t = (1 - \eta\alpha)^t w_0$, and give the condition for monotone decay.
Answer: $0 < \eta\alpha < 1$.

Training Deep Networks: Pre-training, Activations, Initialisation, Batch Normalisation

Chapter Overview

Chapter 7 diagnosed the disease: error signals are multiplied by (weight) $\times$ (activation slope) at every layer, so they vanish or explode with depth; and sigmoid units saturate. This chapter is the treatment, in the historical order the course follows: **greedy layer-wise pre-training** (2006) showed deep nets *could* be trained; then simpler fixes replaced it — **better activation functions**, **better weight initialisation**, and **batch normalisation**.

Learning objectives. Describe greedy layer-wise pre-training and the two hypotheses for why it works; list each activation's formula, derivative, range and failure mode; explain why non-zero-centred activations slow learning; compute ReLU/Leaky/PReLU/ELU/Maxout outputs and gradients; derive the variance argument behind Xavier and He initialisation; compute a batch-normalised output and count its parameters.

13.1 Greedy Layer-Wise Unsupervised Pre-training

13.1.1 Procedure

1. Train the first hidden layer as an autoencoder that reconstructs $\mathbf{x}$ from $\mathbf{h}_1$ (Chapter 11).
2. Freeze it; train the second layer as an autoencoder that reconstructs $\mathbf{h}_1$ from $\mathbf{h}_2$; and so on, one layer at a time (*greedy*: each layer is optimised on its own).
3. Add the output layer and **fine-tune** the whole network with the supervised loss and backpropagation.

The pre-trained weights are just a *starting point* (an initialisation) for supervised training.

13.1.2 Why does it help? Two hypotheses

- **Better optimisation:** starting from weights that already capture the input's structure puts the network in a region from which gradient descent reaches a good minimum; deep nets from random starts often stalled. Experiments showed pre-trained networks reach *lower training error* too.
- **Better regularisation:** the unsupervised objective constrains the weights to regions that model $\mathbf{x}$ well, which acts like a prior; pre-trained nets also *generalise* better, and results are more robust to random seeds.

The larger lesson that the field drew: deep networks are *sensitive to initialisation and to the flow of gradients*. That motivated everything below, which today makes pre-training of this kind largely unnecessary for standard feedforward nets.

13.2 Better Activation Functions

13.2.1 What goes wrong with sigmoid

1. **Saturation:** $\sigma'(a) = \sigma(1 - \sigma) \rightarrow 0$ as $|a| \rightarrow \infty$; gradients through a saturated unit vanish, and with $\sigma' \leq \frac{1}{4}$ they shrink geometrically with depth (Section 7.6).
2. **Not zero-centred:** outputs lie in $(0, 1)$, always positive.
3. **Cost:** computing e^{-a} is more expensive than a comparison.

Course Paper Question: Quiz 2, saturation

A sigmoid layer's weights grow so that $a = 15$. $\sigma'(15) \approx 0$: **the neuron has saturated** ($\sigma(15) = 0.9999997$, $\sigma' \approx 3 \times 10^{-7}$). Not 0.25 (that is the maximum, at $a = 0$), not undefined.

13.2.2 Why zero-centring matters (derivation)

For a unit whose inputs are the previous layer's sigmoid outputs $h_j > 0$, $\frac{\partial \mathcal{L}}{\partial w_j} = \delta h_j$ where $\delta = \partial \mathcal{L} / \partial a$ is *one number* for that unit. Since every $h_j > 0$, **all incoming-weight gradients of that unit have the same sign as δ** . A GD step can therefore only move the weight vector into the “all-increase” or “all-decrease” orthants. If the optimum needs some weights up and others down, the only route is a zig-zag of such steps — slow. Zero-centred activations (tanh, and approximately Leaky ReLU/ELU) produce inputs of both signs and remove this restriction.

13.2.3 The activation zoo

Name	$g(a)$	$g'(a)$	Pros / cons
Sigmoid	$\frac{1}{1+e^{-a}} \in (0, 1)$	$\sigma(1 - \sigma) \leq \frac{1}{4}$	saturates; not zero-centred; expensive
tanh	$\tanh a \in (-1, 1)$	$1 - \tanh^2 a \leq 1$	zero-centred; still saturates; $\tanh a = 2\sigma(2a) - 1$
ReLU	$\max(0, a)$	1 if $a > 0$, else 0	no positive saturation; cheap; fast convergence; <i>dead neurons</i>
Leaky ReLU	$\max(0.1a, a)$	1 or 0.1	never dead; near zero-centred
PReLU	$\max(\alpha a, a)$, learned α	1 or α	adapts the negative slope
ELU	a ($a > 0$); $\alpha(e^a - 1)$ ($a \leq 0$)	1; αe^a	smooth, near zero-centred outputs; uses exp
Maxout	$\max_r(\mathbf{w}_r^\top \mathbf{x} + b_r)$, $r = 1..k$	$\mathbf{x}$ for the winning piece only	generalises ReLU/Leaky; $k \times$ parameters
GELU	$a \Phi(a)$	smooth	used in Transformers (BERT, GPT)
Swish / SiLU	$a \sigma(\beta a)$ ($\beta = 1$: SiLU)	smooth	$\beta = 1.702$ gives $\approx$ GELU

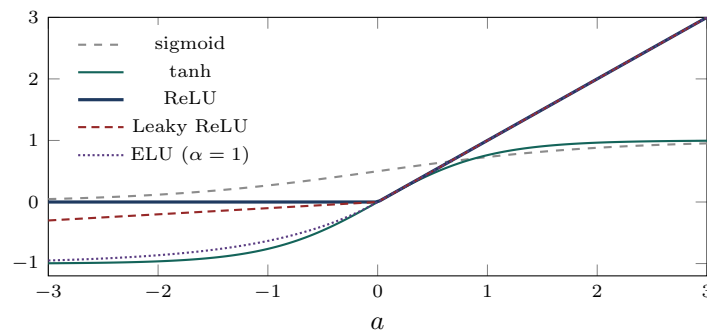


Figure 13.1: Activation functions. Only sigmoid and tanh flatten for large positive a .

13.2.4 Dead ReLUs

If a ReLU unit's pre-activation is ≤ 0 for *every* training input, its output and its gradient are 0 for every input, so its incoming weights never change again: the unit is **dead**, permanently. This often happens after a large update (e.g. too high a learning rate pushes the bias very negative). Remedies: smaller learning rates, a small positive bias initialisation, or Leaky/PReLU/ELU.

Course Paper Question: Quiz 2, dead ReLU

$a = 15$ for most inputs, then a large negative bias update makes $a < 0$ for all future inputs. $f'(a) = 0$; **the neuron stops updating and may stay dead permanently.**

Course Paper Question: Quiz 2, Leaky ReLU

$f(x) = \max(0.1x, x)$, $w = (2, -3)$, $x = (4, 3)$, no bias. $a = 8 - 9 = -1$; $f(a) = -0.1$. Motivation: **allow a small non-zero gradient when $a < 0$, avoiding permanently dead neurons** (not cheaper, not bounded, not exactly zero-centred).

Course Paper Question: End-Term, PReLU

$\mathbf{x} = [3, -2]$, $\mathbf{w} = [0.5, 1.5]$, $b = 0.5$, $\alpha = 0.1$. $z = 1.5 - 3 + 0.5 = -1 \Rightarrow f(z) = \alpha z = -0.1$.

13.2.5 Maxout

A Maxout unit computes k affine functions of its input and outputs the largest. With $k = 2$ and one piece fixed at 0 it is ReLU; with pieces a and $0.1a$ it is Leaky ReLU. Two Maxout units with enough pieces are universal approximators. In backprop the gradient flows only into the *winning* piece, but because some piece always wins, the unit as a whole never dies the way a ReLU can.

Course Paper Question: End-Term, ReLU versus Maxout (comprehension)

Two inputs $\mathbf{x} = [1, 2]^\top$, two hidden units, linear output with $\mathbf{w}_{\text{out}} = [1, 1]$, all biases 0. **Scenario A (ReLU)**: $W = \begin{bmatrix} 1 & -2 \\ -1 & 1 \end{bmatrix}$. **Scenario B (Maxout)**: unit 1 takes $\max([1, -2]\mathbf{x}, [-1, 1]\mathbf{x})$, unit 2 takes $\max([-1, 2]\mathbf{x}, [1, -1]\mathbf{x})$.

Solution. A: $\mathbf{a} = [1 - 4, -1 + 2] = [-3, 1]$; $\mathbf{h} = [0, 1]$; $\hat{y}_A = 1$.

B: unit 1: $\max(-3, 1) = 1$; unit 2: $\max(3, -1) = 3$; $\hat{y}_B = 4$.

$\nabla_{w_{11}} \mathcal{L}$ in A: unit 1 has $a = -3 < 0$, so $\text{ReLU}' = 0$ and the gradient is **0** regardless of the loss.

In B, Maxout unit 1 sees the same two pre-activations (-3 from the first piece) but its *other* piece wins with value 1, so gradient flows into that piece's weights: **yes, non-zero gradient**.

Lesson. Maxout keeps learning where ReLU is silent — the price is $k \times$ the parameters.

13.3 Better Weight Initialisation**13.3.1 What goes wrong with naive choices**

- **All zeros / all equal:** symmetry — every unit in a layer computes the same thing and receives the same gradient forever (Chapter 7, “Symmetry”).
- **Large random weights:** large pre-activations $\Rightarrow$ sigmoid/tanh saturate $\Rightarrow$ vanishing gradients.
- **Small random weights:** activations shrink layer by layer towards 0; gradients (which multiply by the weights) shrink too.

13.3.2 The variance argument (derivation)

Consider $a = \sum_{i=1}^{n_{\text{in}}} w_i x_i$ with independent, zero-mean w_i and x_i . Then

$$\text{Var}(a) = \sum_i \text{Var}(w_i x_i) = \sum_i \text{Var}(w_i) \text{Var}(x_i) = n_{\text{in}} \text{Var}(w) \text{Var}(x).$$

(For independent zero-mean factors, $\text{Var}(wx) = \mathbb{E}[w^2] \mathbb{E}[x^2] = \text{Var}(w) \text{Var}(x)$.) Each layer multiplies the signal variance by $n_{\text{in}} \text{Var}(w)$; after L layers (in the near-linear regime) by $(n_{\text{in}} \text{Var}(w))^L$. To keep it stable we need

$$n_{\text{in}} \text{Var}(w) = 1 \quad \implies \quad \text{Var}(w) = \frac{1}{n_{\text{in}}}.$$

Important: Initialisation rules

- **Xavier / Glorot** (tanh, sigmoid, linear): $\text{Var}(w) = \frac{1}{n_{\text{in}}}$, or the compromise between forward and backward passes $\text{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$, e.g. $w \sim U\left[-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right]$.
- **He / Kaiming** (ReLU): $\text{Var}(w) = \frac{2}{n_{\text{in}}}$, i.e. $w \sim \mathcal{N}(0, 2/n_{\text{in}})$. ReLU zeros half the pre-activations, halving the second moment; the factor 2 compensates.

Worked Example 13.1: Initialising the deep Atlas router, Level 3

Problem. A layer has $n_{\text{in}} = 256$, $n_{\text{out}} = 128$. (a) Xavier ($1/n_{\text{in}}$) and He standard deviations? (b) Uniform Xavier limit? (c) A colleague uses std 0.2 with $n_{\text{in}} = 100$ in 10 tanh layers; another uses std 0.05. Predict what happens.

Solution. (a) Xavier std = $1/\sqrt{256} = \mathbf{0.0625}$; He std = $\sqrt{2/256} = \mathbf{0.0884}$. (b) $\sqrt{6/384} = \mathbf{0.125}$. (c) Per-layer variance factor $n\text{Var}(w)$: $100 \cdot 0.04 = 4 \Rightarrow 4^{10} \approx 10^6$ growth (tanh saturates; gradients vanish through saturated units). $100 \cdot 0.0025 = 0.25 \Rightarrow 0.25^{10} \approx 10^{-6}$ (activations and gradients vanish). Xavier gives factor 1.

Interpretation. Both failures are the *same* exponential effect with the base on either side of 1 — the recurring theme of this course (compare exploding/vanishing gradients in RNNs, Part IV).

13.4 Batch Normalisation

13.4.1 Working principle

During training, the distribution of each layer's inputs keeps shifting as earlier layers change, and pre-activations can drift into saturated ranges. Batch normalisation (BN) re-standardises each unit's pre-activation over the mini-batch, then lets the network choose the scale and shift it actually wants.

Definition: Batch normalisation for one unit

For the values $a^{(1)}, \dots, a^{(B)}$ of one unit over a mini-batch:

$$\mu_B = \frac{1}{B} \sum_i a^{(i)}, \quad \sigma_B^2 = \frac{1}{B} \sum_i (a^{(i)} - \mu_B)^2, \quad \hat{a}^{(i)} = \frac{a^{(i)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y^{(i)} = \gamma \hat{a}^{(i)} + \beta.$$

γ and β are learned (2 parameters per unit). At test time μ_B, σ_B^2 are replaced by running averages collected during training.

Choosing $\gamma = \sqrt{\sigma_B^2 + \epsilon}$ and $\beta = \mu_B$ undoes the normalisation, so BN never *reduces* what the layer can represent.

Worked Example 13.2: Normalising one Atlas unit, Level 2

Problem. A unit's pre-activations on a batch of 4 are 2, 4, 6, 8; $\gamma = 2$, $\beta = 1$, $\epsilon \approx 0$. Compute the BN outputs.

Solution. $\mu_B = 5$; $\sigma_B^2 = \frac{9+1+1+9}{4} = 5$, $\sigma_B = 2.236$. $\hat{a} = [-1.342, -0.447, 0.447, 1.342]$. $y = 2\hat{a} + 1 = [-1.683, 0.106, 1.894, 3.683]$.

Verification. Mean of y is $\beta = 1$; variance is $\gamma^2 = 4 \checkmark$ (note the course uses the *biased* $1/B$ variance).

Interpretation. Whatever scale the incoming weights produce, this unit's outputs have the mean and spread chosen by (β, γ) — so the next layer sees a stable distribution, and sigmoid/tanh stay in their sensitive range.

13.4.2 Consequences

- Larger learning rates become safe and training is faster.
- Less sensitivity to initialisation.
- Mild regularisation: each example's normalisation depends on the other examples in its batch, a form of noise.
- The layer's own bias b is redundant (subtracting μ_B removes it; β replaces it).

13.5 Exam Toolkit

Exam Tip: Recognition rules

- “Weights grow large, sigmoid” $\Rightarrow$ saturation, $\sigma' \approx 0$.
- “Pre-activation ≤ 0 for all inputs, ReLU” $\Rightarrow$ dead neuron, zero gradient forever.
- “Leaky/PReLU with negative a ” $\Rightarrow$ output αa , gradient α .
- “Maxout” $\Rightarrow$ max of affine pieces; gradient only into the winner; never fully dead.
- “Sigmoid inputs, all gradients same sign” $\Rightarrow$ non-zero-centred, zig-zag.
- “Var(w)” $\Rightarrow$ Xavier $1/n_{\text{in}}$ (or $2/(n_{\text{in}} + n_{\text{out}})$); He $2/n_{\text{in}}$ for ReLU.
- “Batch norm numbers” $\Rightarrow$ biased variance, then $\gamma\hat{a} + \beta$; 2 learnable parameters per unit.

Quick Recall: Training deep networks

Pre-training: greedy AE per layer, then fine-tune (helps optimisation and regularisation). Prefer ReLU-family (watch dead units; Leaky/PReLU/ELU/Maxout fix it); tanh over sigmoid if saturating units are needed. Init: $\text{Var}(w) = 1/n_{\text{in}}$ (Xavier) or $2/n_{\text{in}}$ (He). BN: standardise per unit over the batch, then γ, β .

13.6 Concept Gauntlet

- G1.** Is ReLU a linear function? Can a ReLU network represent non-linear boundaries?
Answer: ReLU is piecewise linear, not linear ($\text{ReLU}(-1) + \text{ReLU}(1) \neq \text{ReLU}(0)$). ReLU networks compute piecewise-linear functions with many pieces — fully non-linear boundaries.
- G2.** A ReLU unit outputs 0 on one input. Is it dead?
Answer: No. Dead means $a \leq 0$ for *all* inputs; being off on some inputs is normal and useful.
- G3.** What is $\text{ReLU}'(0)$? Does it matter?
Answer: Undefined mathematically; libraries pick 0 (or 1). It almost never matters because a is exactly 0 with probability ≈ 0 .
- G4.** tanh is zero-centred. Does that fix vanishing gradients?
Answer: No — $\tanh' \leq 1$ and it still saturates. Zero-centring fixes the same-sign-gradient zig-zag, not saturation.
- G5.** He initialisation in a sigmoid network: good idea?
Answer: Too large for sigmoid/tanh (designed for ReLU’s halving); pushes units towards saturation. Use Xavier.
- G6.** After BN, should we still add a bias to that unit?
Answer: No; μ_B subtraction cancels it and β plays its role.
- G7.** BN with batch size 1 at training time?
Answer: $\sigma_B^2 = 0$ and $\hat{a} = 0$ for every example — the unit outputs β always and carries no information. BN needs reasonable batch sizes.
- G8.** Maxout with $k = 2$ doubles which count — neurons or parameters?
Answer: Parameters of that layer (two affine maps per unit); the number of outputs is unchanged.
- G9.** Leaky ReLU with $\alpha = 1$?
Answer: Identity — the network collapses to a linear model (Proposition 6.1).

13.7 Practice Problems

1. **[FOUNDATION]** ELU ($\alpha = 1$) at $a = -1$ and its derivative there.
Answer: $e^{-1} - 1 = -0.632$; $e^{-1} = 0.368$.
2. **[INTERMEDIATE]** Show $\tanh a = 2\sigma(2a) - 1$ and hence $\tanh'(0) = 1$ while $\sigma'(0) = \frac{1}{4}$.
3. **[EXAM]** A BN unit sees batch values 1, 1, 1, 5; $\gamma = 1$, $\beta = 0$. Outputs?
Answer: $\mu = 2$, $\sigma^2 = 3$: $[-0.577, -0.577, -0.577, 1.732]$.
4. **[EXAM]** Number of learnable parameters added by BN on a 512-unit layer (and non-learnable running statistics)?
Answer: 1024 learnable (γ, β); 1024 running statistics (mean, variance).
5. **[ADVANCED]** With He initialisation and ReLU, why is $\mathbb{E}[h^2] = \frac{1}{2}\text{Var}(a)$ for zero-mean symmetric a ?
Hint: ReLU keeps exactly the positive half; $\mathbb{E}[a^2 \mathbf{1}_{a>0}] = \frac{1}{2}\mathbb{E}[a^2]$.

Part II Integration: Master Problems, Carry-overs and Rapid Revision

14.1 Master Integration Problems

Master Integration Problem 4: Designing the Echo telemetry compressor

Problem. Six centred sensor channels have covariance eigenvalues 5, 3, 1, 0.5, 0.3, 0.2. (1) What is the smallest linear-autoencoder bottleneck k that keeps at least 90% of the variance, and what reconstruction error remains? (2) Count parameters for the $6 \rightarrow k \rightarrow 6$ autoencoder, untied and tied. (3) Engineers now add a sigmoid encoder and masking noise with $q = 0.2$; how many channels are zeroed per frame on average, and against what is the loss computed? (4) Why might the non-linear denoising version beat PCA?

Solution. (1) Total variance 10. $k = 1$: 50%; $k = 2$: 80%; $k = 3$: $9/10 = 90\%$. Remaining error $= 0.5 + 0.3 + 0.2 = 1.0$ (Theorem 11.1).

(2) $k = 3$: untied $3(6 + 1) + 6(3 + 1) = 21 + 24 = 45$; tied $18 + 3 + 6 = 27$.

(3) $6 \times 0.2 = 1.2$ channels; loss against the **clean** frame.

(4) PCA is the best *linear* rank-3 reconstruction; a non-linear decoder can follow curved structure, and denoising forces it to exploit dependencies between channels, making the code robust to dropped readings on the radio link.

Complexity. Encoding a frame costs $O(nk)$ multiplications — 18 here.

Master Integration Problem 5: The Pulse overfitting clinic

Problem. A PULSE network has training error 2% and validation error 18%. (1) Diagnose. (2) One weight has $w = -2$, $\partial L / \partial w = 1.5$; apply one L_2 -regularised step with $\alpha = 0.5$, $\eta = 0.1$. Did the weight move towards zero? (3) The first layer is (approximately) linear with weights $[1, -2, 2]$ on sensors with noise variance 0.05: what implicit penalty does training on noisy inputs add? (4) Five bagged models have $V = 10$, $C = 4$: ensemble mse? (5) Recommend a plan.

Solution. (1) Large train-validation gap: **high variance** (overfitting).

(2) $w \leftarrow -2 - 0.1(1.5 + 0.5(-2)) = -2 - 0.05 = -2.05$. Away from zero! The decay pulls w towards 0 by $\eta\alpha|w| = 0.1$, but the data gradient pushes it the other way by 0.15. Weight decay is a pull, not a guarantee.

(3) $\sigma^2 \|\mathbf{w}\|^2 = 0.05 \cdot 9 = 0.45$.

(4) $\frac{10}{5} + \frac{4}{5} \cdot 4 = 2 + 3.2 = 5.2$ (vs 10 for one model; limit 4 as $k \rightarrow \infty$).

(5) Cheapest first: early stopping (free), L_2 /weight decay, dropout on hidden layers, input noise/augmentation; ensembles if compute allows. Re-check the gap after each change; stop regularising once validation error stops improving (too much regularisation raises bias).

Master Integration Problem 6: The deep Atlas router won't train

Problem. An 8-layer ReLU network (512 units per layer) is trained with learning rate 1.0. After one epoch, 40% of hidden units output 0 for *every* training parcel and training loss is stuck. (1) Diagnose. (2) Propose three fixes. (3) He standard deviation for these layers? (4) Parameters of one $512 \rightarrow 512$ layer with BN (with and without the redundant bias). (5) With dropout $p = 0.5$ on hidden units during training (non-inverted), what must happen at test time?

Solution. (1) **Dead ReLUs**: a too-large learning rate pushed biases/weights so that pre-activations are negative for all inputs; zero gradient means they can never recover.

(2) Lower learning rate (or a schedule with warm-up); Leaky ReLU/PReLU/ELU; He initialisation and/or batch normalisation to keep pre-activations centred; small positive initial biases.

- (3) $\sqrt{2/512} = \mathbf{0.0625}$.
 (4) Weights $512^2 = 262,144$; with bias +512 and BN +1024: **263,680**; without the redundant bias **263,168**.
 (5) Multiply each hidden unit's outgoing weights by $p = 0.5$ (or use inverted dropout during training instead).

14.2 Week 4 Carry-overs from Quiz 2

These course questions test Chapter 9; they appeared in the Weeks 5–8 quiz, so they are solved here.

Course Paper Question: Quiz 2, triangular cyclical learning rate

$\eta_t = \eta_{\min} + (\eta_{\max} - \eta_{\min}) \max(0, 1 - \lfloor \frac{t}{\mu} - 2 \lfloor 1 + \frac{t}{2\mu} \rfloor + 1 \rfloor)$, $\eta_{\min} = 0.001$, $\eta_{\max} = 0.1$, $\mu = 500$. When is $\eta_t = \eta_{\min}$?

Solution. At $t = 0$: $\lfloor 1 \rfloor = 1$, $|0 - 2 + 1| = 1 \Rightarrow$ minimum. At $t = 500$: $\lfloor 1.5 \rfloor = 1$, $|1 - 2 + 1| = 0 \Rightarrow$ maximum. At $t = 1000$: $\lfloor 2 \rfloor = 2$, $|2 - 4 + 1| = 1 \Rightarrow$ minimum. The cycle length is $2\mu = 1000$: minima at $t \in \{0, 1000, 2000, \dots\}$, maxima at 500, 1500, ...

Trick. μ is the *half*-cycle (“step size”); test $t = 0$ and $t = \mu$ to orient yourself.

Course Paper Question: Quiz 2, RMSProp versus AdaGrad on a sparse gradient

$\beta = 0.9$, $\eta_0 = 0.01$, $\epsilon = 0$, $v_0 = 0$; gradients 10, 0, 0. (a) RMSProp's effective rate at iteration 3? (b) AdaGrad's behaviour?

Solution. (a) $v_1 = 0.1 \cdot 100 = 10$, $v_2 = 9$, $v_3 = 8.1$; $\eta_0/\sqrt{8.1} = 0.01/2.846 = \mathbf{0.0035}$. (b) AdaGrad: v stays at 100, rate stays at $0.01/10 = \mathbf{0.001}$ (constant).

Lesson. When gradients stop, RMSProp “forgets” and its rate *rises* ($0.00316 \rightarrow 0.00333 \rightarrow 0.00351$); AdaGrad's rate can never rise.

Course Paper Question: Quiz 2, adaptive optimiser statements

True: “AdaGrad's effective rate decays slowly for *sparse* features” (their v grows rarely); “Adam's v_t is an L^2 -type quantity while AdaMax uses L^∞ ”. False: “decays slowly for dense features” (dense features accumulate fastest); “RMSProp's rate is guaranteed non-increasing” (see previous box); “AdaDelta is sensitive to η_0 ” (it has no η_0).

14.3 Index of Course Paper Questions Solved in Part II

Paper	Topic	Where
Quiz 2 Q2	more data $\Rightarrow$ variance down	Ch. 12
Quiz 2 Q6–7	bagging covariance, limit C_0	Ch. 12
Quiz 2 Q8–9	dropout: 7 kept, 2^{10} sub-networks	Ch. 12
End-Term	L_2 -regularised update 2.785	Ch. 12
Quiz 2 Q10–12	Leaky ReLU $-1 \rightarrow -0.1$, motivation	Ch. 13
Quiz 2 Q13–14	sigmoid saturation; dead ReLU	Ch. 13
End-Term	PReLU -0.1	Ch. 13
End-Term	ReLU vs Maxout: outputs 1 and 4, gradients	Ch. 13
Quiz 2 Q3, Q4–5, Q20	cyclical LR, RMSProp/AdaGrad, optimiser statements	Ch. 14

14.4 Part II Master Reference

Formula	Meaning	Use when / assumptions
$\mathbf{h} = g(W\mathbf{x} + \mathbf{b}), \hat{\mathbf{x}} = f(W^*\mathbf{h} + \mathbf{c})$	autoencoder	$k < n$ undercomplete
sigmoid f + CE; linear f + SE	AE loss choice	binary vs real inputs
error = $\sum_{i>k} \lambda_i$	linear AE = PCA	linear enc/dec, SE, normalised input
$\sum_l \rho \ln \frac{\rho}{\hat{\rho}_l} + (1 - \rho) \ln \frac{1-\rho}{1-\hat{\rho}_l}$	sparsity (KL)	sigmoid codes, $\hat{\rho}_l$ = mean activation
$\ J_{\mathbf{x}}(\mathbf{h})\ _F^2 = \sum_l [h_l(1 - h_l)]^2 \sum_j W_{lj}^2$	contractive (sigmoid)	penalises code sensitivity
Bias ² + Var + σ^2	expected test MSE	expectation over training sets
$w \leftarrow (1 - \eta\alpha)w - \eta\nabla L$	L_2 step	$\tilde{\mathcal{L}} = \mathcal{L} + \frac{\alpha}{2} \ w\ ^2$
$\frac{\lambda_i}{\lambda_i + \alpha}$	L_2 shrink per eigen-direction	quadratic approximation
$\sigma^2 \ \mathbf{w}\ ^2$	input-noise penalty	exact for linear model
$\frac{1}{k}V + \frac{k-1}{k}C$	ensemble mse	V variance, C covariance
$np; 2^n$; weights $\times p$ at test	dropout facts	p = retention probability
$\text{Var}(a) = n_{\text{in}} \text{Var}(w) \text{Var}(x)$	variance propagation	independent zero-mean
$1/n_{\text{in}}$ or $\frac{2}{n_{\text{in}} + n_{\text{out}}}$; $2/n_{\text{in}}$	Xavier; He	tanh/sigmoid; ReLU
$\gamma \frac{a - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$	batch norm	biased batch variance; running stats at test

14.4.1 Mixed Concept Gauntlet (Parts I–II)

- M1.** An autoencoder’s hidden layer uses identity activations and a sigmoid decoder. Is it still “just PCA”?
Answer: No — condition (ii) fails; the decoder is non-linear.
- M2.** Dropout on the *input* layer of a denoising autoencoder is ...?
Answer: Exactly masking noise with $q = 1 - p$: the two ideas coincide there.
- M3.** You double the batch size and keep η . Does BN’s regularisation effect grow or shrink?
Answer: Shrink: batch statistics get less noisy.
- M4.** Weight decay and Adam: is L_2 added to the loss the same as decaying the weights directly?
Answer: For plain GD yes; for adaptive methods no, because the αw term gets divided by $\sqrt{\hat{v}}$ — which is why “decoupled” weight decay (AdamW) exists. (*Beyond the core.*)
- M5.** Why is early stopping called “free”?
Answer: It needs only a validation set and makes training shorter, not longer.
- M6.** Symmetric init problem with ReLU and zero biases, weights all 0: what are the hidden outputs and gradients?
Answer: All pre-activations 0 $\Rightarrow$ hidden outputs 0. Weight gradients are (error at destination) $\times$ (source activation) = 0, and hidden errors pass through zero weights and $\text{ReLU}'(0) = 0$. Only the output biases learn — the network is stuck.
- M7.** A sigmoid unit with Xavier init and inputs of scale 100 (not normalised): what happens?
Answer: Pre-activations of size ~ 100 saturate the unit; initialisation rules assume unit-scale inputs — normalise the data (or use BN).
- M8.** Is Maxout “dropout-like”?
Answer: The course notes relate them loosely (a unit picks among several candidates), but Maxout is deterministic — it is an activation, not a regulariser.

End of Part II. Part III continues with Weeks 8–9: word representations, convolutional networks and their visualisation.

Part III

Representations of Words and Images (Weeks 8–9)

Learning Vectorial Representations of Words

Chapter Overview

To feed text to a network, every word needs a vector. This chapter moves from one-hot vectors (no notion of similarity) to **count-based** representations (co-occurrence, PMI/PPMI, SVD) and **prediction-based** ones (CBOW, skip-gram, negative sampling, hierarchical softmax), ending with GloVe, which unifies the two. A new world joins: BABEL, the mission’s translation system between ground crews.

15.1 One-Hot Vectors and Their Limits

With vocabulary V ($|V|$ words), word j is $\mathbf{e}_j \in \mathbb{R}^{|V|}$. Problems: dimension $|V|$ (often $\sim 10^5$), sparse, and *every pair of distinct words is equally far*: $\|\mathbf{e}_i - \mathbf{e}_j\| = \sqrt{2}$, cosine 0. “cat” is as similar to “dog” as to “truck”.

15.2 Count-Based Representations

15.2.1 Co-occurrence and PMI

Distributional hypothesis: a word is characterised by the company it keeps. Build $X \in \mathbb{R}^{|V| \times |V|}$ with X_{ij} = number of times word j appears within a window of k words around word i . Raw counts are dominated by frequent words (“the”), so replace them by **pointwise mutual information**:

$$\text{PMI}(w, c) = \log \frac{p(c | w)}{p(c)} = \log \frac{\text{count}(w, c) \cdot N}{\text{count}(w) \text{count}(c)}.$$

$\text{PMI} > 0$: w, c co-occur more than chance; < 0 : less; $\text{count}(w, c) = 0 \Rightarrow -\infty$. Hence **PPMI** = $\max(\text{PMI}, 0)$.

Course Paper Question: End-Term, PMI and PPMI

$N = 10,000$, $\text{count}(w) = 50$, $\text{count}(c) = 40$, $\text{count}(w, c) = 10$: $\text{PMI} = \ln \frac{10 \cdot 10000}{50 \cdot 40} = \ln 50 = \mathbf{3.91}$. A pair with $\text{PMI} = -1.0986$ is stored as **0** under PPMI.

15.2.2 SVD for dense vectors

$X_{\text{PPMI}} \approx \hat{X} = U_k \Sigma_k V_k^\top$ (best rank- k approximation, Eckart–Young, Theorem 11.1). Word representations: $W_{\text{word}} = U_k \Sigma_k \in \mathbb{R}^{m \times k}$; context representations: $W_{\text{context}} = V_k$. Why $U\Sigma$: $\hat{X}\hat{X}^\top = U\Sigma V^\top V\Sigma U^\top = (U\Sigma)(U\Sigma)^\top$, so dot products of the k -dim rows of $U\Sigma$ equal the dot products of the full rows of $\hat{X}$ — similarity is preserved in k dimensions. SVD is a closed-form matrix factorisation, *not* iterative gradient training.

Course Paper Question: End-Term, SVD storage and statements

$X \in \mathbb{R}^{1000 \times 800}$, rank-1 truncation $\sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top$: store $1000 + 800 + 1 = \mathbf{1801}$ numbers. True: $W_{\text{word}} = U\Sigma$ is a 50-dim representation of the 1000 words; $W_{\text{context}} = V$ represents the 800 context words. False: “SVD needs gradient training like CBOW”; “since $V^\top V = I$, $\hat{X}\hat{X}^\top = W_{\text{context}} W_{\text{context}}^\top$ ” (it equals $W_{\text{word}} W_{\text{word}}^\top$).

15.3 Prediction-Based Models

15.3.1 CBOW

Predict the centre word from context. One-hot context $\mathbf{x}$, $\mathbf{h} = W_{\text{context}} \mathbf{x}$ (for one context word, this just selects a column; for several, average/sum the columns), scores $\mathbf{z} = W_{\text{word}}^\top \mathbf{h}$, then $P(w | c) = \text{softmax}(\mathbf{z})_w$. Loss $-\log P(w | c)$; by Chapter 7, $\nabla_{\mathbf{z}} = \hat{\mathbf{y}} - \mathbf{e}_w$. The update of word vector $\mathbf{u}_w$ is $\mathbf{u}_w \leftarrow \mathbf{u}_w - \eta(\hat{y}_w - 1)\mathbf{h}$: it moves *towards* the context vector — words sharing contexts drift together.

15.3.2 Skip-gram

The reverse: predict each context word from the centre word, with the same softmax structure.

Course Paper Question: End-Term, skip-gram forward pass

Vocabulary [he, sat, on, a, chair], input “sat”. $W_{\text{context}} = \begin{bmatrix} 1 & 0 & -1 & 1 & 0 \\ 0 & 1 & 1 & 0 & -1 \end{bmatrix}$, $W_{\text{word}} = \begin{bmatrix} 1 & 0 & 1 & -1 & 0 \\ 0 & 1 & 1 & 0 & 1 \end{bmatrix}$. $\mathbf{h} = W_{\text{context}} \mathbf{e}_2 = [0, 1]^\top$; $\mathbf{z} = W_{\text{word}}^\top \mathbf{h} = [0, 1, 1, 0, 1]$, sum **3**. $P(\text{on} \mid \text{sat}) = \frac{e^1}{3e+2} = \mathbf{0.268}$.

Trick. $W\mathbf{e}_j$ is just column j ; $W^\top \mathbf{h}$ with $\mathbf{h} = \mathbf{e}_2$ is row 2 of W .

15.3.3 Negative sampling

The softmax denominator sums over $|V|$ words — too expensive. Negative sampling instead trains a binary classifier: true pairs $(w, c) \in D$ should have $\sigma(\mathbf{u}_c^\top \mathbf{v}_w) \rightarrow 1$; random “negative” pairs (w, r) drawn from $p(r) \propto \text{count}(r)^{3/4}$ should have $\sigma(\mathbf{u}_r^\top \mathbf{v}_w) \rightarrow 0$:

$$\mathcal{L} = -\log \sigma(\mathbf{v}_w^\top \mathbf{u}_c) - \sum_{r \in \text{neg}} \log \sigma(-\mathbf{v}_w^\top \mathbf{u}_r).$$

Use k negatives per positive (e.g. 5–20).

Course Paper Question: End-Term, one negative-sampling step

$\mathcal{L} = -\log \sigma(\mathbf{w}^\top \mathbf{c}) - \log \sigma(-\mathbf{w}_{\text{neg}}^\top \mathbf{c})$, $\mathbf{w} = [1, 0]$, $\mathbf{c} = [1, 1]$, $\mathbf{w}_{\text{neg}} = [0, 1]$, $\eta = 0.5$.

Loss. $\mathbf{w}^\top \mathbf{c} = 1$, $\mathbf{w}_{\text{neg}}^\top \mathbf{c} = 1$: $\mathcal{L} = -\ln \sigma(1) - \ln \sigma(-1) = 0.3133 + 1.3133 = \mathbf{1.627}$.

Update. Only the first term depends on $\mathbf{w}$: $\nabla_{\mathbf{w}} \mathcal{L} = -(1 - \sigma(1))\mathbf{c} = -0.2689[1, 1]$. $\mathbf{w}_{\text{new}} = [1.1345, 0.1345]$.
Cosine with $\mathbf{c}$: $\frac{1.2689}{1.1424 \cdot 1.4142} = \mathbf{0.785}$ (up from 0.707).

Trap. Read which vectors appear in each term; the negative term here does not involve $\mathbf{w}$.

15.3.4 Hierarchical softmax

Arrange the vocabulary as leaves of a binary tree; $P(w \mid c)$ is the product of $\approx \log_2 |V|$ binary (sigmoid) decisions along the path to w , each with a learned vector per internal node. Cost per prediction: $O(\log |V|)$ instead of $O(|V|)$; with $|V| = 10^6$, about 20 sigmoids instead of a million-term sum.

15.4 GloVe

Combine counts with prediction: learn vectors so that $\mathbf{v}_i^\top \mathbf{u}_j + b_i + b_j \approx \log X_{ij}$, minimising $\sum_{i,j} f(X_{ij})(\mathbf{v}_i^\top \mathbf{u}_j + b_i + b_j - \log X_{ij})^2$, where the weight f caps the influence of very frequent pairs and ignores $X_{ij} = 0$. It uses global co-occurrence statistics (like SVD) but is trained by gradient descent (like word2vec).

Method	Input	Training	Key trick
PPMI + SVD	co-occurrence counts	closed-form factorisation	$U_k \Sigma_k$ as word vectors
CBOW	context $\rightarrow$ centre	softmax + GD	average context columns
Skip-gram	centre $\rightarrow$ context	softmax + GD	good for rare words
Negative sampling	(pair, label)	sigmoid + GD	k negatives, $p \propto \text{count}^{3/4}$
Hierarchical softmax	tree over V	path of sigmoids	$O(\log V)$
GloVe	$\log X_{ij}$	weighted least squares	global counts + GD

15.5 Walkthrough: Building a Co-occurrence Matrix by Hand

Worked Example 15.1: Babel counts contexts, Level 2

Problem. Corpus: “the cat sat”, “the dog sat”, “the cat ran”. Vocabulary [the, cat, dog, sat, ran], window $k = 1$ (one word on each side, within a sentence). Build X , then PPMI, and compare the similarity of *cat* and *dog*.

Solution. Counting neighbours:

	the	cat	dog	sat	ran
the	0	2	1	0	0
cat	2	0	0	1	1
dog	1	0	0	1	0
sat	0	1	1	0	0
ran	0	1	0	0	0

$N = \sum_{ij} X_{ij} = 12.$

X is symmetric (symmetric window). PMI uses row sums r_i and column sums c_j : $\text{PMI}_{ij} = \ln \frac{X_{ij}N}{r_i c_j}$. E.g. cat–ran: $\ln \frac{1 \cdot 12}{4 \cdot 1} = \ln 3 = 1.10$; cat–sat: $\ln \frac{12}{4 \cdot 2} = 0.41$; the–cat: $\ln \frac{2 \cdot 12}{3 \cdot 4} = 0.69$. All zero-count cells become 0 under PPMI.

Similarity. $\cos(\text{cat}, \text{dog})$: raw rows 0.866; PPMI rows 0.524.

Interpretation. Cat and dog are similar because they share contexts (*the*, *sat*) — they never co-occur with each other at all! That is the distributional hypothesis in action. PPMI lowers the similarity because the shared word *the* is frequent and hence less informative.

Common Trap: Co-occurrence is word $\times$ context

A co-occurrence matrix has words as rows and context words as columns, each cell counting co-occurrences within a window (a course PYQ). A *document–term* or document–document similarity matrix is a different object. Also note: stop-words dominate raw counts — remove them, cap counts, or use PPMI.

15.6 CBOW in Detail

15.6.1 Several context words

With a window of d context words, concatenating one-hot vectors would multiply the first-layer size; the course instead **adds the selected columns**: $\mathbf{h} = \sum_{c \in \text{context}} \mathbf{u}_c$, where $\mathbf{u}_c$ is the column of W_{context} whose bit is on.

Worked Example 15.2: CBOW with two context words, Level 2

Problem. Using the End-Term matrices of the skip-gram question, predict the centre word from contexts {he, on}.

Solution. $\mathbf{h} = \text{col}_1 + \text{col}_3 = [1, 0] + [-1, 1] = [0, 1]$. $\mathbf{z} = W_{\text{word}}^\top \mathbf{h} = [0, 1, 1, 0, 1]$, so $P = [0.098, 0.268, 0.268, 0.098, 0.268]$ over [he, sat, on, a, chair]. *sat* ties with *on* and *chair* at 0.268.

Learning step. With target *sat*, $\nabla_{\mathbf{z}} \mathcal{L} = \hat{\mathbf{y}} - \mathbf{e}_{\text{sat}}$; the *sat* word-vector moves by $+\eta(1 - 0.268)\mathbf{h}$ towards $\mathbf{h}$, the others away in proportion to their probability. Repeated over the corpus, words that share contexts acquire similar vectors.

15.6.2 Why two matrices, and which one is the embedding?

Each word has a *context* vector (column of W_{context}) and a *word* vector (column of W_{word}). Either can be used as the representation; commonly W_{context} (input side) or their average. The course’s SVD analogy: $W_{\text{word}} \leftrightarrow U\Sigma$, $W_{\text{context}} \leftrightarrow V$.

15.7 Negative Sampling: Full Gradients

For one positive pair (w, c) and one negative r :

$$\mathcal{L} = -\log \sigma(\mathbf{v}_w^\top \mathbf{u}_c) - \log \sigma(-\mathbf{v}_w^\top \mathbf{u}_r).$$

Using $\frac{d}{dz} \log \sigma(z) = 1 - \sigma(z)$ and $\frac{d}{dz} \log \sigma(-z) = -\sigma(z)$:

$$\nabla_{\mathbf{v}_w} \mathcal{L} = -(1 - \sigma(\mathbf{v}_w^\top \mathbf{u}_c)) \mathbf{u}_c + \sigma(\mathbf{v}_w^\top \mathbf{u}_r) \mathbf{u}_r, \quad \nabla_{\mathbf{u}_c} \mathcal{L} = -(1 - \sigma(\mathbf{v}_w^\top \mathbf{u}_c)) \mathbf{v}_w, \quad \nabla_{\mathbf{u}_r} \mathcal{L} = \sigma(\mathbf{v}_w^\top \mathbf{u}_r) \mathbf{v}_w.$$

Reading: the word vector is pulled towards its true context and pushed away from the negative; each pull is weighted by how wrong the current prediction is.

Worked Example 15.3: All three vectors move, Level 3

Problem. $\mathbf{v}_w = [1, 0]$, $\mathbf{u}_c = [1, 1]$, $\mathbf{u}_r = [0, 1]$, $\eta = 0.5$ (now the negative term uses $\mathbf{v}_w^\top \mathbf{u}_r$, the standard form). Update all vectors and recompute the loss.

Solution. $\mathbf{v}_w^\top \mathbf{u}_c = 1$ ($\sigma = 0.731$), $\mathbf{v}_w^\top \mathbf{u}_r = 0$ ($\sigma = 0.5$). $\mathcal{L} = 0.313 + 0.693 = 1.006$.
 $\nabla_{\mathbf{v}_w} = -0.269[1, 1] + 0.5[0, 1] = [-0.269, 0.231]$; $\nabla_{\mathbf{u}_c} = -0.269[1, 0]$; $\nabla_{\mathbf{u}_r} = 0.5[1, 0]$.
 New: $\mathbf{v}_w = [1.134, -0.116]$, $\mathbf{u}_c = [1.134, 1]$, $\mathbf{u}_r = [-0.25, 1]$. New loss **0.783**.

Interpretation. Compare the End-Term version (Chapter 15, negative term $\mathbf{w}_{\text{neg}}^\top \mathbf{c}$): there only the positive term moved $\mathbf{w}$. Always read which dot product appears in each term.

15.7.1 Contrastive estimation (course variant)

Instead of probabilities, compute scores s (true pair) and s_r (corrupted pair, e.g. a random word swapped in) and demand a margin m : minimise $\max(0, m - (s - s_r))$. If $s > s_r + m$ nothing is updated. E.g. $s = 2.1$, $s_r = 1.5$, $m = 1$: loss 0.4 — still pushed apart.

15.7.2 Sampling details

Negatives are drawn from $p(r) \propto \text{count}(r)^{3/4}$: the $3/4$ power boosts rare words relative to raw frequency. Frequent words (“the”) are also sub-sampled from the *positive* pairs, since they carry little information.

15.8 Hierarchical Softmax Walkthrough

Leaves of a binary tree are the words; internal node i has vector $\mathbf{u}_i$. At each node on the path to word w , go left with probability $\sigma(\mathbf{v}_c^\top \mathbf{u}_i)$ (the course’s $\pi(w)_k = 1$ for left). Then $P(w | c) = \prod_k P(\text{branch}_k)$.

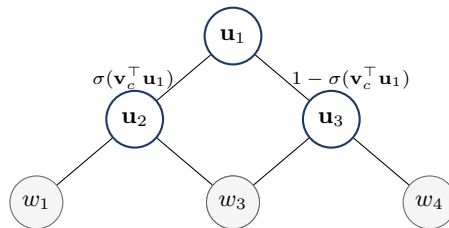


Figure 15.1: Hierarchical softmax over 4 words: each prediction is 2 binary decisions.

Worked Example 15.4: Probabilities from a path, Level 2

Problem. $\mathbf{v}_c = [1, 2]$, $\mathbf{u}_1 = [1, 0]$, $\mathbf{u}_2 = [0, 1]$, $\mathbf{u}_3 = [1, -1]$ (tree of Figure 15.1). Compute $P(w_i | c)$ for all leaves.

Solution. $\sigma(\mathbf{v}_c^\top \mathbf{u}_1) = \sigma(1) = 0.731$; $\sigma(\mathbf{v}_c^\top \mathbf{u}_2) = \sigma(2) = 0.881$; $\sigma(\mathbf{v}_c^\top \mathbf{u}_3) = \sigma(-1) = 0.269$. $P(w_1) = 0.731 \cdot 0.881 = 0.644$, $P(w_2) = 0.731 \cdot 0.119 = 0.087$, $P(w_3) = 0.269 \cdot 0.269 = 0.072$, $P(w_4) = 0.269 \cdot 0.731 = 0.197$. Sum = 1 ✓.

Interpretation. Normalisation is automatic (each node splits its mass). Cost = path length $\approx \log_2 |V|$. The course notes also report that even a *random* assignment of words to leaves works reasonably in

practice.

15.9 GloVe: Derivation Sketch and Weighting

Want $\mathbf{v}_i^\top \mathbf{u}_j \approx \log P(j | i) = \log X_{ij} - \log X_i$. The term $\log X_i$ depends only on i and is absorbed into a learned bias b_i ; add b_j for symmetry. Weighted least squares:

$$J = \sum_{i,j} f(X_{ij}) (\mathbf{v}_i^\top \mathbf{u}_j + b_i + b_j - \log X_{ij})^2, \quad f(x) = \begin{cases} (x/x_{\max})^\alpha & x < x_{\max} \\ 1 & \text{otherwise} \end{cases}$$

with typically $x_{\max} = 100$, $\alpha = 0.75$; $f(0) = 0$ removes $\log 0$. E.g. $X_{ij} = 10$: target $\ln 10 = 2.303$, weight $(0.1)^{0.75} = 0.178$.

15.10 Evaluating Embeddings: Similarity and Analogy

Similarity: rank word pairs by cosine and compare with human judgements. **Analogy:** “man : king :: woman : ?” — find the word closest to $\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}}$. Toy example: king [3, 3], man [2, 1], woman [1, 2] give [2, 4]; queen [2, 4.2] has cosine 0.9998 with it. The *difference* vectors encode relations (gender, tense, capital-of).

Exam Tip: Recognition rules for word vectors

- Counts $\Rightarrow$ PMI with N = total count; zero count $\Rightarrow$ PPMI 0.
- CBOW with several context words $\Rightarrow$ *sum* the selected columns (course convention).
- “Only $\mathbf{w}$ is updated” $\Rightarrow$ differentiate only terms containing $\mathbf{w}$.
- Hierarchical softmax $\Rightarrow$ product of sigmoids along the path; leaves sum to 1.
- Analogy $\Rightarrow$ nearest neighbour of $b - a + c$ by cosine (exclude a, b, c themselves).

15.11 Scenario Bank

S1. Two words never co-occur but have identical context rows. Their PPMI cosine?

Answer: 1 — similarity comes from shared contexts, not from co-occurring with each other.

S2. Window size grows from 1 to 10. What kind of similarity is emphasised?

Answer: Small windows capture syntactic/functional similarity (words that can substitute); large windows capture topical similarity.

S3. In negative sampling, why not use uniformly random negatives?

Answer: Uniform sampling picks mostly rare, uninformative words; $\text{count}^{3/4}$ balances frequent and rare.

S4. A word appears only once. Which is likelier to give it a sensible vector, CBOW or skip-gram?

Answer: Skip-gram: each occurrence generates several (centre, context) training pairs; CBOW averages it away inside a context sum.

S5. You factorise PPMI with SVD at $k = 300$ for $|V| = 50,000$. Storage vs the full matrix?

Answer: $50,000 \cdot 300 \cdot 2 + 300 \approx 3 \times 10^7$ vs 2.5×10^9 .

15.12 Practice Problems

1. **[FOUNDATION]** $\text{count}(w, c) = 4$, $\text{count}(w) = 20$, $\text{count}(c) = 10$, $N = 1000$. PMI?

Answer: $\ln 20 = 3.00$.

2. **[INTERMEDIATE]** In the BABEL corpus, compute $\text{PPMI}(\text{dog}, \text{sat})$.

Answer: $\ln \frac{1 \cdot 12}{2 \cdot 2} = \ln 3 = 1.10$.

3. **[EXAM]** Hierarchical softmax with $|V| = 2^{20}$: number of sigmoid evaluations per prediction?

Answer: 20.

4. **[ADVANCED]** Show that the gradient of $-\log \sigma(\mathbf{v}^\top \mathbf{u})$ w.r.t. $\mathbf{v}$ vanishes as $\mathbf{v}^\top \mathbf{u} \rightarrow \infty$ and explain what that means for already-well-learned pairs.

15.13 Concept Gauntlet

G1. Cosine similarity of two distinct one-hot vectors?

Answer: 0 — the core defect.

G2. Why not keep negative PMI values?

Answer: Unreliable (few counts) and $-\infty$ for unseen pairs; PPMI clips them.

G3. Is $W_{\text{context}}\mathbf{x}$ with one-hot $\mathbf{x}$ a real matrix multiplication?

Answer: Mathematically yes; computationally it is a column lookup (embedding lookup).

G4. Negative sampling with $k = 0$?

Answer: Only positives: trivially maximised by making all vectors huge and aligned — every word similar to every word.

G5. Does the rank- k SVD keep the k *most frequent* words?

Answer: No — it keeps the k directions of largest singular value for *all* words.

Convolutional Neural Networks

Chapter Overview

Images have spatial structure: nearby pixels are related, and a feature (edge, rock texture) means the same thing wherever it appears. CNNs build this in through **local connectivity**, **weight sharing** and **pooling**. This chapter derives the output-size formula, counts parameters, backpropagates through convolution and pooling, and surveys LeNet $\rightarrow$ ResNet.

16.1 The Convolution Operation

2-D convolution of image I with kernel K (as implemented in deep learning — strictly, cross-correlation):

$$S(i, j) = \sum_{a=0}^{m-1} \sum_{b=0}^{n-1} I(i+a, j+b) K(a, b).$$

For a multi-channel input of depth D , the kernel is $F \times F \times D$ and the sum runs over all channels; each kernel produces one 2-D *feature map*; K kernels give an output of depth K .

Worked Example 16.1: Lumen edge detector, Level 1

Problem. $I = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 0 & 1 & 3 & 1 \\ 2 & 1 & 0 & 0 \\ 1 & 0 & 1 & 2 \end{bmatrix}$, $K = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$, stride 1, no padding. Compute the output.

Solution. Each output = top-left – bottom-right of the 2×2 window: $\begin{bmatrix} 0 & -1 & -1 \\ -1 & 1 & 3 \\ 2 & 0 & -2 \end{bmatrix}$ (size $3 \times 3 = (4-2)/1+1$).

Interpretation. This kernel responds to diagonal intensity change; a CNN *learns* such kernels instead of hand-designing them.

16.2 Output Size and Parameters

Important: CNN arithmetic

Input $W_1 \times H_1 \times D_1$, K filters of size $F \times F \times D_1$, stride S , padding P :

$$W_2 = \frac{W_1 - F + 2P}{S} + 1, \quad H_2 = \frac{H_1 - F + 2P}{S} + 1, \quad D_2 = K.$$

Parameters: $K(F \cdot F \cdot D_1 + 1)$ — independent of image size (weight sharing). Same-size output with $S = 1$: $P = (F - 1)/2$. Pooling ($F \times F$, stride S) uses the same size formula, keeps depth, and has **no parameters**.

Course Paper Question: Quiz 2 and End-Terms, CNN arithmetic

(a) $48 \times 48 \times 3$, 7×7 , $S = 1$, same size: $P = \mathbf{3}$. (b) $P = 0$, $K = 4$: $\frac{48-7}{1} + 1 = 42 \Rightarrow \mathbf{42 \times 42 \times 4}$. (c) Parameters: $4(7 \cdot 7 \cdot 3 + 1) = \mathbf{592}$. (d) $300 \times 300 \times 6 \rightarrow 96 \times 96 \times 10$ with $15 \times 15 \times 6$ kernels, $P = 0$: $\frac{285}{S} + 1 = 96 \Rightarrow S = \mathbf{3}$.

Worked Example 16.2: Counting a whole CNN (End-Term), Level 3

Problem. $32 \times 32 \times 3 \rightarrow \text{Conv1} (16, 3 \times 3, S = 1, P = 0) \rightarrow \text{MaxPool } 2 \times 2/2 \rightarrow \text{Conv2} (32, 3 \times 3) \rightarrow \text{MaxPool } 2 \times 2/2 \rightarrow \text{FC } 64 \rightarrow \text{Output } 10$. Total parameters?

Solution. Conv1: $16(27 + 1) = 448$; output $30 \times 30 \times 16$; pool $\rightarrow 15 \times 15 \times 16$.

Conv2: $32(144 + 1) = 4640$; output $13 \times 13 \times 32$; pool $\rightarrow \lfloor 13/2 \rfloor = 6$: $6 \times 6 \times 32 = 1152$.

FC: $64(1152 + 1) = 73,792$. Output: $10(64 + 1) = 650$. Total **79,530**.

Traps. Pool of odd size: $\lfloor (13 - 2)/2 \rfloor + 1 = 6$. Pools have 0 parameters. The FC layer dominates the count.

16.3 Why Convolution? Sparse Connectivity and Sharing

A fully connected layer from a $32 \times 32 \times 3$ image to a $30 \times 30 \times 16$ map would need $3072 \cdot 14400 \approx 4.4 \times 10^7$ weights; the convolution needs 448. Each output depends only on a local *receptive field*; stacking layers grows it (three 3×3 layers see 7×7 of the input — which is why VGG uses only 3×3 kernels: same receptive field as one 7×7 , fewer parameters ($3 \cdot 9C^2$ vs $49C^2$) and more non-linearities).

16.4 Backpropagation through Convolution and Pooling

Because each weight is used at every position, its gradient is the **sum over positions**:

$$\frac{\partial \mathcal{L}}{\partial w_{ab}} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial S(i,j)} I(i+a, j+b),$$

i.e. a convolution of the input with the output gradient. Gradients for the input are a convolution of the (flipped) weights with the output gradient. **Max pooling** routes each gradient only to the position that was the max; **average pooling** splits it equally.

Course Paper Question: Quiz 2, which filter weights learn?

$$I = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}, 2 \times 2 \text{ filter } \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \end{bmatrix}, S = 1, P = 0, \text{ReLU, no bias.}$$

Solution. Top outputs see only zeros: $o_{11} = o_{12} = 0$ (pre-activation 0, $\text{ReLU}' = 0$). Bottom outputs: $o_{21} = o_{22} = w_3 + w_4$ (the filter's bottom row sits on the ones). Gradients: $\frac{\partial \mathcal{L}}{\partial w_k} = \sum_{ij} \frac{\partial \mathcal{L}}{\partial o_{ij}} \mathbf{1}[\text{active}] I(\cdot)$. w_1, w_2 always multiply zeros $\Rightarrow 0$. **Only** w_3, w_4 can learn (if $w_3 + w_4 > 0$). With $\partial \mathcal{L} / \partial o_{ij} = 1$ and $w_3 + w_4 > 0$: $\frac{\partial \mathcal{L}}{\partial w_3} = 1 \cdot 1 + 1 \cdot 1 = \mathbf{2}$.

16.5 Landmark Architectures

Network	Year	Key idea
LeNet-5	1998	conv–pool–conv–pool–FC for digits; established the template
AlexNet	2012	$227 \times 227 \times 3$ input; $11 \times 11/4$ first conv ($96(11 \cdot 11 \cdot 3 + 1) = 34,944$ params); ReLU, dropout, GPUs; $\approx 60\text{M}$ params (mostly FC)
ZFNet	2013	AlexNet re-tuned via visualisation: $7 \times 7/2$ first layer, wider conv 3–5 (512, 1024, 512 filters)
VGGNet	2014	only 3×3 convs (stride 1, pad 1) and 2×2 pools; 16–19 layers; $\approx 138\text{M}$ params
GoogLeNet	2014	Inception module: parallel 1×1 , 3×3 , 5×5 convs and pooling, concatenated; 1×1 convs reduce depth first; global average pooling instead of big FC; $\approx 5\text{M}$ params
ResNet	2015	residual blocks $\mathbf{h} = F(\mathbf{x}) + \mathbf{x}$: identity skip connections let gradients flow, enabling 152+ layers

Insight: Why 1×1 convolutions and residuals work

A 1×1 conv mixes channels at each pixel: $28 \times 28 \times 192 \rightarrow 28 \times 28 \times 32$ costs $32 \cdot 192$ weights and shrinks the depth before an expensive 5×5 . A residual block only needs to learn a *correction* $F(\mathbf{x})$; if the best is “do nothing”, $F = 0$ is easy. And $\partial \mathbf{h} / \partial \mathbf{x} = \partial F / \partial \mathbf{x} + I$: the identity term keeps gradients from vanishing through depth (compare Section 7.6).

16.6 Walkthroughs: Padding, Stride and Channels

16.6.1 Picturing the sliding window

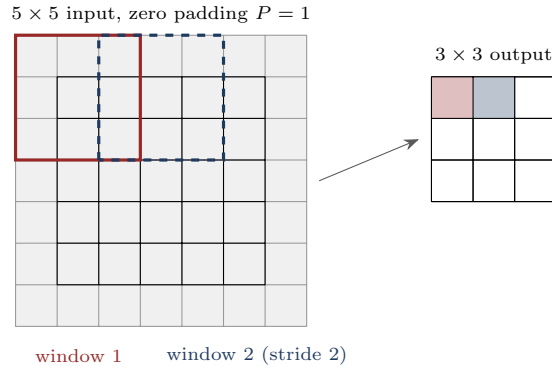


Figure 16.1: A 3×3 kernel with $S = 2$, $P = 1$ on a 5×5 image: $\frac{5-3+2}{2} + 1 = 3$.

Common Trap: Non-integer output size

If $\frac{W-F+2P}{S}$ is not an integer, the hyper-parameters are invalid for a clean tiling (frameworks floor it and silently drop border pixels). In exam “find S ” or “find P ” questions, the intended answer always makes it an integer.

Worked Example 16.3: Two-channel convolution, Level 2

Problem. Input channels $I_1 = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 0 \\ 2 & 1 & 1 \end{bmatrix}$, $I_2 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$; kernel slices $K_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, $K_2 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$;

$S = 1$, $P = 0$, no bias.

Solution. Each output sums over *both* channels. Top-left: K_1 part $1 + 1 = 2$, K_2 part $1 + 1 = 2$, total 4. Full output $\begin{bmatrix} 4 & 0 \\ 2 & 4 \end{bmatrix}$ — **one** 2×2 map from one $2 \times 2 \times 2$ kernel.

Trap. A kernel always spans the full input depth; the output depth equals the *number of kernels*, not the input depth.

16.7 Receptive Field Arithmetic

For layers with kernel F_ℓ and stride S_ℓ , the receptive field in the input grows as

$$r_\ell = r_{\ell-1} + (F_\ell - 1) \prod_{i < \ell} S_i, \quad r_0 = 1.$$

Example: conv $3 \times 3/1 \rightarrow$ pool $2 \times 2/2 \rightarrow$ conv $3 \times 3/1$: $r = 3$, then $3 + 1 \cdot 1 = 4$, then $4 + 2 \cdot 2 = 8$. Pooling makes later layers see *faster-growing* regions — that is how deep layers “see” whole objects.

16.8 Pooling in Depth

Worked Example 16.4: Backprop through max-pooling, Level 2

Problem. 4×4 input $\begin{bmatrix} 1 & 3 & 2 & 0 \\ 4 & 2 & 1 & 5 \\ 0 & 1 & 3 & 2 \\ 2 & 6 & 1 & 1 \end{bmatrix}$, 2×2 max-pool stride 2; upstream gradients $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$. Find the gradient w.r.t. the input.

Solution. Maxima: 4 (row 2, col 1), 5 (row 2, col 4), 6 (row 4, col 2), 3 (row 3, col 3). Gradient $= \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 2 \\ 0 & 0 & 4 & 0 \\ 0 & 3 & 0 & 0 \end{bmatrix}$.

Average pooling instead. Each input gets $\frac{1}{4}$ of its window's gradient: top-left block all 0.25, top-right 0.5, etc.

Global average pooling (GoogLeNet, ResNet) averages each $H \times W$ map to one number: $7 \times 7 \times 512 \rightarrow 512$ with *zero* parameters, versus VGG's flatten-then-FC $25088 \rightarrow 4096$ costing 102,764,544 parameters.

16.9 Architecture Walkthroughs

16.9.1 LeNet-5 (simplified, full connections)

Layer	Operation	Output	Parameters
Input		$32 \times 32 \times 1$	0
C1	6 filters 5×5	$28 \times 28 \times 6$	$6(25 + 1) = 156$
S2	pool $2 \times 2/2$	$14 \times 14 \times 6$	0
C3	16 filters $5 \times 5 \times 6$	$10 \times 10 \times 16$	$16(150 + 1) = 2416$
S4	pool $2 \times 2/2$	$5 \times 5 \times 16$	0
C5	120 filters $5 \times 5 \times 16$	$1 \times 1 \times 120$	$120(400 + 1) = 48,120$
F6	FC	84	$84(121) = 10,164$
Out	FC	10	$10(85) = 850$
Total			61,706

(The original C3 connected each map to only some S2 maps; with full connectivity as above the count is 2416.)

16.9.2 AlexNet's first layer

$227 \times 227 \times 3$, 96 filters 11×11 , stride 4: $\frac{227-11}{4} + 1 = 55 \Rightarrow 55 \times 55 \times 96$; parameters $96(11 \cdot 11 \cdot 3 + 1) = 34,944$; then $3 \times 3/2$ max-pool $\Rightarrow 27 \times 27 \times 96$. Note how the big stride shrinks the map immediately.

16.9.3 Inception: the 1×1 bottleneck in numbers

Input $28 \times 28 \times 192$, want 32 maps from a 5×5 convolution (same padding).

- Direct: $28 \cdot 28 \cdot 32 \times (5 \cdot 5 \cdot 192) = 120,422,400 \approx 120\text{M}$ multiplications.
- With a 1×1 reduction to 16 channels first: $28 \cdot 28 \cdot 16 \cdot 192 + 28 \cdot 28 \cdot 32 \cdot (5 \cdot 5 \cdot 16) = 2.41\text{M} + 10.04\text{M} = 12.4\text{M}$ — **about 10× cheaper**.

The module runs 1×1 , 3×3 , 5×5 convs and a pooling branch in parallel and *concatenates along depth*, letting the network pick the right scale per location.

16.9.4 VGG and ResNet in numbers

Two 3×3 convs (64 channels) = $2 \cdot 9 \cdot 64^2 = 73,728$ weights with a 5×5 receptive field, vs one 5×5 : $25 \cdot 64^2 = 102,400$. For ResNet, with local derivative $F' = 0.1$ per block over 10 blocks, a plain chain passes $0.1^{10} = 10^{-10}$ of the gradient; a residual chain passes $(1 + 0.1)^{10} \approx 2.6$ — the identity path guarantees the gradient never vanishes.

Net	Depth	Params	Remember it by
LeNet	7	60K	template conv-pool-FC
AlexNet	8	60M	ReLU + dropout + GPUs, $11 \times 11/4$
ZFNet	8	~60M	visualisation-tuned AlexNet, $7 \times 7/2$
VGG-16	16	138M	only 3×3 ; FC-heavy
GoogLeNet	22	5M	Inception, 1×1 bottlenecks, GAP
ResNet-152	152	60M	identity skips

16.10 Scenario Bank

- S1.** Your model must classify LUMEN rock images of any size. Which architectural choice makes that possible?
Answer: Global average pooling before the classifier (fully convolutional body): FC layers fix the input size.
- S2.** A 1×1 conv layer maps $56 \times 56 \times 256 \rightarrow 56 \times 56 \times 64$. Parameters?
Answer: $64(256 + 1) = 16,448$.
- S3.** Feature maps shrink to 1×1 after 5 poolings of a 32×32 image. Consequence?
Answer: Deeper layers cannot pool further; use padding / fewer poolings for small images.
- S4.** You double every stride. Receptive field and compute?
Answer: Receptive field grows faster, maps (and compute) shrink about $4\times$ per affected layer.
- S5.** Why can a CNN trained on centred objects fail on objects near the border?
Answer: Padding effects and pooling alignment; translation *equivariance* of convolution is exact only away from borders.

16.11 Practice Problems

- [FOUNDATION]** $64 \times 64 \times 3$, 32 filters 5×5 , $S = 1$, $P = 2$: output and parameters?
Answer: $64 \times 64 \times 32$; $32(76) = 2432$.
- [INTERMEDIATE]** Receptive field of three 3×3 stride-2 convs?
Answer: 3, $3 + 2 \cdot 2 = 7$, $7 + 2 \cdot 4 = 15$.
- [EXAM]** $227 \times 227 \times 3$ with 11×11 , $S = 4$, $P = 0$, 96 filters, then $3 \times 3/2$ pool. Output?
Answer: $27 \times 27 \times 96$.
- [ADVANCED]** Show that for stride 1 the input gradient of a convolution is a full convolution of the output gradient with the 180° -rotated kernel.

16.12 Concept Gauntlet

- G1.** Does doubling the image size change a conv layer's parameter count?
Answer: No; only the output size and compute.
- G2.** A 1×1 convolution on a single-channel image is ...?
Answer: Just scaling each pixel — useful only with many channels.
- G3.** Why does max-pooling give some translation invariance?
Answer: Small shifts inside the window leave the max unchanged.
- G4.** Two stacked 3×3 convs vs one 5×5 (C channels in and out)?
Answer: Same receptive field; $18C^2$ vs $25C^2$ weights; one extra non-linearity.
- G5.** Can a filter weight get zero gradient on an image with non-zero pixels?
Answer: Yes — if every position it touches is zero or every corresponding ReLU is inactive (Quiz 2 example).

Visualising and Fooling Convolutional Networks

Chapter Overview

What has a CNN learned? We can look at (i) which inputs excite a neuron, (ii) which pixels an output depends on, and (iii) what inputs we can *synthesise* by optimising the input instead of the weights. The last idea yields Deep Dream, Deep Art (style transfer) and adversarial examples.

17.1 Receptive Fields and Patches

A neuron in layer ℓ sees a region of the input (its receptive field); for ℓ stacked 3×3 , stride-1 convs, it is $(2\ell + 1) \times (2\ell + 1)$. Feed many images, record which patches maximally activate the neuron: early layers respond to edges/colours, deeper layers to textures, parts, objects.

17.2 Visualising Filters and Occlusion

- **Filters as images:** first-layer weights look like the patterns they detect (edges, blobs), because $\mathbf{w}^\top \mathbf{x}$ is largest when $\mathbf{x} \propto \mathbf{w}$ (for fixed $\|\mathbf{x}\|$).
- **Occlusion experiments:** slide a grey patch over the image and record the drop in the correct class's probability; a large drop marks an important region.

17.3 Gradients with Respect to the Input

Treat the pixels as variables: $\partial h / \partial \mathbf{x}$ (a **saliency map**) shows how each pixel affects neuron h ; it is computed by ordinary backprop continued to the input. Plain gradients are noisy.

Guided backpropagation. At each ReLU, during the backward pass, pass a gradient only if (a) the forward input was positive (normal ReLU rule) *and* (b) the incoming gradient is positive. Keeping only positive influences gives much cleaner images of “what excites this neuron”.

17.4 Optimising the Input

- **Class visualisation:** find $\mathbf{x}^* = \arg \max_{\mathbf{x}} (S_c(\mathbf{x}) - \lambda \|\mathbf{x}\|^2)$ by gradient *ascent* on the image, weights frozen.
- **Deep Dream:** pick a layer, and repeatedly change the image to *increase* that layer's activations ($\mathbf{x} \leftarrow \mathbf{x} + \eta \nabla_{\mathbf{x}} \frac{1}{2} \|\mathbf{h}_\ell\|^2$). Whatever the layer faintly detects gets amplified — patterns “appear” in clouds.
- **Deep Art (neural style transfer):** generate an image G matching the *content* of C and the *style* of S :

$$\mathcal{L}(G) = \alpha \|A_\ell(C) - A_\ell(G)\|^2 + \beta \sum_{\ell} \|\mathcal{G}_\ell(S) - \mathcal{G}_\ell(G)\|_F^2, \quad \mathcal{G}_\ell = A_\ell^\top A_\ell,$$

where A_ℓ holds layer- ℓ activations with one column per channel (rows = positions). The **Gram matrix** $\mathcal{G}_\ell$ keeps channel correlations (which textures co-occur) but discards *where* they occur — exactly “style”.

17.5 Fooling CNNs: Adversarial Examples

A tiny, human-invisible perturbation can flip a prediction. Fast gradient sign method:

$$\mathbf{x}_{\text{adv}} = \mathbf{x} + \epsilon \operatorname{sign}(\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, y)).$$

Why so effective: for a linear score $\mathbf{w}^\top \mathbf{x}$, this changes the score by $\epsilon \|\mathbf{w}\|_1 = \epsilon \sum_i |w_i|$ — in high dimension (e.g. 150,000 pixels) a change of $\epsilon = 0.007$ per pixel can shift the score by hundreds. One can also optimise an

input from noise until the network is 99% sure it is a “school bus” — images unrecognisable to humans.

Worked Example 17.1: How much can ϵ move a score? Level 3

Problem. A linear LUMEN classifier has $n = 10,000$ inputs with $|w_i| = 0.05$ on average. Budget $\epsilon = 0.01$ per pixel. Maximum score change?

Solution. $\epsilon \sum |w_i| = 0.01 \cdot 10000 \cdot 0.05 = 5$ — enough to flip most decisions, although no pixel changed by more than 1%.

Interpretation. Adversarial vulnerability comes from *linearity in high dimensions*, not from some exotic non-linearity.

17.6 Walkthrough: Three Backward Rules at a ReLU

All gradient-based visualisations run backprop to the *input*; they differ only in what happens at each ReLU.

Worked Example 17.2: Plain, deconvnet and guided backprop, Level 2

Problem. A ReLU layer received forward inputs $\mathbf{a} = [2, -1, 3, 0.5]$; the gradient arriving from above is $\mathbf{g} = [-0.5, 0.8, 1.2, -0.3]$. Compute the gradient passed down under each rule.

Solution.

Rule	Mask	Result
Plain backprop	$\mathbf{1}[a > 0]$ (forward sign)	$[-0.5, 0, 1.2, -0.3]$
Deconvnet	$\mathbf{1}[g > 0]$ (backward sign)	$[0, 0.8, 1.2, 0]$
Guided backprop	both	$[0, 0, 1.2, 0]$

Interpretation. Guided backprop keeps only paths that were active *and* increase the neuron, suppressing negative evidence — hence its sharp, clean images.

17.7 Occlusion Walkthrough

Slide a grey $k \times k$ patch over the image; at each position record $P(\text{correct class})$. For a LUMEN “basalt” image: occluding sky $0.91 \rightarrow 0.90$, occluding the rock’s texture $0.91 \rightarrow 0.12$. The heat-map of drops localises the evidence *without any gradients* — model-agnostic, but costs one forward pass per patch position.

17.8 Style Transfer in Numbers

Worked Example 17.3: Gram matrix ignores position, Level 2

Problem. A layer has 3 positions and 2 channels: $A = \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 3 \end{bmatrix}$ (rows: positions). Compute $\mathcal{G} = A^\top A$,

then shuffle the rows and recompute.

Solution. $\mathcal{G} = \begin{bmatrix} 1+4+0 & 0+2+0 \\ 2 & 0+1+9 \end{bmatrix} = \begin{bmatrix} 5 & 2 \\ 2 & 10 \end{bmatrix}$. Any row permutation gives the same $\mathcal{G}$ (it is a sum over positions).

Interpretation. Diagonal: how strongly each feature (texture) occurs; off-diagonal: which features co-occur — the “style” — with spatial layout (the “content”) discarded. Content loss compares A itself, which keeps positions.

Optimisation starts from noise (or the content image) and updates *pixels* with gradient descent on $\alpha \mathcal{L}_{\text{content}} + \beta \mathcal{L}_{\text{style}}$; the ratio α/β trades faithfulness for stylisation.

17.9 Adversarial Walkthrough

Worked Example 17.4: One FGSM step, Level 2

Problem. Pixels $\mathbf{x} = [0.5, 0.2, 0.9]$, loss gradient $\nabla_{\mathbf{x}}\mathcal{L} = [0.3, -1.2, 0.05]$, $\epsilon = 0.1$, valid range $[0, 1]$.

Solution. $\text{sign} = [+1, -1, +1]$: $\mathbf{x}_{\text{adv}} = [0.6, 0.1, 1.0]$ (clip to $[0, 1]$).

Notes. The step *ascends* the loss (opposite of training). Only the sign is used, so every pixel moves by exactly ϵ — maximal L_{∞} -bounded damage to first order. Training on such examples (*adversarial training*) is a data-augmentation defence (Chapter 12).

Technique	Optimises / computes	Fixed	Answers the question
Patch mining	nothing (search dataset)	all	which inputs excite a neuron?
Occlusion	forward passes	all	which regions matter?
Saliency / guided BP	$\partial h / \partial \mathbf{x}$	all	which pixels influence h ?
Class visualisation	$\mathbf{x}$ (ascent on S_c)	weights	what does class c “look like”?
Deep Dream	$\mathbf{x}$ (ascent on $\ \mathbf{h}_{\ell}\ ^2$)	weights	what does layer ℓ see?
Style transfer	$\mathbf{x}$ (descent)	weights	combine content + Gram style
FGSM	$\mathbf{x}$ (one ascent step on loss)	weights	how fragile is the model?

17.10 Scenario Bank

S1. A saliency map highlights the background snow, not the wolf. What does that reveal?

Answer: The classifier exploits a spurious correlation (dataset bias): wolves photographed on snow.

S2. Can Deep Dream be run on a network with random weights?

Answer: Yes, but it amplifies meaningless patterns; trained weights are what make the hallucinations object-like.

S3. Why does FGSM transfer between different models?

Answer: Models trained on similar data learn similar (largely linear) decision directions.

17.11 Concept Gauntlet

G1. In Deep Dream, what is updated — weights or pixels?

Answer: Pixels; weights are frozen.

G2. Why does the Gram matrix capture style but not content?

Answer: It sums over positions, discarding spatial arrangement.

G3. Guided backprop vs deconvnet vs plain backprop differ only at ...?

Answer: The ReLU backward rule.

G4. Is an adversarial perturbation just random noise of the same size?

Answer: No — random noise mostly cancels ($\sim \sqrt{n}$ growth); the sign of the gradient aligns every pixel ($\sim n$ growth).

End of Part III. Part IV: sequences, gates, attention and Transformers.

Part IV

Sequences, Gates, Attention and Transformers (Weeks 10–12)

Recurrent Neural Networks and BPTT

Chapter Overview

Feedforward networks take fixed-size inputs and treat inputs independently. Sequences (sentences, PULSE vital-sign streams) have variable length and order-dependent meaning. An RNN applies *the same* function at every time step, carrying a state. Training unrolls it in time — backpropagation through time (BPTT) — which exposes the vanishing/exploding gradient problem in its sharpest form.

18.1 The RNN Model

Definition: Recurrent neural network

$$\mathbf{s}_t = \sigma(U\mathbf{x}_t + W\mathbf{s}_{t-1} + \mathbf{b}), \quad \hat{\mathbf{y}}_t = O(V\mathbf{s}_t + \mathbf{c}),$$

with the *same* $U, W, V, \mathbf{b}, \mathbf{c}$ at every t (σ : sigmoid or tanh). Parameter sharing across time makes the model length-agnostic: longer sequences add no parameters.

Modes: one-to-many (image captioning), many-to-one (sentiment), many-to-many synchronous (POS tagging), many-to-many asynchronous (translation — Chapter 20). Total loss $\mathcal{L} = \sum_t \mathcal{L}_t$.

18.2 Backpropagation Through Time

$\mathcal{L}_t$ depends on W through *every* earlier state. With $\mathbf{a}_j = W\mathbf{s}_{j-1} + \dots$ and $\mathbf{s}_j = \sigma(\mathbf{a}_j)$:

$$\frac{\partial \mathcal{L}_t}{\partial W} = \sum_{k=1}^t \frac{\partial \mathcal{L}_t}{\partial \mathbf{s}_t} \left(\prod_{j=k+1}^t \frac{\partial \mathbf{s}_j}{\partial \mathbf{s}_{j-1}} \right) \frac{\partial^+ \mathbf{s}_k}{\partial W}, \quad \frac{\partial \mathbf{s}_j}{\partial \mathbf{s}_{j-1}} = \text{diag}(\sigma'(\mathbf{a}_j))W,$$

where ∂^+ is the “immediate” derivative treating $\mathbf{s}_{k-1}$ as constant.

18.2.1 Vanishing and exploding gradients

The product of $t - k$ Jacobians behaves like $(\gamma\lambda)^{t-k}$ where γ bounds σ' ($\frac{1}{4}$ sigmoid, 1 tanh) and λ is the scale of W : $\gamma\lambda < 1$ vanishes (long-range dependencies not learned), $\gamma\lambda > 1$ explodes (unstable training).

Course Paper Question: End-Term, two RNNs

$\partial \mathbf{s}_j / \partial \mathbf{s}_{j-1} = \text{diag}(\sigma'(\mathbf{a}_j))W$, $\sigma' = 0.5$ everywhere. (a) $W = 0.8I$: $\partial \mathbf{s}_6 / \partial \mathbf{s}_1 = (0.5 \cdot 0.8)^5 I = 0.01024 I$, trace **0.020**. (b) $W = 3I$: $(1.5)^5 I = 7.594 I$, trace **15.19**. (c) First **vanishes**, second **explodes**.

Trap. Five factors, not six: from $\mathbf{s}_1$ to $\mathbf{s}_6$ there are $j = 2, \dots, 6$.

Worked Example 18.1: Pulse remembers a spike, Level 3

Problem. Scalar RNN $s_t = \tanh(ws_{t-1} + ux_t)$, $w = 0.5$, $u = 1$, $s_0 = 0$, inputs $x = (1, 0, 0)$ (one spike). Compute s_1, s_2, s_3 and $\partial s_3 / \partial s_1$.

Solution. $s_1 = \tanh 1 = 0.7616$; $s_2 = \tanh(0.3808) = 0.3634$; $s_3 = \tanh(0.1817) = 0.1797$. $\frac{\partial s_3}{\partial s_1} = w(1 - s_2^2) \cdot w(1 - s_3^2) = 0.5(0.8679) \cdot 0.5(0.9677) = \mathbf{0.210}$.

Interpretation. The memory of the spike decays by ≈ 0.46 per step; after 20 steps its influence is $\sim 10^{-7}$. The RNN cannot learn that a spike 20 steps ago matters.

18.2.2 Remedies

Truncated BPTT: backpropagate only τ steps (cheaper, bounds explosion, but cannot learn dependencies longer than τ). **Gradient clipping:** if $\|\mathbf{g}\| > \theta$, rescale $\mathbf{g} \leftarrow \theta \mathbf{g} / \|\mathbf{g}\|$ (fixes explosion only). **Gates** (next chapter) fix vanishing.

18.3 Picturing the Unrolled RNN

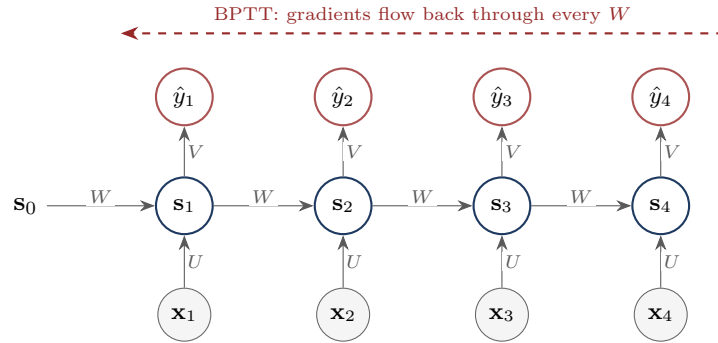


Figure 18.1: An RNN unrolled over 4 steps: the *same* U, W, V at every step.

18.4 BPTT Walkthrough with Numbers

Worked Example 18.2: Pulse computes $\partial\mathcal{L}/\partial w$ through time, Level 4

Problem. Scalar RNN $s_t = \tanh(ws_{t-1} + ux_t)$, $w = 0.5$, $u = 1$, $s_0 = 0$, $x = (1, 0, 0)$, loss only at the end: $\mathcal{L} = \frac{1}{2}(s_3 - y)^2$ with $y = 0.5$. Compute $\partial\mathcal{L}/\partial w$ as a sum over time and check it.

Given. $s_1 = 0.7616$, $s_2 = 0.3634$, $s_3 = 0.1797$ (Chapter 18).

Strategy. $\frac{\partial\mathcal{L}}{\partial w} = \frac{\partial\mathcal{L}}{\partial s_3} \sum_{k=1}^3 \left(\prod_{j=k+1}^3 \frac{\partial s_j}{\partial s_{j-1}} \right) \frac{\partial^+ s_k}{\partial w}$, with $\frac{\partial s_j}{\partial s_{j-1}} = w(1 - s_j^2)$ and $\frac{\partial^+ s_k}{\partial w} = (1 - s_k^2)s_{k-1}$.

Solution. $\partial\mathcal{L}/\partial s_3 = 0.1797 - 0.5 = -0.3203$.

- $k = 3$: $(1 - s_3^2)s_2 = 0.9677 \cdot 0.3634 = 0.3517 \Rightarrow$ term -0.1126 .
- $k = 2$: $w(1 - s_3^2) \cdot (1 - s_2^2)s_1 = 0.4839 \cdot 0.8679 \cdot 0.7616 = 0.3198 \Rightarrow$ term -0.1024 .
- $k = 1$: $(1 - s_1^2)s_0 = 0 \Rightarrow$ term 0 .

$\frac{\partial\mathcal{L}}{\partial w} = -0.2151$.

Verification. Finite difference on w : $-0.21506 \checkmark$.

Interpretation. The $k = 2$ term (“how w acted at step 2, carried to step 3”) is as large as the direct term here; for long sequences, the far-back terms carry products of many factors < 1 and vanish — the network then learns only from recent steps.

18.5 Counting Parameters

Input dimension n , hidden d , output k : $U \in \mathbb{R}^{d \times n}$, $W \in \mathbb{R}^{d \times d}$, $\mathbf{b} \in \mathbb{R}^d$, $V \in \mathbb{R}^{k \times d}$, $\mathbf{c} \in \mathbb{R}^k$. E.g. $n = 100$, $d = 64$, $k = 10$: $6400 + 4096 + 64 + 640 + 10 = \mathbf{11,210}$ — the same for any sequence length.

18.6 Remedies in Numbers

Gradient clipping: $\mathbf{g} = [3, 4]$ ($\|\mathbf{g}\| = 5$), threshold $1 \Rightarrow \mathbf{g} \leftarrow [0.6, 0.8]$: direction kept, length capped.

Truncated BPTT: a 1000-step sequence with $\tau = 50$ is processed in 20 chunks; the hidden state is carried forward across chunks (forward memory is unlimited) but gradients stop at chunk boundaries (backward credit ≤ 50 steps).

Teacher forcing: during training feed the true y_{t-1} as the next input; at test time the model consumes its own predictions, so early errors can compound (*exposure bias*).

18.7 Choosing the Sequence Mode

Task	Mode	Loss
PULSE alarm from a vital stream	many-to-one	at last step only
Part-of-speech tagging	many-to-many (synchronous)	every step
Image captioning	one-to-many	every output step
BABEL translation	many-to-many (encoder–decoder)	decoder steps

18.8 Scenario Bank

- S1.** A language model must match a verb with a subject 40 words earlier; your vanilla RNN fails. Diagnose and fix.
Answer: Vanishing gradients over 40 steps; use LSTM/GRU (gated additive path), possibly with attention.
- S2.** Loss suddenly becomes NaN after thousands of steps.
Answer: Exploding gradient spike; add gradient clipping (and/or lower η).
- S3.** Truncated BPTT with $\tau = 20$: can the model still *use* information from 100 steps ago at test time?
Answer: In principle yes (the state carries it), but it was never trained to — so in practice rarely.

18.9 Practice Problems

- [FOUNDATION]** $\partial \mathbf{s}_{10} / \partial \mathbf{s}_4$ has how many Jacobian factors?
Answer: 6.
- [INTERMEDIATE]** With $\sigma' = 0.25$ and $W = 2I$, per-step factor and 10-step product?
Answer: 0.5; $0.5^{10} \approx 10^{-3}$ — vanishing despite $\|W\| > 1$.
- [EXAM]** Clip $\mathbf{g} = [6, 8]$ to norm 5.
Answer: $[3, 4]$.

18.10 Concept Gauntlet

- G1.** Does an RNN processing a 1000-step sequence have more parameters than one processing 10 steps?
Answer: No — shared across time.
- G2.** Does gradient clipping solve vanishing gradients?
Answer: No; it only caps large norms.
- G3.** Why is the gradient a *sum* over k ?
Answer: W is used at every step; shared parameters sum their per-use gradients (as in CNNs).

Gated Recurrent Units and LSTMs

Chapter Overview

The RNN overwrites its whole state every step through a squashing function. Gates let the network *selectively* read, write and forget, and — crucially — create an additive path through time along which gradients need not vanish.

19.1 LSTM

Definition: LSTM (course notation)

$$\begin{aligned} \mathbf{o}_t &= \sigma(W_o \mathbf{h}_{t-1} + U_o \mathbf{x}_t + \mathbf{b}_o), & \mathbf{i}_t &= \sigma(W_i \mathbf{h}_{t-1} + U_i \mathbf{x}_t + \mathbf{b}_i), & \mathbf{f}_t &= \sigma(W_f \mathbf{h}_{t-1} + U_f \mathbf{x}_t + \mathbf{b}_f), \\ \tilde{\mathbf{s}}_t &= \tanh(W \mathbf{h}_{t-1} + U \mathbf{x}_t + \mathbf{b}), & \mathbf{s}_t &= \mathbf{f}_t \odot \mathbf{s}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{s}}_t, & \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{s}_t). \end{aligned}$$

Forget gate $\mathbf{f}$: how much old cell state to keep. Input gate $\mathbf{i}$: how much new content to write. Output gate $\mathbf{o}$: how much of the cell to expose. Parameters: 4 sets of $(W, U, \mathbf{b})$ — four times a vanilla RNN.

Worked Example 19.1: One LSTM step, Level 2

Problem. Scalar LSTM with gate pre-activations $f : 2, i : 0, o : 1$, candidate pre-activation 1, previous cell $s_{t-1} = 1$. Compute s_t, h_t .

Solution. $f = \sigma(2) = 0.881, i = 0.5, o = 0.731, \tilde{s} = \tanh 1 = 0.762$. $s_t = 0.881 \cdot 1 + 0.5 \cdot 0.762 = \mathbf{1.262}$; $h_t = 0.731 \tanh(1.262) = 0.731 \cdot 0.851 = \mathbf{0.622}$.

Note. The cell state can exceed 1; only h_t is squashed.

19.2 Why LSTMs Solve Vanishing Gradients

Along the cell path, $\partial \mathbf{s}_t / \partial \mathbf{s}_{t-1}$ contains the direct term $\text{diag}(\mathbf{f}_t)$ (plus indirect terms through the gates). Backpropagating k steps multiplies by $\prod \mathbf{f}$: the network *learns* forget gates near 1 where information must be kept, so the product need not shrink — unlike the RNN's forced $\prod \sigma' W$. Gradients vanish only when the network *chooses* to forget. (LSTMs can still explode; clipping is used.)

19.3 GRU

Definition: GRU (course notation)

$$\begin{aligned} \mathbf{o}_t &= \sigma(W_o \mathbf{s}_{t-1} + U_o \mathbf{x}_t + \mathbf{b}_o), & \mathbf{i}_t &= \sigma(W_i \mathbf{s}_{t-1} + U_i \mathbf{x}_t + \mathbf{b}_i), \\ \tilde{\mathbf{s}}_t &= \tanh(W(\mathbf{o}_t \odot \mathbf{s}_{t-1}) + U \mathbf{x}_t + \mathbf{b}), & \mathbf{s}_t &= (1 - \mathbf{i}_t) \odot \mathbf{s}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{s}}_t. \end{aligned}$$

No separate cell state or output gate; input and forget are *tied* ($\mathbf{f} = 1 - \mathbf{i}$). Three weight sets instead of four.

Worked Example 19.2: One GRU step, Level 1

$i_t = \sigma(0) = 0.5, s_{t-1} = 0.8, \tilde{s}_t = \tanh(0.5) = 0.462$: $s_t = 0.5(0.8) + 0.5(0.462) = \mathbf{0.631}$ — a convex blend of old and new.

	RNN	LSTM	GRU
State	$\mathbf{s}_t$	cell $\mathbf{s}_t$ + output $\mathbf{h}_t$	$\mathbf{s}_t$
Gates	none	forget, input, output	update ($\mathbf{i}$), reset ($\mathbf{o}$)
Weight sets	1	4	3
Gradient path	$\prod \text{diag}(\sigma')W$	$\prod \mathbf{f}_t$ (learned)	$\prod (1 - \mathbf{i}_t)$

19.4 Picturing the LSTM Cell

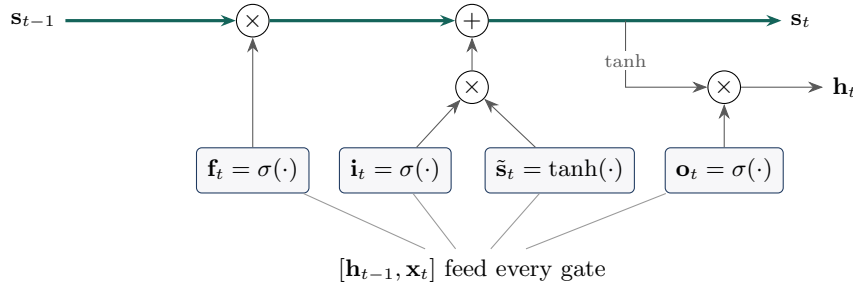


Figure 19.1: The LSTM cell. The top (bold) line is the cell-state highway: only a multiplication by $\mathbf{f}_t$ and an addition — no squashing.

19.5 Two-Step Walkthrough

Worked Example 19.3: A scalar LSTM over two inputs, Level 3

Problem. All gates and the candidate share pre-activation $0.5h_{t-1} + x_t$ (no bias, for arithmetic simplicity). $h_0 = s_0 = 0$; inputs $x_1 = 1$, $x_2 = -1$.

Solution. Step 1: pre = 1: $f = i = o = \sigma(1) = 0.731$, $\tilde{s} = \tanh 1 = 0.762$. $s_1 = 0.731 \cdot 0 + 0.731 \cdot 0.762 = 0.557$, $h_1 = 0.731 \tanh(0.557) = 0.370$.

Step 2: pre = $0.5(0.370) - 1 = -0.815$: $f = i = o = 0.307$, $\tilde{s} = -0.672$. $s_2 = 0.307 \cdot 0.557 + 0.307(-0.672) = -0.035$, $h_2 = -0.011$.

Interpretation. The negative input closed the forget gate (0.307) and wrote strongly negative content: the cell was mostly *overwritten*. A trained LSTM learns when to do this and when to hold ($f \approx 1$).

19.6 How Long Can Each Model Remember?

Over 50 steps: an RNN with per-step factor 0.46 (the PULSE example) retains $0.46^{50} \approx 10^{-17}$ of a gradient; an LSTM with forget gate 0.95 retains $0.95^{50} \approx 0.077$. The LSTM does not *guarantee* memory; it makes long memory *learnable*.

Worked Example 19.4: Parameter counts, Level 1

$n = 100$, $d = 64$: RNN core $d^2 + nd + d = 10,560$; GRU $3\times$ that = 31,680; LSTM $4\times$ = 42,240 (output layer extra).

19.7 Designing with Gates: Scenarios

S1. To carry “the subject is plural” across a long clause, which gate settings?

Answer: Write it once ($\mathbf{i} \approx 1$ at the subject), then $\mathbf{f} \approx 1$, $\mathbf{i} \approx 0$ for that unit; expose it ($\mathbf{o} \approx 1$) at the verb.

S2. A new sentence starts in a stream; old content is irrelevant.

Answer: Forget gate ≈ 0 resets the cell.

S3. GRU with update gate $\mathbf{i}_t \equiv 0$?

Answer: State copied forever; GRU becomes a pure memory. $\mathbf{i}_t \equiv 1$ with reset $\mathbf{o}_t \equiv 1$: a vanilla-RNN-like update.

S4. Initialising the forget-gate bias to a positive value (e.g. 1): why?

Answer: Starts with $\mathbf{f} \approx 0.73$ so gradients flow early; the network remembers by default until it learns to forget.

Beyond the Core Syllabus: Bidirectional and stacked RNNs

A bidirectional RNN runs one RNN left-to-right and another right-to-left and concatenates their states, so each position sees its whole context (tagging, encoders). Stacking RNN layers (the states of one layer are the inputs of the next) adds depth per time step.

19.8 Practice Problems

1. **[FOUNDATION]** $f = 0.9$, $i = 0.2$, $s_{t-1} = 2$, $\tilde{s} = 1$, $o = 0.5$: s_t and h_t ?

Answer: $1.8 + 0.2 = 2.0$; $0.5 \tanh 2 = 0.482$.

2. **[INTERMEDIATE]** GRU: $i_t = 0.25$, $s_{t-1} = 1$, $\tilde{s}_t = -1$: s_t ?

Answer: $0.75 - 0.25 = 0.5$.

3. **[EXAM]** Which LSTM gate being near 0 blocks gradient flow along the cell path?

Answer: The forget gate.

19.9 Concept Gauntlet

G1. An LSTM with $\mathbf{f}_t \equiv 1$, $\mathbf{i}_t \equiv 0$?

Answer: Cell state frozen forever — perfect memory, no learning of new content.

G2. Why sigmoid (not tanh) for gates?

Answer: Gates are fractions in $(0, 1)$: “how much to let through”.

G3. Parameter count of an LSTM layer with input n , hidden d ?

Answer: $4(d^2 + nd + d)$.

Encoder–Decoder Models and Attention

Chapter Overview

Translation maps a sequence to a sequence of different length. An encoder RNN reads the input; a decoder RNN writes the output. Squeezing everything into one vector fails for long inputs; **attention** lets the decoder look back at all encoder states, choosing where to look at every step.

20.1 Encoder–Decoder

Encoder: $\mathbf{h}_j = \text{RNN}(\mathbf{h}_{j-1}, \mathbf{x}_j)$, $j = 1..T$. Decoder starts from $\mathbf{s}_0 = \mathbf{h}_T$ and generates y_t with $\mathbf{s}_t = \text{RNN}(\mathbf{s}_{t-1}, e(\hat{y}_{t-1}))$, $P(y_t) = \text{softmax}(V\mathbf{s}_t + \mathbf{c})$. Training with *teacher forcing*: feed the true previous word, not the predicted one. For images, the encoder is a CNN ($\mathbf{s}_0 = \text{fc7}(I)$).

Course Paper Question: End-Term, why quality drops on long sentences

The fixed-dimensional $\mathbf{s}_0$ becomes an information bottleneck as length grows. Not: vocabulary size, teacher-forcing validity, or one-hot dimension (which depends on vocabulary, not sentence length).

20.2 Attention

At decoder step t , score every encoder state, normalise, and take a weighted average:

$$e_{jt} = f_{\text{ATT}}(\mathbf{s}_{t-1}, \mathbf{h}_j), \quad \alpha_{jt} = \frac{\exp(e_{jt})}{\sum_{j'} \exp(e_{j't})}, \quad \mathbf{c}_t = \sum_j \alpha_{jt} \mathbf{h}_j.$$

Additive (Bahdanau) scoring: $e_{jt} = V_{\text{att}}^\top \tanh(U_{\text{att}} \mathbf{s}_{t-1} + W_{\text{att}} \mathbf{h}_j)$. $\mathbf{c}_t$ is fed to the decoder at step t . The α_{jt} form an interpretable alignment heat-map; no supervision of the alignment is needed — it is learned through backprop.

Course Paper Question: End-Term, additive attention by hand

$d = 2$, $U = W = I$, $V = [1, 1]^\top$, $\mathbf{s}_{t-1} = [0, 1]$, $\mathbf{c}_1 = [1, 0]$, $\mathbf{c}_2 = [-1, 1]$, $\mathbf{c}_3 = [2, -1]$ (encoder states).

Scores. $\mathbf{s} + \mathbf{c}_1 = [1, 1] \Rightarrow e_1 = 2(0.7616) = 1.523$; $\mathbf{s} + \mathbf{c}_2 = [-1, 2] \Rightarrow e_2 = -0.7616 + 0.9640 = 0.202$; $\mathbf{s} + \mathbf{c}_3 = [2, 0] \Rightarrow e_3 = 0.964$.

Weights. $e^e = 4.587, 1.224, 2.622$; sum 8.433: $\alpha = [0.544, 0.145, 0.311]$. Highest: $\mathbf{c}_1$.

Context. First component $0.544(1) + 0.145(-1) + 0.311(2) = 1.02$.

20.3 Attention over Images and Hierarchical Attention

Replace the single fc7 vector by a convolutional feature map: a $7 \times 7 \times 2048$ ResNet map gives **49** encoder states (one per location), each **2048**-dimensional (End-Term); VGG's $14 \times 14 \times 512$ conv5 gives 196 states of dimension 512. The decoder attends to image regions word by word.

Hierarchical attention (e.g. document classification): word-level attention builds each sentence vector from its words; sentence-level attention builds the document vector from sentences. Two levels of weighted averaging, each with its own context vector.

20.4 Picturing Attention

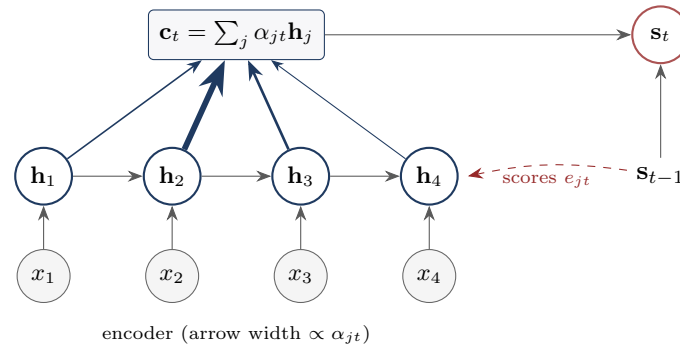


Figure 20.1: At decoder step t , the previous state scores every encoder state; the context is their weighted sum.

20.5 Scoring Functions Compared

Name	e_{jt}	Parameters / note
Dot	$\mathbf{s}_{t-1}^\top \mathbf{h}_j$	none; needs equal dimensions
General (multiplicative)	$\mathbf{s}_{t-1}^\top W \mathbf{h}_j$	$W \in \mathbb{R}^{d \times d}$
Additive (course)	$V^\top \tanh(U \mathbf{s}_{t-1} + W \mathbf{h}_j)$	U, W, V ; a small MLP
Scaled dot (Transformer)	$\mathbf{q}^\top \mathbf{k} / \sqrt{d_k}$	projections W_Q, W_K

Worked Example 20.1: Same state, different scores, Level 2

Problem. $\mathbf{s} = [1, 2]$, encoder states $\mathbf{h}_1 = [1, 0]$, $\mathbf{h}_2 = [0, 1]$, $\mathbf{h}_3 = [1, 1]$. Compare dot, scaled-dot ($d = 2$) and general with $W = \text{diag}(1, -1)$.

Solution. Dot: $[1, 2, 3] \Rightarrow \alpha = [0.090, 0.245, 0.665]$. Scaled: $[0.71, 1.41, 2.12] \Rightarrow [0.140, 0.284, 0.576]$ (flatter). General: $\mathbf{s}^\top W \mathbf{h}_j = [1, -2, -1] \Rightarrow [0.844, 0.042, 0.114]$ — a learned W can completely change *what* counts as relevant.

20.6 Training and Decoding

Training loss: $\mathcal{L} = -\sum_{t=1}^{T'} \log P(y_t^* | y_{<t}^*, \mathbf{x})$ with teacher forcing. E.g. probabilities of the correct words 0.5, 0.8, 0.9 give $\mathcal{L} = 0.693 + 0.223 + 0.105 = 1.02$.

Greedy decoding picks the most probable word each step. It can miss better sequences:

Worked Example 20.2: Greedy vs beam search, Level 3

Problem. Step 1: $P(A) = 0.5$, $P(B) = 0.4$. Then $P(\cdot | A)$ has max 0.4; $P(\cdot | B)$ has max 0.9.

Solution. Greedy: A then best = $0.5 \cdot 0.4 = 0.20$. Beam search with width 2 keeps both A and B : B -path gives $0.4 \cdot 0.9 = 0.36$ — better.

Note. Beam search keeps the k best partial sequences at every step (cost $\times k$). (Standard decoding technique; beyond the course core.)

20.7 Attention over Images: Full Walkthrough

VGG conv5: $14 \times 14 \times 512 \Rightarrow L = 196$ annotation vectors $\mathbf{a}_j \in \mathbb{R}^{512}$. With decoder state $\mathbf{s} \in \mathbb{R}^d$ and additive attention, parameters $U \in \mathbb{R}^{d \times d}$, $W \in \mathbb{R}^{d \times 512}$, $V \in \mathbb{R}^d$ — independent of L . Each word step produces 196 weights summing to 1; plotting them over the 14×14 grid shows *where* the model looked while emitting “dog” or “frisbee”. Why not fc7: it is a single vector with no spatial positions to choose among.

20.8 Hierarchical Attention in Equations

For document $\rightarrow$ sentences $i \rightarrow$ words t (encoder states $\mathbf{h}_{it}$):

$$\mathbf{u}_{it} = \tanh(W_w \mathbf{h}_{it} + \mathbf{b}_w), \quad \alpha_{it} = \frac{\exp(\mathbf{u}_{it}^\top \mathbf{u}_w)}{\sum_{t'} \exp(\mathbf{u}_{it'}^\top \mathbf{u}_w)}, \quad \mathbf{s}_i = \sum_t \alpha_{it} \mathbf{h}_{it},$$

then the same pattern over sentence encodings with a sentence-level context vector $\mathbf{u}_s$ gives the document vector $\mathbf{d} = \sum_i \beta_i \mathbf{h}_i$. $\mathbf{u}_w, \mathbf{u}_s$ are learned “what is informative?” queries. The course’s **dialog** example uses the same two-level idea without attention: an utterance-level RNN encodes each utterance’s words; a second RNN encodes the sequence of utterance vectors; the decoder conditions on that.

20.9 Scenario Bank

S1. An attention heat-map for English $\rightarrow$ French is mostly diagonal with one crossed block. Meaning?

Answer: Monotone alignment except where word order differs (e.g. adjective–noun swap).

S2. Attention weights are almost uniform for every step. Likely problem?

Answer: Scores too similar — under-trained attention, or scores squashed (e.g. saturating tanh); the model ignores alignment.

S3. Can attention help a many-to-one sentiment RNN?

Answer: Yes: attention pooling over all hidden states instead of using only the last state.

20.10 Concept Gauntlet

G1. Do attention weights at step t sum to 1 over output words or over input positions?

Answer: Over input positions j .

G2. Can attention be computed for all t in parallel in an RNN decoder?

Answer: No — e_{jt} needs $\mathbf{s}_{t-1}$, produced sequentially. Transformers remove this.

G3. If all e_{jt} are equal, $\mathbf{c}_t$ is ...?

Answer: The plain average of encoder states.

Transformers

Chapter Overview

Attention alone, without recurrence: every position attends to every other position in parallel. The Transformer stacks self-attention, feed-forward layers, residual connections and layer normalisation, with positional encodings supplying order.

21.1 Scaled Dot-Product Self-Attention

Stack token embeddings as rows of $X \in \mathbb{R}^{T \times d_{\text{model}}}$. With learned $W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad Z = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Row i : $\mathbf{z}_i = \sum_j \alpha_{ij} \mathbf{v}_j$ with $\alpha_{ij} = \text{softmax}_j(\mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{d_k})$. **Why $\sqrt{d_k}$:** if $\mathbf{q}, \mathbf{k}$ have independent unit-variance components, $\text{Var}(\mathbf{q} \cdot \mathbf{k}) = d_k$; dividing by $\sqrt{d_k}$ keeps scores $O(1)$ so the softmax does not saturate (vanishing gradients, cf. Section 7.6).

Course Paper Question: End-Term, self-attention by hand

$$X = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 2 & 0 & 1 \end{bmatrix}, \quad W_Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad W_K = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad W_V = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad d_k = 2.$$

Projections. $\mathbf{q}_1 = [1, 2]$; K rows $[1, 3], [1, 1], [3, 2]$; V rows $[3, 2], [1, 2], [2, 1]$.

Scores. $\mathbf{q}_1 \cdot \mathbf{k}_j = 7, 3, 7$: lowest for **token 2**.

Weights. $\text{softmax}([7, 3, 7]/\sqrt{2}) = [0.486, 0.029, 0.486]$: $\alpha_{13} = \mathbf{0.486}$.

Output. $\mathbf{z}_1 = 0.486[3, 2] + 0.029[1, 2] + 0.486[2, 1] = [2.457, 1.514]$; sum **3.97**.

Shortcut. Equal scores for tokens 1 and 3 $\Rightarrow$ equal weights; only one exponential ratio $e^{-4/\sqrt{2}}$ is needed.

21.2 Multi-Head Attention

Run h heads with separate $W_Q^i, W_K^i, W_V^i \in \mathbb{R}^{d_{\text{model}} \times d_k}$ (usually $d_k = d_{\text{model}}/h$), concatenate outputs, project with $W_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$. Each head can specialise (syntax, coreference, position).

Course Paper Question: End-Term, parameter counts

(a) $d_{\text{model}} = 512$, $h = 8$, $d_k = 64$, no biases: $3 \cdot 8 \cdot 512 \cdot 64 + 512^2 = 786,432 + 262,144 = \mathbf{1,048,576}$ ($= 4d_{\text{model}}^2$). (b) Q, K, V maps $\mathbb{R}^D \rightarrow \mathbb{R}^D$ with biases: $\mathbf{3D}^2 + \mathbf{3D}$ — independent of N . (c) Attention weights for $N = 100$: $N^2 = \mathbf{10,000}$ (independent of D). (d) A fully connected layer mapping all DN inputs to all DN outputs: $\mathbf{D}^2\mathbf{N}^2 + \mathbf{DN}$ — why attention scales: its parameters do not grow with sequence length.

21.3 The Transformer Block

Encoder layer: multi-head self-attention $\rightarrow$ add & layer-norm $\rightarrow$ position-wise FFN, $\text{FFN}(\mathbf{z}) = \max(0, \mathbf{z}W_1 + \mathbf{b}_1)W_2 + \mathbf{b}_2$ (same for every position) $\rightarrow$ add & layer-norm. **Decoder layer:** *masked* self-attention (scores for future positions set to $-\infty$ before softmax, so $\alpha = 0$: no peeking), then *cross-attention* (queries from the decoder, keys and values from the encoder output), then FFN. **Layer normalisation** normalises across the features of one token (unlike batch norm across the batch), so it works for any batch size and sequence length. Computation is parallel across positions; cost $O(T^2d)$ per layer.

21.4 Positional Encoding

Self-attention is permutation-invariant: shuffling the tokens shuffles the outputs identically. Order is injected by adding

$$PE(pos, 2i) = \sin \frac{pos}{10000^{2i/d_{\text{model}}}}, \quad PE(pos, 2i+1) = \cos \frac{pos}{10000^{2i/d_{\text{model}}}}.$$

E.g. $pos = 1$, $d_{\text{model}} = 4$: $[\sin 1, \cos 1, \sin 0.01, \cos 0.01] = [0.841, 0.540, 0.010, 1.000]$. Each pair of dimensions is a clock of different frequency; relative offsets are linear (rotation) in this encoding.

21.5 The Architecture at a Glance

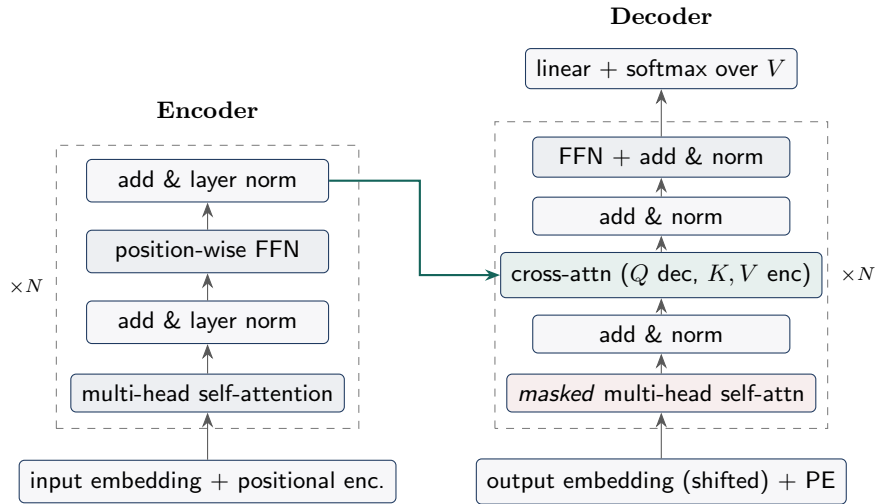


Figure 21.1: Encoder-decoder Transformer. Every sub-layer is wrapped as $\text{LayerNorm}(\mathbf{x} + \text{Sublayer}(\mathbf{x}))$. Base model: $N = 6$, $d_{\text{model}} = 512$, $h = 8$, $d_{\text{ff}} = 2048$.

21.6 From Tokens to Vectors

1. **Tokenisation:** text $\rightarrow$ tokens (words or sub-words, e.g. “unhappiness” $\rightarrow$ “un”, “happi”, “ness”), so rare words are still representable. A $\langle \text{eos} \rangle$ (and in the decoder a $\langle \text{GO} \rangle$ /start) token is added.
2. **Embedding:** a learned table $E \in \mathbb{R}^{|V| \times d_{\text{model}}}$ — one row per token (the same idea as W_{context} in word2vec, Chapter 15).
3. **Positional encoding** added element-wise (next sections).

21.7 Notation Bridge: Row vs Column Convention

The course slides stack tokens as *columns*: $Q = W_Q H$, $K = W_K H$, $V = W_V H$ with $H = [\mathbf{h}_1, \dots, \mathbf{h}_T]$ and

$$Z = \text{softmax} \left(\frac{Q^\top K}{\sqrt{d_k}} \right) V^\top \quad (\text{course, column form}),$$

whereas many papers (and the End-Term) stack tokens as *rows*: $Q = X W_Q$, $Z = \text{softmax}(Q K^\top / \sqrt{d_k}) V$. They are transposes of each other; the (i, j) score is always $\mathbf{q}_i \cdot \mathbf{k}_j$ and the softmax is always over j (the keys) for a fixed query i . **In an exam, first identify which index is the token index.**

21.8 Walkthrough: One Encoder Layer by Hand

Worked Example 21.1: A complete encoder layer on two tokens, Level 4

Problem. Two tokens, $d_{\text{model}} = 3$, embeddings (already including positions) $\mathbf{x}_1 = [1, 0, 1]$, $\mathbf{x}_2 = [0, 2, 0]$. One head with $W_Q = W_K = W_V = I_3$ ($d_k = 3$). FFN: $\mathbf{h} \mapsto \text{ReLU}(\mathbf{h}W_1)W_2$ with $W_1 =$

$$\begin{bmatrix} 1 & 0 & 1 & -1 \\ 0 & 1 & -1 & 0 \\ 1 & -1 & 0 & 1 \end{bmatrix}, W_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & -1 \end{bmatrix}, \text{ no biases. LayerNorm with } \gamma = 1, \beta = 0, \epsilon \approx 0. \text{ Compute the}$$

layer output.

Step 1: self-attention. Scores $\mathbf{x}_i \cdot \mathbf{x}_j / \sqrt{3}$: $\begin{bmatrix} 2 & 0 \\ 0 & 4 \end{bmatrix} / \sqrt{3} = \begin{bmatrix} 1.155 & 0 \\ 0 & 2.309 \end{bmatrix}$. Row softmax: $\alpha_1 = [0.760, 0.240]$, $\alpha_2 = [0.090, 0.910]$. $\mathbf{z}_1 = 0.760\mathbf{x}_1 + 0.240\mathbf{x}_2 = [0.760, 0.479, 0.760]$; $\mathbf{z}_2 = [0.090, 1.819, 0.090]$.

Step 2: add & norm. $\mathbf{r}_1 = \mathbf{x}_1 + \mathbf{z}_1 = [1.760, 0.479, 1.760]$: mean 1.333, variance 0.365 $\Rightarrow$ LN = $[0.707, -1.414, 0.707]$. $\mathbf{r}_2 = [0.090, 3.819, 0.090] \Rightarrow [-0.707, 1.414, -0.707]$.

Step 3: FFN (same weights for each position). Token 1: $\mathbf{h}W_1 = [1.414, -2.121, 2.121, 0]$, $\text{ReLU} \rightarrow [1.414, 0, 2.121, 0]$, $\times W_2 = [1.414, 0, 2.121]$. Token 2: $\text{ReLU}([-1.414, 2.121, -2.121, 0]) = [0, 2.121, 0, 0]$, $\times W_2 = [0, 2.121, 0]$.

Step 4: add & norm. Token 1: $[0.707, -1.414, 0.707] + [1.414, 0, 2.121] = [2.121, -1.414, 2.828] \Rightarrow [0.508, -1.397, 0.889]$. Token 2: $[-0.707, 3.536, -0.707] \Rightarrow [-0.707, 1.414, -0.707]$.

Interpretation. Each output row mixes information from the other token (step 1), keeps its own identity through the residual (step 2), and is transformed independently by the FFN (step 3). Every row is normalised to mean 0, variance 1. Shapes never change: $T \times d_{\text{model}}$ in, $T \times d_{\text{model}}$ out — which is what lets us stack N layers.

Common Trap: LayerNorm is per token, across features

LayerNorm computes mean/variance over the d_{model} features of *one* token (compare batch norm, Chapter 13, which normalises one feature across the batch). Consequence: it works with batch size 1 and variable lengths. Curiosity: with only 2 features, LayerNorm always outputs ± 1 — tiny toy examples can mislead!

21.9 Masked Self-Attention: Walkthrough

In the decoder, position i may attend only to positions $j \leq i$. Add $-\infty$ above the diagonal before the softmax.

Worked Example 21.2: Applying the causal mask, Level 2

Problem. Raw decoder scores $S = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 2 & 2 \end{bmatrix}$ (already scaled). Apply the mask and softmax.

Solution. Masked $S = \begin{bmatrix} 2 & -\infty & -\infty \\ 1 & 3 & -\infty \\ 0 & 2 & 2 \end{bmatrix} \Rightarrow A = \begin{bmatrix} 1 & 0 & 0 \\ 0.119 & 0.881 & 0 \\ 0.063 & 0.468 & 0.468 \end{bmatrix}$.

Interpretation. Row 1 (the $\langle \text{GO} \rangle$ position) can only look at itself. Because the mask is applied inside one matrix operation, all T positions are still trained *in parallel* with teacher forcing — the decisive speed advantage over RNN decoders.

21.10 Cross-Attention and Shapes

Queries come from the decoder (T' positions), keys and values from the final encoder output (T positions):

$$\underbrace{Q}_{T' \times d_k}, \underbrace{K, V}_{T \times d_k} \Rightarrow \underbrace{QK^\top}_{T' \times T} \Rightarrow \underbrace{Z}_{T' \times d_v}.$$

E.g. $T = 6$ source tokens, $T' = 4$ target tokens: a 4×6 attention matrix — exactly the RNN-attention alignment of Chapter 20, now computed for all target positions at once. (No causal mask is needed here: the whole source is known.)

21.11 Multi-Head Attention: Shape Walkthrough

$T = 10$, $d_{\text{model}} = 512$, $h = 8$, $d_k = d_v = 64$:

Quantity	Shape	Note
X	10×512	input
$Q_i = XW_Q^i$ (each head)	10×64	$W_Q^i \in \mathbb{R}^{512 \times 64}$
$Q_i K_i^\top / 8$	10×10	$\sqrt{64} = 8$
$Z_i = \text{softmax}(\cdot) V_i$	10×64	per head
$\text{Concat}(Z_1, \dots, Z_8)$	10×512	
$\cdot W_O$	10×512	$W_O \in \mathbb{R}^{512 \times 512}$

Splitting 512 into 8×64 keeps the cost about equal to one full-width head while allowing 8 different attention patterns — the analogue of multiple filters in a CNN layer (course notes).

21.12 Counting a Full Transformer

Worked Example 21.3: Parameters of the base model, Level 3

Problem. $d_{\text{model}} = 512$, $d_{ff} = 2048$, $h = 8$, all biases included. Count one encoder layer and one decoder layer.

Solution. MHA: $4(512^2 + 512) = 1,050,624$. FFN: $512 \cdot 2048 + 2048 + 2048 \cdot 512 + 512 = 2,099,712$. LayerNorm: $2 \cdot 512 = 1024$ each.

Encoder layer = $1,050,624 + 2,099,712 + 2(1024) = \mathbf{3,152,384}$.

Decoder layer (two MHAs, three LNs) = $2(1,050,624) + 2,099,712 + 3(1024) = \mathbf{4,204,032}$.

6 + 6 layers: 44.1M; with a shared 37,000-token embedding (18.9M): $\approx 63\text{M}$ (the paper reports $\approx 65\text{M}$).

Insight. The FFN holds **two-thirds** of each encoder layer’s parameters; attention mixes, the FFN does most of the per-token computation.

21.13 Why Scale by $\sqrt{d_k}$: in Numbers

With $d_k = 64$ and unit-variance components, $\mathbf{q} \cdot \mathbf{k}$ has standard deviation 8. Scores like $[8, 0, -8]$ give softmax $[0.9997, 0.0003, 0.0000]$ — effectively one-hot, with gradients $\hat{y}_i(1 - \hat{y}_i) \approx 0$. Scaled scores $[1, 0, -1]$ give $[0.665, 0.245, 0.090]$ — a soft, trainable distribution.

21.14 Positional Encoding: Properties

- Values for $\text{pos} = 2$, $d_{\text{model}} = 4$: $[\sin 2, \cos 2, \sin 0.02, \cos 0.02] = [0.909, -0.416, 0.020, 1.000]$. Low dimensions oscillate fast (fine position), high dimensions slowly (coarse position) — like the hands of a clock.
- Relative position is linear:** for each frequency ω ,
$$\begin{bmatrix} \sin \omega(p+k) \\ \cos \omega(p+k) \end{bmatrix} = \begin{bmatrix} \cos \omega k & \sin \omega k \\ -\sin \omega k & \cos \omega k \end{bmatrix} \begin{bmatrix} \sin \omega p \\ \cos \omega p \end{bmatrix},$$
 a rotation depending only on the offset k . Attention can therefore learn “look 2 tokens back”.

- Deterministic, no parameters, defined for any length; learned position embeddings are the common alternative (BERT, GPT).

21.15 Transformer vs RNN vs CNN

Layer type	Cost per layer	Sequential steps	Longest path between tokens
Self-attention	$O(n^2d)$	$O(1)$	$O(1)$
Recurrent	$O(nd^2)$	$O(n)$	$O(n)$
Convolution (kernel k)	$O(knd^2)$	$O(1)$	$O(\log_k n)$

Course Paper Question: Course PYQ, parallelism

Statement I: Transformers allow parallel computation across tokens during training; RNNs process tokens sequentially. Statement II: Transformers significantly reduce training time for long sequences. **Both true; I explains II** ($\mathbf{s}_t$ needs $\mathbf{s}_{t-1}$ in an RNN, while $Z = \text{softmax}(QK^\top/\sqrt{d_k})V$ is one batched matrix product). Caveat: attention memory grows as n^2 .

21.16 Training and Inference

Training: the decoder input is the target shifted right ($\langle \text{GO} \rangle, y_1, \dots, y_{T'-1}$); with the causal mask, all positions predict their next token in one pass; loss = sum of cross-entropies, often with **label smoothing** $\varepsilon = 0.1$ (Chapter 12). Optimiser: Adam ($\beta_2 = 0.98$) with the **warm-up schedule**

$$\eta_t = d_{\text{model}}^{-0.5} \min(t^{-0.5}, t \cdot w^{-1.5}), \quad w = 4000,$$

linear increase for w steps, then $\propto 1/\sqrt{t}$. With $d_{\text{model}} = 512$: $\eta_{1000} = 0.000175$, peak $\eta_{4000} = 0.000699$, $\eta_{16000} = 0.000349$ (warm-up from Chapter 9 in action).

Inference (autoregressive): start with $\langle \text{GO} \rangle$, run the decoder, take (greedy/beam) the next token, append it, repeat until $\langle \text{eos} \rangle$ or a maximum length T_1 (course notes). The encoder runs *once*; the decoder runs once per generated token.

21.17 The Transformer Family

Beyond the Core Syllabus: Encoder-only, decoder-only, and vision

Encoder-only (BERT): bidirectional self-attention, trained by masking random input tokens and predicting them — for understanding tasks. **Decoder-only** (GPT): causal self-attention, next-token prediction — for generation; no cross-attention. **Encoder-decoder** (original, T5): translation/summarisation. **Vision Transformer:** a $224 \times 224 \times 3$ image cut into 16×16 patches gives $14 \times 14 = 196$ tokens of dimension $16 \cdot 16 \cdot 3 = 768$ — the same “grid of regions as a sequence” idea as attention over CNN feature maps (Chapter 20).

21.18 Scenario Bank

S1. You remove the residual connections from a 12-layer Transformer. Expected effect?

Answer: Much harder optimisation (gradients must pass through every sub-layer; cf. ResNet) and loss of the token’s own information after attention mixing.

S2. Sequence length doubles from 512 to 1024. Parameters, attention memory?

Answer: Parameters unchanged (except learned positional tables); attention score memory $\times 4$.

S3. A decoder is trained *without* the causal mask. Training loss becomes tiny but generation is garbage. Why?

Answer: During training each position could read the answer (the next token) directly; at inference those tokens do not exist.

S4. All tokens in a sentence are identical words (“the the the”). Without positional encoding, what are the self-attention outputs?

Answer: All identical: uniform attention over identical values.

S5. Can the FFN mix information across positions?

Answer: No — it is applied to each position independently; only attention mixes positions.

S6. Heads $h = 16$ with $d_{\text{model}} = 512$: d_k and MHA parameters (no bias)?

Answer: $d_k = 32$; still $4 \cdot 512^2 = 1,048,576$ — head count does not change the total if $hd_k = d_{\text{model}}$.

21.19 Practice Problems

1. **[FOUNDATION]** Scores $[3, 3, 0]$ for query 1 (already scaled). Attention weights?

Answer: $[0.488, 0.488, 0.024]$.

2. **[INTERMEDIATE]** Causal mask on a 4-token decoder: how many attention weights are forced to zero?

Answer: $\binom{4}{2} = 6$.

3. **[EXAM]** $d_{\text{model}} = 256$, $d_{ff} = 1024$: FFN parameters with biases?

Answer: $256 \cdot 1024 + 1024 + 1024 \cdot 256 + 256 = 525,568$.

4. **[EXAM]** Encoder output for 20 source tokens, decoder at 7 target tokens: shape of cross-attention weight matrix?

Answer: 7×20 .

5. **[ADVANCED]** Show that without positional encodings, permuting the input rows of X permutes the rows of self-attention’s output identically (permutation equivariance).

Hint: PX gives scores $P(QK^\top)P^\top$; row-softmax commutes with the permutation.

6. **[CHALLENGE]** Explain why scaling by $\sqrt{d_k}$ becomes more important as d_k grows, using $\text{Var}(\mathbf{q} \cdot \mathbf{k}) = d_k$.

21.20 Concept Gauntlet

G1. Without positional encoding, do “dog bites man” and “man bites dog” give the same set of outputs?

Answer: Yes — permuted identically; order is invisible.

G2. Why mask in the decoder but not the encoder?

Answer: At generation time future tokens do not exist; the encoder sees the whole source.

G3. Does self-attention cost grow with N ?

Answer: Parameters no; computation and memory yes (N^2 scores).

G4. What if we forget the $\sqrt{d_k}$ with $d_k = 512$?

Answer: Scores $\sim \pm 22$: softmax becomes nearly one-hot and gradients vanish.

Final Master Reference (Weeks 8–12)

22.1 Formula Sheet

Formula	Meaning	Note
$\ln \frac{\text{count}(w,c)N}{\text{count}(w)\text{count}(c)}; \max(\cdot, 0)$	PMI; PPMI	natural log unless told
$W_{\text{word}} = U_k \Sigma_k, W_{\text{context}} = V_k$	SVD embeddings	rank-1 storage $m + n + 1$
$-\log \sigma(\mathbf{v}_w^\top \mathbf{u}_c) - \sum \log \sigma(-\mathbf{v}_w^\top \mathbf{u}_r)$	negative sampling	$p(r) \propto \text{count}^{3/4}$
$\frac{W-F+2P}{S} + 1; K(F^2 D + 1)$	conv output; params	same size: $P = \frac{F-1}{2}$
$\partial \mathcal{L} / \partial w = \sum_{\text{positions}} \delta \cdot \text{input}$	shared-weight gradient	CNN and RNN alike
$\mathbf{x} + \epsilon \text{sign } \nabla_{\mathbf{x}} \mathcal{L}$	FGSM adversarial	shift $\epsilon \ \mathbf{w}\ _1$
$\mathcal{G} = A^\top A$	style (Gram)	position-free
$\mathbf{s}_t = \sigma(U\mathbf{x}_t + W\mathbf{s}_{t-1} + \mathbf{b})$	RNN	shared over time
$\prod_j \text{diag}(\sigma'(\mathbf{a}_j))W$	BPTT Jacobian	$(\gamma\lambda)^k$ vanishes/explodes
$\mathbf{s}_t = \mathbf{f} \odot \mathbf{s}_{t-1} + \mathbf{i} \odot \tilde{\mathbf{s}}_t, \mathbf{h}_t = \mathbf{o} \odot \tanh \mathbf{s}_t$	LSTM	4 weight sets
$\mathbf{s}_t = (1 - \mathbf{i}) \odot \mathbf{s}_{t-1} + \mathbf{i} \odot \tilde{\mathbf{s}}_t$	GRU	3 weight sets
$V^\top \tanh(U\mathbf{s}_{t-1} + W\mathbf{h}_j); \mathbf{c}_t = \sum \alpha_{jt} \mathbf{h}_j$	additive attention	softmax over j
$\text{softmax}(QK^\top / \sqrt{d_k})V$	scaled dot-product	$\text{Var}(\mathbf{q} \cdot \mathbf{k}) = d_k$
$3hd_{\text{model}}d_k + d_{\text{model}}^2$	MHA params (no bias)	$= 4d_{\text{model}}^2$ if $hd_k = d_{\text{model}}$
$\sin / \cos(\text{pos}/10000^{2i/d})$	positional encoding	even/odd dims

22.2 Recognition Rules (Weeks 8–12)

- Counts given $\Rightarrow$ PMI; negative $\Rightarrow$ PPMI 0.
- One-hot times matrix $\Rightarrow$ column lookup.
- “Same size output” $\Rightarrow P = (F - 1)/2$; odd pooling $\Rightarrow$ floor.
- Filter weights over zero regions or inactive ReLUs $\Rightarrow$ zero gradient.
- Feature map $H \times W \times C$ as attention states $\Rightarrow HW$ states of dimension C .
- $\partial \mathbf{s}_T / \partial \mathbf{s}_k \Rightarrow T - k$ Jacobian factors; $\gamma\lambda \leq 1$.
- Long-range memory $\Rightarrow$ LSTM/GRU (additive, gated path); explosion $\Rightarrow$ clipping.
- Long source sentence degrades $\Rightarrow$ fixed-vector bottleneck $\Rightarrow$ attention.
- Transformer parameter questions $\Rightarrow$ independent of sequence length; attention weights $= N^2$.
- Equal dot-product scores $\Rightarrow$ equal attention weights — compute one exponential.